



UNIVERSIDAD REY JUAN CARLOS

TRABAJO DE FIN DE GRADO

JARVIS RAG:

**Sistema Local de Consulta Documental con
Generación Aumentada mediante Recuperación**

Autor:

Ángel María Ballesteros Limones

Tutor:

Jesús María González Barahona

Grado en Ingeniería de Tecnología de Telecomunicaciones

Curso 2025/2026

Licencia de distribución

Esta memoria se distribuye bajo una licencia **Creative Commons Reconocimiento-CompartirIgual 4.0 Internacional (CC BY-SA 4.0)**.

Usted es libre de:

- **Compartir** — copiar y redistribuir el material en cualquier medio o formato.
- **Adaptar** — remezclar, transformar y construir a partir del material para cualquier finalidad, incluso comercial.

Bajo los siguientes términos:

- **Reconocimiento** — debe reconocer adecuadamente la autoría, proporcionar un enlace a la licencia e indicar si se han realizado cambios.
- **CompartirIgual** — si remezcla, transforma o crea a partir del material, deberá difundir sus contribuciones bajo la misma licencia que el original.

El texto legal completo de la licencia está disponible en <https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

El **software** desarrollado y descrito en esta memoria se publica de forma independiente como código abierto bajo la licencia **MIT**, cuyos términos se incluyen en el fichero **LICENSE** del repositorio del proyecto.

Copyright © 2026 Ángel María Ballesteros Limones.

Dedicatoria

*A mi familia y a mis amigos,
por su apoyo incondicional.*

*En especial a mi madre y a mi abuelo,
por confiar en mi incluso
cuando yo no lo hacía.*

Agradecimientos

Este proyecto no habría sido posible sin el apoyo y la colaboración de numerosas personas e instituciones que han contribuido, directa o indirectamente, a su realización.

En primer lugar, quiero agradecer a mi tutor de TFG por su orientación académica y su disponibilidad a lo largo de todo el proceso de desarrollo, siendo nuestros horarios tan diferentes. Sus revisiones y conocimiento han sido determinantes para conseguir dar forma a la idea que tenía en mente.

Agradezco también a la empresa Europavia por facilitar el entorno real de despliegue, con una necesidad existente a nivel corporativo que ha servido como banco de pruebas del sistema. La oportunidad de desarrollar un proyecto con impacto práctico inmediato en un contexto profesional ha enriquecido enormemente la experiencia. Sobre todo Nacho y CIVEX por ser siempre de ayuda y facilitar cualquier desarrollo que considerara.

A mis compañeros de carrera, sobre todo a los de Rugby y a aquellos del Erasmus, que me enseñaron el verdadero sentido de la universidad. El camino se hace más llevadero cuando se recorre en buena compañía.

A mi familia, por su paciencia infinita durante los meses más intensos de exámenes. Su apoyo incondicional ha sido el pilar fundamental sobre el que se sustenta todo lo demás.

Por último, quiero reconocer a la comunidad de código abierto, cuyos proyectos — Ollama, Qdrant, FastAPI, OpenWebUI, Playwright, PaddleOCR, LiteLLM, entre muchos otros — han hecho posible construir un sistema de esta envergadura sin dependencias comerciales. Este TFG es, en gran medida, un tributo a la filosofía del software libre.

Ángel María Ballesteros Limones
Madrid, 2026

Resumen

Este Trabajo de Fin de Grado presenta el diseño, desarrollo y despliegue de un sistema de Generación Aumentada mediante Recuperación (*Retrieval-Augmented Generation*, RAG) de ámbito corporativo, ejecutado íntegramente en infraestructura local (*on-premise*) e integrando herramientas y modelos de código abierto con desarrollo propio.

El sistema permite consultar en lenguaje natural los documentos internos de una organización, páginas web y fuentes legislativas oficiales, respondiendo siempre con las fuentes citadas. Una pila de microservicios contenerizados con Docker integra modelos de lenguaje locales (Ollama), una base de datos vectorial (Qdrant) y un agente encaminador denominado JARVIS que detecta la intención de cada consulta y la delega al modo de operación más adecuado. Entre las fuentes integradas destacan Microsoft SharePoint —principal entorno de validación práctica—, páginas web extraídas mediante scraping, documentos escaneados tratados con OCR acelerado por GPU y el Boletín Oficial del Estado.

El sistema ha sido desplegado y validado en un entorno real con resultados satisfactorios: tiempos de respuesta inferiores a 3.5 segundos y una reducción muy significativa de las alucinaciones en modo RAG gracias a una estrategia de recuperación documental orientada a la trazabilidad.

Palabras clave: RAG, LLM, inteligencia artificial, embeddings, base de datos vectorial, microservicios, Docker, Ollama, Qdrant, on-premise, código abierto.

Abstract

This Bachelor’s Thesis presents the design, development and deployment of a corporate Retrieval-Augmented Generation (RAG) system, executed entirely on local infrastructure (on-premise) and built by integrating open-source tools and models together with original development.

The system allows the internal documents of an organization, web pages and official legislative sources to be queried in natural language, always answering with cited sources. A stack of Docker-containerized microservices integrates locally executed language models (Ollama), a vector database (Qdrant) and a routing agent called JARVIS, which detects the intent of each query and delegates it to the most suitable operating mode. Integrated sources include Microsoft SharePoint —the main real-world validation environment—, web pages processed through scraping, scanned documents handled with GPU-accelerated OCR, and the Spanish Official State Gazette (BOE).

The system has been deployed and validated in a real environment with satisfactory results: response times under 3.5 seconds and a very significant reduction of hallucinations in RAG mode thanks to a document retrieval strategy focused on traceability.

Keywords: RAG, LLM, artificial intelligence, embeddings, vector database, micro-services, Docker, Ollama, Qdrant, on-premise, open source.

Índice general

Agradecimientos	iii
Resumen	iv
Abstract	v
1. Introducción	1
1.1. Motivación	1
1.1.1. Problemática del Shadow AI en entornos corporativos	2
1.2. Solución propuesta y enfoque del trabajo	3
1.3. Código y recursos del proyecto	4
1.4. Objetivos del proyecto	5
1.5. Alcance y limitaciones	6
1.5.1. Alcance	6
1.5.2. Limitaciones	7
1.6. Contribuciones principales	8
1.7. Estructura de la memoria	8
2. Fundamentos de la solución propuesta	10
2.1. Grandes Modelos de Lenguaje (LLMs)	10
2.1.1. Propietarios vs. pesos abiertos	11
2.2. Por qué un LLM por sí solo no resuelve este problema	11
2.3. Generación Aumentada mediante Recuperación (RAG)	12
2.3.1. Mejoras prácticas: búsqueda híbrida y memoria acotada	12
2.4. Modelos de Embeddings	13
2.5. Bases de datos vectoriales	13
2.6. Cuantización y ejecución local	14
2.7. Ajuste fino eficiente con LoRA	14
2.8. Model Context Protocol (MCP)	15

3. Tecnologías utilizadas y criterio de elección	16
3.1. Python como lenguaje de integración principal	16
3.2. OpenWebUI: interfaz conversacional reutilizada	17
3.2.1. Arquitectura interna de OpenWebUI	17
3.2.2. Por qué OpenWebUI	18
3.3. Ollama: ejecución local de modelos	19
3.3.1. Modelos desplegados	19
3.3.2. Stack de modelos en producción	19
3.3.3. Query Processor: enrutamiento inteligente y expansión de con- sultas	20
3.4. Qdrant: base de datos vectorial	21
3.5. FastAPI: backend del sistema	21
3.6. LiteLLM: proxy unificado de modelos	22
3.7. Docker y orquestación de contenedores	22
3.8. Herramientas de procesamiento documental y web	22
3.8.1. PaddleOCR: reconocimiento óptico de caracteres	23
3.8.2. Playwright: automatización de navegadores	23
3.8.3. Trafilatura: extracción de contenido web	23
3.8.4. SentenceTransformers y reranker: recuperación	23
3.9. Stack de monitorización	23
3.9.1. Prometheus	24
3.9.2. Grafana	24
3.9.3. NVIDIA DCGM Exporter	24
3.9.4. Loki y Promtail	24
3.10. Otras tecnologías del ecosistema	24
4. Sistemas relacionados	26
4.1. Asistentes comerciales basados en LLM	26
4.1.1. ChatGPT y GPT Enterprise (OpenAI)	26
4.1.2. Microsoft Copilot para Microsoft 365	27
4.1.3. Google Gemini y Vertex AI	27
4.1.4. Amazon Bedrock	27
4.2. Herramientas abiertas	27
4.2.1. PrivateGPT	27
4.2.2. AnythingLLM	28
4.2.3. Danswer / Onyx	28
4.2.4. OpenClaw y la aceleración reciente del ecosistema	28

4.2.5. NotebookLM y asistentes centrados en conocimiento aportado por el usuario	28
4.2.6. Frameworks: LangChain y LlamaIndex	29
4.3. Análisis comparativo	29
4.4. Posicionamiento del sistema desarrollado	29
5. Descripción Funcional del Sistema	31
5.1. Visión General	31
5.2. Acceso al Sistema e Inicio de Sesión	33
5.3. Interfaz de Usuario	33
5.3.1. Panel Principal de Chat	33
5.3.2. Panel Lateral de Conversaciones	34
5.3.3. Carga de Archivos Adjuntos	34
5.4. Modos de Operación	34
5.4.1. Consulta de Documentos Internos (RAG)	34
5.4.2. Búsqueda en Internet	35
5.4.3. Análisis de Páginas Web (Scraping)	35
5.4.4. Análisis de Imágenes (Visión/OCR)	36
5.4.5. Consulta Legislativa del BOE	36
5.4.6. Listar Documentos Disponibles	37
5.4.7. Estado del Sistema	37
5.4.8. Ayuda y Capacidades	37
5.4.9. Conversación General	37
5.5. Escenarios de Uso Completos	38
5.5.1. Escenario 1: Consulta de un procedimiento corporativo	38
5.5.2. Escenario 2: Análisis de una normativa del BOE	38
5.5.3. Escenario 3: Análisis de una web tecnológica	38
5.6. Panel de Administración y Monitorización	39
5.6.1. Dashboards de Grafana	39
5.6.2. Administración de la Base de Datos	39
5.6.3. Gestión de Modelos	39
6. Diseño e Implementación	40
6.1. Filosofía de Diseño: Necesidad, Solución, Iteración	40
6.2. Arquitectura General del Sistema	43
6.2.1. Flujo de Datos End-to-End	45
6.3. Desarrollo Propio vs. Herramientas Externas	46
6.4. Pipeline de OpenWebUI	47

6.5.	Agente Encaminador JARVIS: Inteligencia Central	47
6.6.	Backend RAG: Motor de Búsqueda Semántica	50
6.6.1.	Búsqueda Híbrida	51
6.7.	Extracción Web, OCR e integraciones	51
6.8.	Integración con el Boletín Oficial del Estado (BOE)	52
6.9.	Seguridad Perimetral y Autenticación	52
6.10.	Observabilidad, caché y almacenamiento	53
6.11.	Agente SQL: Consultas en Lenguaje Natural sobre Datos Estructurados	54
6.12.	Fine-Tuning con LoRA	55
7.	Proceso de desarrollo del proyecto	56
7.1.	Organización del Código	57
7.2.	Los Sprints en Detalle	57
7.2.1.	Sprint 1 (octubre de 2025): Infraestructura base	58
7.2.2.	Sprint 2 (noviembre de 2025): Backend y pipeline RAG	58
7.2.3.	Sprint 3 (diciembre de 2025): SharePoint y OCR	59
7.2.4.	Sprint 4 (enero de 2026): Agente JARVIS, scraping y búsqueda web	60
7.2.5.	Sprint 5 (febrero de 2026): BOE, <i>fine-tuning</i> y observabilidad	60
7.2.6.	Sprint 6 (marzo-abril de 2026): Agente SQL, seguridad y v2.2.0	61
7.2.7.	Sprint 7 (mayo de 2026): QueryProcessor y v2.3.0	62
8.	Resultados y Evaluación	63
8.1.	Métricas de Rendimiento	63
8.1.1.	Análisis de Latencia	64
8.1.2.	Gestión de VRAM	65
8.2.	Evaluación Funcional	65
8.2.1.	Query Processor: intent detection, smart routing y query ex- pansion	65
8.2.2.	Validación final con documentación del TFG	66
8.2.3.	Consulta RAG sobre Documentos Corporativos	70
8.2.4.	Scraping de URL	71
8.2.5.	Búsqueda Legal en el BOE	71
8.2.6.	Análisis de Imagen	71
8.2.7.	Consultas NL→SQL con el Agente SQL	71
8.2.8.	Encaminamiento Inteligente	72
8.3.	Mitigación de Alucinaciones	72
8.4.	Impacto de la Caché	73

9. Conclusiones y Trabajo Futuro	75
9.1. Resumen General	75
9.2. Consecución de Objetivos	75
9.2.1. Funcionalidades logradas	77
9.3. Tecnologías y Entorno Empleados	77
9.3.1. Stack Tecnológico	77
9.3.2. Entorno Hardware	79
9.3.3. Herramientas de Trabajo y Control de Versiones	79
9.4. Comparación con Otros Sistemas	79
9.5. Esfuerzo Realizado y Presupuesto	80
9.5.1. Dedicación Total	80
9.5.2. Planificación Temporal por Sprints	81
9.5.3. Estimación de Presupuesto	82
9.5.4. Gestión de Riesgos	82
9.6. Conocimientos del Grado Aplicados	83
9.7. Conocimientos Adquiridos Fuera del Grado y Lecciones Aprendidas	84
9.7.1. Conocimientos Adquiridos de Forma Autónoma	84
9.7.2. Lecciones Aprendidas	85
9.7.3. Uso de herramientas de IA: una nueva forma de trabajar	85
9.8. Trabajo Futuro y Líneas de Expansión	86
9.9. Impacto y Transferencia	87
9.10. Código y Sitio Web del Proyecto	88
 A. Detalle de implementación del Pipeline JARVIS	 89
A.1. Contrato de la Clase Pipeline	89
A.2. El Método <code>pipe()</code> : Corazón del Sistema	90
A.3. Sistema Valves: Configuración Dinámica	91
A.4. Sticky Mode: Persistencia de Contexto	92
A.5. Memoria de Sesión por Usuario	92
A.6. Control de Acceso Departamental	92
A.7. Enrutamiento por reglas frente a un router basado en LLM	93
 B. Detalle del Backend RAG y el procesamiento documental	 95
B.1. Pipeline de Ingesta Documental	95
B.2. Búsqueda Híbrida: modelos densos, dispersos y reranker	96
B.3. Construcción del Prompt Contextual	97
B.4. Extracción Web: Scraping y Búsqueda en Internet	97
B.4.1. Motor de Renderizado con Playwright	97

B.4.2. Cadena de Extracción con Fallback	97
B.4.3. Búsqueda en Internet	98
B.5. Reconocimiento Óptico de Caracteres (OCR)	98
B.6. Agente SQL: mecanismos de seguridad	98
C. Integraciones, seguridad y observabilidad en detalle	100
C.1. Integración con Microsoft SharePoint	100
C.2. Resolución Semántica de Leyes (BOE)	100
C.3. Validación JWT en el Backend	102
C.4. Métricas, telemetría GPU y dashboards	103
D. Fine-tuning con LoRA: procedimiento	105
D.1. Generación del Dataset de Entrenamiento	105
D.2. Entrenamiento con Low-Rank Adaptation	105
D.3. Resultado y Evaluación	105
E. Evolución de modelos y diagrama tecnológico	107
E.1. Evolución del modelo principal: trayectoria de cuatro etapas	107
E.2. Diagrama de flujo de tecnologías usadas	108
F. Contenido del repositorio entregado	110
F.1. Estructura del archivo	110
F.2. Elementos excluidos y su justificación	111
F.3. Verificación del contenido	111
Bibliografía	113

Índice de figuras

5.1. Interfaz de JARVIS (sobre OpenWebUI) en funcionamiento.	31
6.1. Estado del stack Docker en la validación final.	44
6.2. Documentación interactiva del Backend FastAPI.	45
6.3. Respuesta real del modo de listado documental.	49
6.4. Estado de las colecciones en Qdrant durante la demo final.	50
6.5. Pantalla de autenticación de OpenWebUI.	53
6.6. Dashboard principal de Grafana durante la validación final.	54
8.1. Descomposición de la latencia por componente (consulta RAG).	64
8.2. Listado real de documentos cargados en la colección de demo.	67
8.3. Respuesta RAG real sobre la colección de demostración.	68
8.4. Consulta RAG desde OpenWebUI durante la demo final.	69
8.5. Sección de demo publicada en GitHub Pages.	70
8.6. Comportamiento del sistema sin documentos disponibles.	73
9.1. Diagrama de GANTT del proyecto (octubre de 2025 – julio de 2026).	82
E.1. Diagrama general de las tecnologías utilizadas en el sistema.	109

Índice de tablas

2.1. Comparativa de bases de datos vectoriales evaluadas para el proyecto.	13
3.1. Comparativa de interfaces de usuario para LLMs locales.	18
3.2. Modelos de IA desplegados en Ollama y su función en el sistema. . .	19
3.3. Stack de modelos en producción (versión TFG y sistema empresarial).	20
4.1. Comparación del sistema JARVIS con soluciones alternativas.	29
6.1. Servicios principales de la arquitectura y su función.	43
6.2. Resumen sintético de reutilización y desarrollo dentro del TFG. . . .	46
6.3. Categorías de intención y su prioridad de evaluación.	48
8.1. Métricas de rendimiento del sistema en producción.	63
8.2. Desglose de latencia por componente para una consulta RAG típica. .	64
8.3. Consumo de VRAM por modelo y combinaciones de carga simultánea.	65
8.4. Resultados de la evaluación del encaminamiento de JARVIS.	72
9.1. Checklist de funcionalidades logradas.	77
9.2. Stack tecnológico completo del proyecto.	78
9.3. Especificaciones del entorno hardware de despliegue.	79
9.4. Distribución de horas dedicadas al proyecto.	80
9.5. Sprints del desarrollo, entregables y horas dedicadas.	81
9.6. Estimación de presupuesto para el despliegue del sistema.	82
9.7. Riesgos técnicos identificados y estrategias de mitigación.	83
E.1. Trayectoria completa de LLMs empleados en el proyecto, desde el prototipo hasta producción.	107

Capítulo 1

Introducción

1.1 Motivación

En los entornos corporativos actuales, el volumen de información disponible crece de forma constante, pero no crece de forma ordenada. La documentación relevante suele estar repartida entre repositorios como Microsoft SharePoint, PDFs internos, muchos de ellos escaneados, wikis, portales de proveedores, procedimientos técnicos y fuentes públicas como el Boletín Oficial del Estado (BOE). En la práctica, el problema habitual no es la falta de información, sino la dispersión, la heterogeneidad de formatos y la dificultad para recuperar contenido útil cuando se necesita.

Los métodos tradicionales de búsqueda por palabras clave fallan en situaciones muy comunes: cuando el usuario no conoce el término exacto, cuando la información está contenida en documentos largos, cuando existen sinónimos o acrónimos internos, o cuando el contenido está en PDFs escaneados y requiere reconocimiento óptico de caracteres (OCR, *Optical Character Recognition*). Como consecuencia, tareas rutinarias como confirmar un procedimiento, localizar un requisito normativo o recuperar una especificación se convierten en procesos lentos y propensos a errores.

En paralelo, la popularización de los grandes modelos de lenguaje (LLMs, *Large Language Models*) ha permitido interactuar con sistemas de información mediante lenguaje natural. Sin embargo, en un contexto corporativo, una respuesta fluida pero incorrecta tiene un coste elevado: puede inducir decisiones erróneas y generar confianza injustificada. El problema se agrava cuando el modelo no dispone de acceso al conocimiento interno, cuando la documentación cambia con frecuencia o cuando la consulta requiere precisión y respaldo documental.

Por este motivo, resulta especialmente relevante el enfoque de Generación Aumentada mediante Recuperación (RAG, *Retrieval-Augmented Generation*), que combina la generación de lenguaje con la recuperación de información desde una base do-

cumental. En lugar de responder de memoria, el sistema fundamenta la respuesta en documentos recuperados, mejorando la utilidad, la trazabilidad y reduciendo la probabilidad de respuestas no justificadas [1].

Este Trabajo de Fin de Grado no parte de entrenar un modelo desde cero. Sería un objetivo desproporcionado para el alcance del proyecto y, además, no resolvería por sí mismo el problema principal, que no es crear otro LLM generalista, sino poner modelos ya existentes a trabajar de forma útil, segura y controlada sobre información corporativa cambiante. Por ello, el trabajo se centra en seleccionar, integrar, adaptar y orquestar tecnologías existentes de código abierto y pesos abiertos: modelos de lenguaje ejecutados localmente, embeddings (representaciones numéricas que capturan el significado del texto), una base de datos vectorial, un backend propio en Python, mecanismos de ingesta documental y una interfaz conversacional empresarial.

Con esta motivación, el proyecto aborda el diseño, desarrollo e integración de un sistema RAG corporativo desplegado en infraestructura controlada (*on-premise*), capaz de ingerir documentos desde fuentes heterogéneas y ofrecer a los usuarios una interfaz conversacional orientada a consultas técnicas, con recuperación semántica y soporte documental.

1.1.1 Problemática del Shadow AI en entornos corporativos

La adopción de asistentes de IA comerciales en el ámbito laboral ha impulsado un fenómeno cada vez más habitual: el uso no autorizado de herramientas externas por parte de empleados sin supervisión del departamento de TI ni revisión legal o de cumplimiento. Este patrón se conoce como *Shadow AI* y puede considerarse una extensión del *Shadow IT*, con un riesgo añadido: la exposición de datos puede producirse en el mismo momento en que un usuario copia y pega información interna en un servicio externo [2].

La magnitud del fenómeno es considerable. Según el informe *Work Trend Index* de Microsoft y LinkedIn (2024), el 75 % de los trabajadores del conocimiento utiliza herramientas de IA en su trabajo, y el 78 % de ellos emplea herramientas propias no autorizadas por la organización [2]. En un análisis de Cyberhaven sobre 1,6 millones de trabajadores, se detectó que el 8,6 % de los empleados introducía datos corporativos en ChatGPT, y que el 11 % de esos datos estaba clasificado como confidencial [3]. Además, entre marzo de 2023 y marzo de 2024, la cantidad de datos corporativos introducidos en herramientas de IA se multiplicó por 4,85, y la proporción de datos sensibles casi se triplicó, pasando del 10,7 % al 27,4 % [4].

Los riesgos principales asociados a este fenómeno son:

- *Exposición de información sensible:* al introducir documentos, fragmentos de procedimientos, incidencias, datos operativos o contenido contractual en herramientas externas, la información puede salir del perímetro de seguridad corporativo. Un estudio de Komprise (2025) reveló que el 44 % de las organizaciones ha experimentado fugas de datos sensibles hacia herramientas de IA [5].
- *Riesgo regulatorio y de cumplimiento:* en sectores regulados o con datos personales o sensibles, el tratamiento de la información exige garantías sobre finalidad, acceso, ubicación y medidas de seguridad. El uso informal de herramientas externas introduce incertidumbre y eleva el riesgo de incumplimiento normativo.
- *Calidad y consistencia:* los modelos generalistas pueden producir respuestas plausibles incluso cuando carecen de base documental. En ámbitos técnicos o normativos, esto se traduce en errores silenciosos con apariencia de certeza.
- *Ausencia de gobernanza:* sin control centralizado, la organización no sabe qué herramientas se usan, con qué datos, con qué finalidad ni con qué impacto.

Este proyecto aborda directamente esta problemática mediante una alternativa interna: un sistema conversacional desplegado en la infraestructura de la organización, donde los documentos no se envían a servicios externos y donde las respuestas se apoyan en recuperación documental, reduciendo respuestas no fundamentadas y permitiendo trazabilidad hacia fuentes internas.

1.2 Solución propuesta y enfoque del trabajo

La solución propuesta consiste en construir un asistente corporativo que actúe como punto único de acceso a la información de la organización. La idea no es sustituir todos los sistemas documentales existentes, sino superponer una capa de consulta en lenguaje natural que permita a cualquier usuario localizar y comprender información dispersa sin necesidad de conocer de antemano dónde está almacenada ni en qué formato se encuentra.

Para resolver los problemas descritos en la sección anterior, el sistema combina varios bloques:

- *Modelos ya existentes ejecutados en local:* se reutilizan LLMs y modelos de embeddings disponibles públicamente, desplegados dentro de la infraestructura propia.

- *Un backend propio en Python*: se desarrolla la lógica que conecta los modelos con las fuentes documentales, aplica reglas de acceso y construye respuestas trazables.
- *Un pipeline RAG*: antes de responder, el sistema recupera fragmentos relevantes de la documentación para que la generación se apoye en evidencia.
- *Conectores e integraciones*: se incorporan mecanismos para trabajar con Share-Point, documentos escaneados, páginas web y normativa del BOE.
- *Una interfaz conversacional reutilizable*: en lugar de crear una interfaz desde cero, se aprovecha OpenWebUI y se amplía mediante un pipeline propio.

Este enfoque tiene dos ventajas. La primera es práctica: permite construir un sistema funcional en el contexto de un TFG sin depender del entrenamiento masivo de modelos. La segunda es metodológica: separa claramente qué aporta la tecnología base y qué aporta el trabajo realizado en el proyecto. En este caso, la contribución no está en crear un nuevo modelo fundacional, sino en diseñar una arquitectura útil, explicar por qué cada pieza es necesaria y desarrollar la lógica que convierte herramientas sueltas en un sistema corporativo coherente.

1.3 Código y recursos del proyecto

Todo el código fuente, la documentación técnica y los materiales para reproducir el despliegue están publicados de forma abierta. Dos recursos se referencian a lo largo de la memoria y conviene destacarlos desde el principio:

- **Repositorio de código en GitHub**¹: código de los microservicios, definición de la arquitectura Docker, configuración de los modelos locales y documentación de despliegue.
- **Página del proyecto en GitHub Pages**²: *landing page* con la descripción del sistema, capturas reales y un vídeo de demostración que recorre la aplicación en funcionamiento.

Se destacan aquí de forma explícita por dos razones: una parte importante del valor del trabajo reside en la combinación de componentes reutilizados con código propio integrador, y la página del proyecto permite ver el sistema en funcionamiento sin necesidad de desplegarlo.

¹<https://github.com/acidxlemons/TFG-RAG-URJC>

²<https://acidxlemons.github.io/TFG-RAG-URJC/>

1.4 Objetivos del proyecto

El objetivo principal de este trabajo es desarrollar un asistente conversacional corporativo sustentado en una arquitectura RAG *on-premise*, capaz de consultar documentación cambiante almacenada en la nube o en otras fuentes heterogéneas sin sacar los datos del entorno controlado de la organización. Aunque la idea general es poder trabajar con repositorios documentales de distinto tipo, la implementación y las pruebas de este TFG se han realizado principalmente sobre SharePoint como caso real de integración.

La motivación operativa es clara: si los problemas detectados son dispersión documental, formatos heterogéneos, riesgo de *Shadow AI* y falta de trazabilidad, la respuesta técnica debe ser un sistema que recupere información de forma centralizada, la explique con lenguaje natural y deje claro en qué documentos se apoya.

Para alcanzar este objetivo general, se han definido los siguientes objetivos específicos:

1. *Diseñar una arquitectura modular reutilizando bloques ya disponibles y añadiendo desarrollo propio donde aporta valor*: combinar modelos de lenguaje locales, embeddings, una base de datos vectorial y una interfaz conversacional, añadiendo un backend en Python que orqueste todo el sistema.
2. *Permitir ingesta multicanal y actualización de conocimiento*: incorporar carga y actualización documental desde fuentes heterogéneas, con especial foco en SharePoint, incluyendo OCR para documentos escaneados y extracción web cuando sea necesario.
3. *Ofrecer respuestas fundamentadas y trazables*: implementar un flujo RAG que priorice la recuperación documental frente a la generación libre y muestre al usuario sobre qué fuentes se construye cada respuesta.
4. *Añadir capacidades complementarias sin entrenar modelos desde cero*: integrar búsqueda web controlada, scraping, análisis de imágenes y consulta normativa del BOE reutilizando herramientas ya disponibles y conectándolas mediante lógica propia.
5. *Explorar MCP como línea de integración futura*: desarrollar un servidor compatible con el enfoque de Model Context Protocol (MCP) [6] para el caso del BOE, dejando preparada una vía de evolución aunque la interfaz web del proyecto siga usando integración por API en la implementación actual.

6. *Desplegar el sistema de forma reproducible y observable*: contenerizar el ecosistema con Docker y habilitar métricas y paneles de monitorización que permitan operar y analizar el sistema.

1.5 Alcance y limitaciones

1.5.1 Alcance

El proyecto cubre el ciclo completo de construcción de un sistema RAG empresarial, desde el diseño de la arquitectura hasta su despliegue reproducible. El alcance real del trabajo se resume en los siguientes puntos:

- Selección y despliegue de modelos existentes de lenguaje, visión y embeddings; no se entrena un modelo fundacional desde cero.
- Diseño e implementación de una arquitectura basada en microservicios contenerizados.
- Desarrollo de un backend propio en Python para orquestación, recuperación documental, control de acceso y conexión con herramientas externas.
- Integración de un pipeline de ingesta documental con soporte para PDFs, HTML, imágenes y documentación alojada en SharePoint.
- Construcción de un motor de búsqueda semántica basado en embeddings y almacenamiento vectorial.
- Reutilización de OpenWebUI como interfaz conversacional e integración de su autenticación con el proveedor corporativo de identidad mediante OpenID Connect (OIDC) sobre Azure AD, es decir, inicio de sesión único corporativo (SSO, *Single Sign-On*).
- Extensión de OpenWebUI mediante un pipeline propio que añade encaminamiento, memoria de sesión y control de acceso documental.
- Integración funcional con fuentes externas como SharePoint, BOE e Internet.
- Desarrollo de un servidor MCP para normativa del BOE como prueba de concepto técnica orientada a futuras integraciones.
- Incorporación de un stack de monitorización y observabilidad para analizar uso y rendimiento.

En otras palabras, el alcance combina dos tipos de trabajo. Por un lado, se reutilizan componentes ya maduros, como OpenWebUI, Ollama, Qdrant o PaddleOCR. Por otro, se desarrollan específicamente para el TFG las piezas que faltaban para convertirlos en un sistema coherente: el backend RAG, el pipeline JARVIS, el indexador, la integración BOE y la configuración operativa del despliegue.

En relación con MCP, conviene precisar el alcance para evitar ambigüedades: en este TFG no se desarrolla un cliente MCP completo dentro de la interfaz web, sino un servidor MCP propio validado en entorno compatible. La funcionalidad visible por el usuario final se presta actualmente mediante llamadas a API, mientras que MCP queda como capacidad implementada y preparada para evolucionar.

1.5.2 Limitaciones

Se identifican las siguientes limitaciones reconocidas:

- *Requisitos hardware*: el sistema requiere una GPU NVIDIA con suficiente VRAM (memoria de vídeo de la tarjeta gráfica) para mantener un rendimiento aceptable en tareas de OCR e inferencia local.
- *Predominio de SharePoint en la validación*: aunque la arquitectura está pensada para documentos cambiantes en la nube y de distinta naturaleza, la validación experimental se ha centrado sobre todo en SharePoint, que es la integración más desarrollada del proyecto.
- *BOE con doble nivel de integración*: el sistema permite consultas normativas desde la interfaz mediante API, pero la integración MCP nativa en la web no se incorpora todavía por limitaciones del cliente utilizado.
- *MCP validado fuera de la interfaz principal*: el servidor MCP desarrollado funciona en entorno de terminal con clientes compatibles, pero su adopción dentro de la interfaz web queda como trabajo futuro.
- *Sesgo en el ajuste fino*: el *fine-tuning* LoRA se realiza con datos sintéticos generados a partir del propio sistema base, lo que puede introducir sesgos y limitar la generalización.
- *Evaluación acotada*: la evaluación se realiza principalmente de forma funcional y cualitativa; benchmarks académicos más formales quedan fuera del alcance temporal del proyecto.
- *Obsolescencia rápida del contexto tecnológico*: el ecosistema de modelos, herramientas y estándares evoluciona a gran velocidad, de modo que algunas

decisiones tecnológicas razonables durante el desarrollo pueden quedar superadas en pocos meses.

1.6 Contribuciones principales

Las contribuciones técnicas concretas de este trabajo son:

1. Una arquitectura reproducible basada en microservicios, definida declarativamente en Docker Compose, con servicios orientados a ingesta, almacenamiento, recuperación, generación y monitorización.
2. Un agente encaminador, JARVIS, capaz de seleccionar distintos modos de operación en función de la intención detectada y de guiar cada consulta hacia el subsistema adecuado.
3. Un pipeline de ingesta multicanal que unifica SharePoint, scraping web, OCR y carga manual en una única base de conocimiento vectorial.
4. Una integración funcional con el BOE dentro del sistema, permitiendo búsquedas y consultas normativas desde la experiencia conversacional.
5. El desarrollo y validación de un servidor MCP para consulta normativa del BOE, operativo fuera de la interfaz principal y planteado como base de evolución futura.
6. Un flujo de adaptación de modelos base mediante LoRA orientado al dominio del proyecto, evaluado como complemento y no como núcleo de la solución.
7. La publicación del sistema como proyecto de código abierto, acompañado de una página web de presentación y de documentación para su despliegue reproducible. Las direcciones del repositorio y de la página del proyecto se recogen al final del Capítulo 9.

1.7 Estructura de la memoria

El resto de este documento se estructura de la siguiente manera:

En el **Capítulo 2** se presentan los fundamentos que permiten entender la solución propuesta: qué son los LLMs, por qué un LLM por sí solo no basta en este problema y qué papel desempeñan RAG, los embeddings, las bases vectoriales, la cuantización, LoRA y MCP.

El **Capítulo 3** aterriza las tecnologías concretas utilizadas en el proyecto, explicando para qué sirve cada una, por qué se ha elegido y cuál es su papel dentro del sistema completo.

El **Capítulo 4** analiza sistemas existentes similares al desarrollado, tanto comerciales como de código abierto, para situar la propuesta dentro del ecosistema actual.

El **Capítulo 5** ofrece una descripción funcional completa del sistema desde la perspectiva del usuario: interfaz conversacional, modos de operación, escenarios de uso y panel de administración.

El **Capítulo 6** constituye el núcleo técnico del proyecto. Describe la arquitectura, la separación entre desarrollo propio y herramientas externas, el mecanismo Pipeline de OpenWebUI, el agente JARVIS, el motor RAG y las integraciones principales.

El **Capítulo 7** describe la metodología seguida: el proceso de desarrollo, los sprints en detalle —con sus tareas, decisiones tecnológicas y lecciones aprendidas— y el uso de herramientas de inteligencia artificial como apoyo.

El **Capítulo 8** presenta los resultados obtenidos: métricas de rendimiento, evaluación funcional y análisis del comportamiento del sistema.

Por último, el **Capítulo 9** recoge las conclusiones: el grado de consecución de los objetivos, el balance tecnológico y de esfuerzo del proyecto, los conocimientos aplicados y adquiridos, las líneas futuras de trabajo y las direcciones públicas del código y de la página web del proyecto.

Capítulo 2

Fundamentos de la solución propuesta

Este capítulo no pretende enumerar todas las tecnologías del sector, sino introducir únicamente los conceptos necesarios para entender la solución desarrollada en este TFG. La idea es guiar al lector desde el problema hasta la arquitectura elegida: primero se explica qué puede aportar un LLM y por qué no basta por sí solo; después se presenta RAG como mecanismo para responder con apoyo documental; y finalmente se describen los componentes que permiten ejecutar todo ello de forma local, controlada y útil en un contexto corporativo.

2.1 Grandes Modelos de Lenguaje (LLMs)

Los grandes modelos de lenguaje (LLMs) son modelos neuronales entrenados para predecir el siguiente *token* en una secuencia. El salto de rendimiento moderno llega con la arquitectura *Transformer* [7], donde el mecanismo de atención permite capturar relaciones de contexto con mucha más eficacia que arquitecturas previas.

Desde el punto de vista del usuario, un LLM permite dialogar en lenguaje natural, resumir contenido, reformular instrucciones o generar texto coherente. Sin embargo, conviene aclarar un punto importante para este trabajo: un LLM no es una base de datos ni un buscador documental. Si se le pregunta por información corporativa que no forma parte de su entrenamiento o que ha cambiado recientemente, tenderá a responder con estimaciones plausibles, no necesariamente con hechos verificables.

2.1.1 Propietarios vs. pesos abiertos

En la práctica, existen dos formas habituales de utilizar LLMs:

- *Modelos propietarios vía API*: ofrecen un producto muy maduro y alto rendimiento, pero implican dependencia de un proveedor externo y procesamiento fuera del perímetro corporativo.
- *Modelos de pesos abiertos (open-weights)*: permiten ejecución local u *on-premise*, lo que habilita mayor control sobre datos, configuración y despliegue.

En este proyecto se prioriza el segundo enfoque porque el requisito principal es operar en local. También conviene diferenciar entre *pesos abiertos* y *código abierto*: que un modelo pueda descargarse y ejecutarse localmente no implica necesariamente una licencia plenamente abierta [8], [9], [10].

El sistema desarrollado reutiliza modelos ya existentes, principalmente de la familia Qwen 2.5 y, en fases previas del proyecto, también Llama 3.1, por ser viables en despliegue local y contar con un ecosistema maduro [11], [12]. Esta decisión refuerza la idea central del proyecto: el valor no está en crear un nuevo modelo fundacional, sino en integrarlo correctamente con el conocimiento de la organización.

2.2 Por qué un LLM por sí solo no resuelve este problema

El problema abordado en este TFG no consiste en mantener una conversación general, sino en responder sobre documentación interna, normativa y contenidos que cambian con el tiempo. Un LLM aislado presenta varias limitaciones en este escenario:

- No conoce por defecto los documentos privados de la organización.
- No accede en tiempo real a repositorios documentales salvo que se le conecte explícitamente a ellos.
- Puede mezclar conocimiento general con suposiciones plausibles.
- No ofrece trazabilidad automática hacia los documentos utilizados.

Por ello, aunque los LLMs son la pieza que permite la interacción en lenguaje natural, no constituyen por sí mismos la solución completa. Hace falta una arquitectura adicional que recupere información, filtre fuentes, construya contexto y limite la generación a aquello que realmente se ha encontrado.

2.3 Generación Aumentada mediante Recuperación (RAG)

La arquitectura que responde a ese problema es RAG (*Retrieval-Augmented Generation*) [1]. Su idea básica es sencilla: antes de generar una respuesta, el sistema busca fragmentos relevantes de documentos y se los proporciona al modelo como contexto. De esta forma, el LLM deja de apoyarse solo en su memoria estadística y pasa a responder utilizando evidencia concreta.

Un pipeline RAG típico tiene dos fases:

1. *Recuperación*: la pregunta del usuario se transforma en una representación numérica y se buscan fragmentos relevantes en una base documental indexada.
2. *Generación*: el LLM recibe la pregunta junto con ese contexto recuperado y redacta una respuesta basada en él.

Para un lector no técnico, puede entenderse como una combinación entre un buscador y un asistente conversacional. El buscador encuentra la información; el asistente la explica. Esa combinación es la razón por la que RAG encaja mejor en este proyecto que un chatbot convencional.

2.3.1 Mejoras prácticas: búsqueda híbrida y memoria acotada

En un sistema real, RAG no suele quedarse en una búsqueda semántica simple:

- *Búsqueda híbrida*: combina una recuperación basada en significado, obtenida al comparar embeddings, con otra basada en coincidencias exactas de palabras, habitualmente implementada con métodos como BM25. Esto evita perder siglas, identificadores, nombres de procedimientos o artículos legales [13], [14], [15].
- *Reordenación (reranking)*: un segundo modelo, llamado *reranker*, vuelve a puntuar los fragmentos recuperados según su relevancia real para la pregunta y selecciona los más útiles antes de enviarlos al modelo. Mejora la precisión frente a usar solo la similitud de embeddings.
- *Memoria conversacional acotada*: se conserva parte del historial cuando aporta contexto, pero evitando que el *prompt* crezca sin control.

La solución de este TFG utiliza precisamente esa variante práctica de RAG, porque es la que mejor se adapta a consultas corporativas reales.

2.4 Modelos de Embeddings

Los embeddings convierten texto en vectores numéricos donde la cercanía entre vectores representa similitud semántica. De forma intuitiva, puede pensarse que cada texto se transforma en una huella numérica compacta: si dos huellas quedan cerca en ese espacio, sus significados tienden a ser parecidos. Son el mecanismo que permite pasar de una pregunta escrita por una persona a una búsqueda de contenido relacionada por significado y no solo por palabras exactas.

Una medida habitual de similitud es la similitud del coseno:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|} \quad (2.1)$$

En este proyecto se emplean modelos de SentenceTransformers [16], [17], ya existentes y desplegados en local, porque permiten generar embeddings sin depender de servicios externos. Esta decisión es importante por dos razones: preserva la privacidad y desacopla la búsqueda semántica del proveedor del LLM generativo.

También se explora la adaptación del modelo de embeddings al dominio del proyecto. Esto es relevante porque, en un entorno corporativo, términos internos, acrónimos y nomenclatura especializada pueden no estar bien representados por un modelo genérico.

2.5 Bases de datos vectoriales

Una vez calculados los embeddings, hace falta un sistema capaz de almacenarlos y consultarlos con rapidez. Ese papel lo cumplen las bases de datos vectoriales. En lugar de buscar solo texto, permiten buscar vectores parecidos al vector de la consulta.

En colecciones grandes se usan índices aproximados para acelerar la búsqueda. Uno de los enfoques más extendidos es HNSW [18], un grafo navegable jerárquico diseñado para encontrar vecinos cercanos de forma eficiente.

Tabla 2.1: Comparativa de bases de datos vectoriales evaluadas para el proyecto.

Base de datos	Licencia	On-Premise	Filtrado	Híbrida
Qdrant	Apache 2.0	Sí	Sí	Sí
Weaviate	BSD-3	Sí	Sí	Sí
Milvus	Apache 2.0	Sí	Sí	Sí
Chroma	Apache 2.0	Sí	Limitado	Limitado
Pinecone	Propietaria	No (SaaS)	Sí	Parcial

Se selecciona Qdrant por ser *on-premise*, por su filtrado por metadatos y por su buen soporte para búsqueda híbrida [15], [19]. Para este TFG no es solo una decisión de rendimiento, sino también de control: permite separar colecciones, añadir metadatos y aplicar restricciones de acceso documentales.

2.6 Cuantización y ejecución local

Ejecutar LLMs localmente exige gestionar bien los recursos de hardware, especialmente la VRAM de la GPU. La cuantización reduce la precisión numérica de los pesos para disminuir consumo de memoria y hacer viable la inferencia en hardware más modesto.

En el ecosistema local, el formato GGUF se utiliza ampliamente para empaquetar modelos cuantizados junto con sus metadatos [20], [21]. Herramientas como Ollama permiten servir estos modelos por API y simplifican mucho el despliegue [22].

Este concepto es importante para entender una de las decisiones prácticas del proyecto: no basta con elegir un buen modelo, también hay que elegir una variante que realmente pueda ejecutarse en el hardware disponible sin bloquear el resto del sistema.

2.7 Ajuste fino eficiente con LoRA

El ajuste fino (*fine-tuning*) adapta un modelo generalista a un dominio concreto. Sin embargo, reentrenar por completo un LLM moderno está fuera del alcance normal de un TFG y tampoco era el objetivo principal de este trabajo.

La técnica LoRA (*Low-Rank Adaptation*) reduce drásticamente ese coste reentrenando solo una pequeña parte de la red neuronal del modelo, en lugar de modificar todos sus pesos [23]. Concretamente, inserta capas adicionales de tamaño muy reducido —matrices de bajo rango— en las capas de atención del transformador y reentrena únicamente esas capas añadidas, dejando los pesos originales congelados. En la práctica, puede entenderse como ampliar la red neuronal con nuevas conexiones especializadas sin tener que reentrenar toda la arquitectura de partida.

Formalmente, el cambio en el peso de una capa se expresa como:

$$W' = W + \Delta W = W + BA \quad (2.2)$$

donde A y B son matrices de rango reducido con $r \ll \min(d, k)$, lo que hace que el número de parámetros entrenables pase de millones a miles.

En este proyecto, LoRA se estudia como una vía de adaptación incremental sobre modelos ya existentes, no como el eje principal de la solución. La experiencia obtenida

muestra que, para consultas RAG puras, suele tener más impacto un buen contexto recuperado y un buen modelo de embeddings que modificar directamente los pesos del LLM generativo.

2.8 Model Context Protocol (MCP)

MCP es un estándar abierto propuesto por Anthropic en 2024 para integrar herramientas externas con agentes basados en LLM mediante una interfaz cliente-servidor tipada [24]. Un servidor MCP expone herramientas y un cliente compatible puede descubrirlas e invocarlas de forma estructurada.

Para este TFG, MCP es relevante por dos motivos. Primero, porque representa una forma moderna de conectar modelos con herramientas externas. Segundo, porque permite aclarar el alcance del trabajo: el proyecto no implementa una plataforma MCP completa desde cero, sino un *servidor MCP propio* orientado a consulta normativa del BOE, desarrollado como prueba de integración real.

Ese servidor se ha validado en ejecución por terminal con clientes compatibles. Además, OpenWebUI ya ofrece soporte nativo para servidores MCP en versiones recientes [25]. Conviene matizar un punto que suele causar confusión: el cliente MCP reside en la aplicación (OpenWebUI), no en el motor de inferencia —Ollama no «usa» MCP, solo sirve el modelo—, y ese flujo exige además que sea el propio LLM quien decida invocar las herramientas (*function calling*). Aun así, en este TFG la funcionalidad visible dentro de la interfaz web sigue resolviéndose principalmente mediante API tradicional y pipeline propio, porque esa fue la vía integrada y validada durante el desarrollo para mantener mayor control sobre autenticación, encaminamiento y operación del sistema. Las razones técnicas concretas y una valoración de cuándo conviene activar MCP se detallan en el Anexo C. Por tanto, MCP aparece en este trabajo como tecnología implementada y validada, pero todavía no como flujo principal de uso para el usuario final.

Capítulo 3

Tecnologías utilizadas y criterio de elección

Este capítulo aterriza la solución en tecnologías concretas. Su objetivo principal es describir qué es cada tecnología y qué papel desempeña dentro del sistema completo. Cuando resulta útil, se menciona también por qué encaja en el proyecto, pero el detalle de cómo se ha combinado con otras piezas se deja para el Capítulo 6. También resulta importante distinguir entre herramientas preexistentes reutilizadas en el proyecto y componentes desarrollados específicamente en este TFG.

3.1 Python como lenguaje de integración principal

El lenguaje vertebrador del proyecto es Python. La razón principal no es solo su popularidad, sino su capacidad para actuar como lenguaje de integración entre bibliotecas de IA, servicios web, automatización y procesamiento documental. En este TFG, Python permite conectar en un mismo ecosistema el backend, el indexador, el pipeline de OpenWebUI, la integración con SharePoint, el scraping, el OCR y el servidor MCP.

Conviene separar aquí dos planos. Por un lado, hay paquetes que se usan directamente en el código desarrollado en el TFG, como FastAPI, qdrant-client, sentence-transformers o msal. Por otro, hay dependencias que vienen dadas por herramientas ya reutilizadas en el stack, como parte del ecosistema de OpenWebUI, Docker o Grafana. Esta distinción es importante porque el proyecto no consiste en construir esas plataformas de base, sino en apoyarse en ellas y extenderlas donde era necesario.

Además, buena parte de las herramientas reutilizadas en el proyecto exponen su mejor soporte precisamente en este lenguaje. Entre los paquetes y bibliotecas más importantes utilizados se encuentran:

- **FastAPI** y **Uvicorn** para los servicios web y la API REST.
- **Pydantic** para validación de datos y configuración tipada.
- **sentence-transformers** para generación de embeddings densos y **fastembed** para los vectores dispersos (BM25).
- **qdrant-client** para interacción con la base de datos vectorial.
- **msal** y clientes asociados para autenticación y acceso a Microsoft Graph.
- **playwright** para automatización de navegador.
- **trafilatura**, **readability** y **BeautifulSoup** para extracción de contenido web.
- **PaddleOCR** para OCR de documentos escaneados y **Ray** para paralelizar su procesamiento.
- **duckduckgo__search** para búsqueda web controlada.
- **fastmcp** para el servidor MCP del BOE.
- **prometheus-client** para exponer métricas del backend.

Esta elección también ayuda a explicar la naturaleza del trabajo realizado: el proyecto no consiste en inventar estas bibliotecas, sino en utilizarlas de forma coordinada y construir sobre ellas la lógica que faltaba para el caso de uso corporativo.

3.2 OpenWebUI: interfaz conversacional reutilizada

OpenWebUI¹ es la interfaz conversacional utilizada como punto de entrada del sistema [26]. Es una herramienta externa, ya existente, que he reutilizado para no invertir el esfuerzo del TFG en construir una interfaz de chat desde cero. Esta decisión permite concentrar el trabajo en la parte realmente diferencial del proyecto: la integración con las fuentes documentales y la lógica del asistente.

3.2.1 Arquitectura interna de OpenWebUI

OpenWebUI se estructura como una aplicación web con tres bloques principales:

- *Frontend*: interfaz tipo chat accesible desde navegador, con historial de conversaciones, carga de adjuntos y streaming de respuestas.

¹<https://openwebui.com/>

- *Backend*: lógica de autenticación, persistencia de chats y gestión de modelos.
- *Base de datos*: almacenamiento de usuarios, conversaciones y metadatos.

Para el lector, lo importante es entender su papel dentro del proyecto: OpenWebUI aporta la experiencia de usuario, pero no constituye la inteligencia principal del sistema. Esa inteligencia se añade mediante el pipeline desarrollado en este TFG. En consecuencia, cuando en esta memoria se habla de “integrar autenticación corporativa con la interfaz conversacional”, no se está describiendo la creación de una interfaz nueva, sino la reutilización de OpenWebUI junto con su conexión al proveedor corporativo de identidad y su ampliación mediante un pipeline propio.

3.2.2 Por qué OpenWebUI

La elección de OpenWebUI se justifica porque ya ofrece varias capacidades deseables en un entorno corporativo: interfaz madura, persistencia de conversaciones, compatibilidad con modelos locales y soporte de autenticación. Frente a hacer una interfaz propia, reduce tiempo de desarrollo y riesgo técnico.

Tabla 3.1: Comparativa de interfaces de usuario para LLMs locales.

Criterio	OpenWebUI	Gradio	Streamlit	ChatUI
UI tipo ChatGPT	✓	Parcial	Parcial	✓
Compatibilidad con Ollama	Nativa	Manual	Manual	Nativa
Sistema de Pipelines/Plugins	✓	×	×	×
Gestión de usuarios	✓	×	×	✓
Persistencia de chats	✓	×	×	✓
SSO / OIDC	✓	×	×	×
Código abierto	✓	✓	✓	✓
Carga de archivos adjuntos	✓	✓	✓	Parcial

El factor decisivo es el sistema de Pipelines, porque permite insertar lógica personalizada sin modificar el núcleo de la herramienta. Gracias a ello, la interfaz sigue siendo una pieza reutilizada, pero el comportamiento final del asistente pasa a estar controlado por desarrollo propio. El funcionamiento de este mecanismo se describe en el Capítulo 6 y su implementación detallada en el Anexo A.

3.3 Ollama: ejecución local de modelos

Ollama² es la herramienta externa utilizada para ejecutar modelos de lenguaje localmente. Su función es cargar modelos, servirlos por API y gestionar su uso en el hardware disponible [27]. Se ha elegido porque facilita el despliegue de modelos locales sin construir una capa de inferencia propia, gestiona la VRAM de manera operativa (carga y descarga modelos según necesidad), expone una API sencilla y encaja bien como un servicio más dentro de Docker Compose.

3.3.1 Modelos desplegados

En el sistema se emplean varias familias de modelos ya existentes, cada una con una función concreta:

Tabla 3.2: Modelos de IA desplegados en Ollama y su función en el sistema.

Modelo	Función	Cuantización	VRAM
rag-qwen-ft:latest	Modelo principal del alias JARVIS para chat y RAG	LoRA FT	~8 GB
qwen2.5:32b-instruct-q4_K_M	Fallback de texto y comparación base	Q4_K_M	~19 GB
qwen2.5-coder:32b	Modelo auxiliar instalado para tareas de código	—	~19 GB
qwen2.5vl:7b	Análisis de imágenes y OCR multimodal	—	~6 GB

La tabla refuerza una idea importante: los modelos no se han creado en este TFG, pero sí se han seleccionado, desplegado, probado y asignado a tareas concretas dentro de una arquitectura coherente. En el despliegue actual del proyecto, la familia Qwen 2.5 concentra las funciones principales de texto, visión y adaptación al dominio.

3.3.2 Stack de modelos en producción

La elección del LLM principal no fue inmediata: evolucionó a lo largo del proyecto en cuatro etapas —de llama3.1:8b en cuantización Q4 y Q8, a Qwen 2.5 7B base y, finalmente, al modelo ajustado con LoRA—, cada una motivada por limitaciones reales de calidad, ventana de contexto o rendimiento en español detectadas durante el uso. Esa trayectoria completa, con sus causas técnicas (incluido el problema de los *stop tokens* de Qwen), se recoge en el Anexo E. La Tabla 3.3 resume el stack de modelos resultante, que combina el LLM fine-tuned, un modelo grande para análisis, un modelo de visión y los modelos auxiliares de recuperación (embeddings densos, dispersos y reranker).

²<https://ollama.com/>

Tabla 3.3: Stack de modelos en producción (versión TFG y sistema empresarial).

Alias / Rol	Modelo	Función	VRAM	Contexto
JARVIS / europav-IA	rag-qwen-ft:latest	Chat RAG principal con fine-tuning LoRA	~8 GB	32 K
qwen2.5-32b	qwen2.5:32b-instruct-q4_K_M	Fallback y análisis complejos	~19 GB	32 K
qwen2.5vl	qwen2.5vl:7b	Visión e interpretación de imágenes	~6 GB	128 K
Embeddings (densos)	paraphrase-multilingual-MiniLM-L12-v2	Vectorización semántica de texto (CPU)	—	512 tok.
Sparse (BM25)	Qdrant/bm25 (fastembed)	Vectores dispersos para búsqueda léxica (CPU)	—	—
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2	Reordenación de candidatos por relevancia (CPU)	—	—

El stack de producción cubre la generación con modelos especializados e independientes para no saturar la VRAM: el modelo fine-tuned para RAG corporativo, un modelo de 32 B para análisis complejo y un modelo multimodal para visión. Tanto el modelo fine-tuned (JARVIS) como el de 32 B heredan la ventana de contexto de 32 K tokens de la familia Qwen 2.5. La recuperación se apoya en tres modelos ligeros que se ejecutan en CPU: el de embeddings densos, el de vectores dispersos BM25 y el reranker Cross-Encoder. LiteLLM actúa como proxy unificado con enrutamiento automático, caché Redis y fallback entre modelos, y mantiene alias de compatibilidad hacia atrás (llama3.1-8b → qwen2.5-32b) para integraciones anteriores.

3.3.3 Query Processor: enrutamiento inteligente y expansión de consultas

El QueryProcessor (core/query_processor.py) es un componente transversal que intercepta cada consulta del usuario antes de que llegue a la base de datos vectorial, enriqueciéndola en tres dimensiones:

Detección de intención (*Intent Detection*). Clasifica la consulta en cuatro tipos mediante expresiones regulares sobre el texto en español:

- **FACTUAL** — pregunta puntual (“¿qué es...?”, “¿cuándo...?”): `top_k=5`, búsqueda densa.
- **PROCEDURAL** — cómo hacer algo (“¿cómo...?”, “pasos para...”): `top_k=8`, búsqueda híbrida.
- **ANALYTICAL** — análisis o comparación (“diferencia entre...”, “resume...”): `top_k=12`, búsqueda híbrida.
- **CONVERSATIONAL** — saludo o chat general: `top_k=0`, sin búsqueda RAG.

Enrutamiento de modelos (*Smart Model Routing*). La intención detectada determina qué modelo LLM genera la respuesta final: las consultas FACTUAL y

PROCEDURAL van a **JARVIS** (`rag-qwen-ft:latest`: fine-tuned, rápido, citas correctas), y las ANALYTICAL a **qwen2.5-32b** (ventana de 32 K, mayor capacidad de razonamiento y síntesis multi-documento). Es el principio de *right tool for the job*.

Expansión de consultas (*Query Expansion*). Para consultas no conversacionales, el `QueryProcessor` genera dos reformulaciones adicionales usando JARVIS como LLM auxiliar (con timeout de 12 s). Las tres variantes se lanzan en paralelo contra Qdrant y sus resultados se fusionan eliminando duplicados antes de pasarlos al reranker, lo que mejora el *recall* semántico cuando el usuario usa terminología distinta a la del documento:

“normas seguridad escaleras” $\xrightarrow{\text{expansión}}$ *“requisitos técnicos escaleras portátiles”*,
“normativa PRL trabajo en altura escaleras”

La combinación de los tres mecanismos —intent detection, model routing y query expansion— convierte el pipeline RAG en un sistema adaptativo que ajusta automáticamente su estrategia de recuperación y generación según la naturaleza de cada pregunta, sin intervención del usuario.

3.4 Qdrant: base de datos vectorial

Qdrant³ es la base de datos vectorial empleada como almacén principal del conocimiento documental del sistema [28]. Es una herramienta externa, pero su estructura, colecciones y criterios de uso se definen dentro del proyecto. De sus capacidades, el sistema aprovecha el *índice HNSW* (búsquedas vectoriales rápidas sobre grandes volúmenes de fragmentos), la *búsqueda híbrida* (combina similitud semántica densa y coincidencia léxica dispersa), el *filtrado por metadatos* (segmentación por fuente, documento o departamento) y el soporte *multi-colección* (separación de documentos por contexto de uso). Para el proyecto, esto significa que Qdrant no es solo un repositorio técnico, sino el componente que hace posible que el sistema encuentre evidencia relevante antes de responder.

3.5 FastAPI: backend del sistema

FastAPI⁴ es el framework seleccionado para construir los servicios backend en Python [29]. A diferencia de OpenWebUI, Ollama o Qdrant, que son herramientas reutilizadas, aquí sí aparece una parte importante de desarrollo propio: los servicios

³<https://qdrant.tech/>

⁴<https://fastapi.tiangolo.com/>

implementados con FastAPI contienen gran parte de la lógica central del sistema. Se ha elegido por su facilidad para construir APIs tipadas y mantenibles, su soporte asíncrono (útil cuando el sistema depende de múltiples llamadas a servicios externos o internos) y su integración natural con el ecosistema Python (IA, scraping, OCR y autenticación en el mismo entorno). En este TFG, FastAPI sustenta el backend RAG, el indexador de documentos y parte de las integraciones auxiliares.

3.6 LiteLLM: proxy unificado de modelos

LiteLLM⁵ es una herramienta externa que se utiliza como capa intermedia entre los servicios propios y los modelos servidos por Ollama [30]. Su utilidad está en unificar llamadas, aplicar alias de modelos, gestionar caché y simplificar el encaminamiento técnico. Desde la perspectiva del lector no técnico, su papel puede resumirse así: en lugar de que cada parte del sistema tenga que conocer todos los detalles de cada modelo, LiteLLM ofrece un punto único y homogéneo de acceso.

3.7 Docker y orquestación de contenedores

Docker⁶ y Docker Compose son la base del despliegue reproducible [31]. Se utilizan para empaquetar todos los componentes y ponerlos en marcha como un conjunto coordinado. Sus ventajas en este proyecto son la *reproducibilidad* (el mismo sistema se despliega de forma controlada en distintos entornos), el *aislamiento* (cada servicio mantiene sus dependencias separadas), la *seguridad perimetral* (solo ciertos servicios se exponen al exterior) y el *acceso a GPU bajo demanda* (se reserva aceleración solo para los servicios que la necesitan). En el contexto del TFG, esto también es parte de la contribución: no solo se han elegido tecnologías, sino que se han dejado integradas dentro de una arquitectura desplegable de forma repetible.

3.8 Herramientas de procesamiento documental y web

Esta sección reúne herramientas ya existentes que no han sido desarrolladas en el proyecto, pero que sí se han integrado y combinado para cubrir necesidades concretas del sistema.

⁵<https://docs.litellm.ai/>

⁶<https://www.docker.com/>

3.8.1 PaddleOCR: reconocimiento óptico de caracteres

PaddleOCR⁷ es un motor OCR externo utilizado para extraer texto de PDFs escaneados [32]. No se ha implementado un OCR propio; el trabajo realizado consiste en decidir cuándo usarlo, cómo integrarlo en la ingesta y cómo hacerlo convivir con el resto de componentes. La paralelización de su procesamiento se apoya en **Ray**.

3.8.2 Playwright: automatización de navegadores

Playwright⁸ es una herramienta externa de automatización de navegador [33]. Se utiliza porque muchas páginas web modernas dependen de JavaScript y no pueden procesarse con una simple petición HTTP. Su papel dentro del proyecto es permitir obtener el contenido real que ve el usuario en páginas dinámicas.

3.8.3 Trafilatura: extracción de contenido web

Trafilatura [34] es una biblioteca externa especializada en extraer el contenido principal de una página web, eliminando menús, barras laterales y ruido. En este TFG no se ha desarrollado un extractor propio desde cero; se ha integrado Trafilatura como primera opción dentro de una cadena de extracción con *fallbacks*, porque resuelve bien el problema de aislar el contenido relevante.

3.8.4 SentenceTransformers y reranker: recuperación

SentenceTransformers [16], [17] es la biblioteca utilizada para generar los embeddings densos, con el modelo `paraphrase-multilingual-MiniLM-L12-v2`, elegido por su equilibrio entre tamaño, soporte multilingüe y facilidad de despliegue. La recuperación se completa con dos piezas adicionales: los vectores dispersos **BM25** (Qdrant/bm25 vía `fastembed`), que aportan coincidencia léxica exacta, y un **reranker** de tipo *Cross-Encoder* (`cross-encoder/ms-marco-MiniLM-L-6-v2`) que reordena los candidatos por relevancia. Aquí también es importante la distinción metodológica: los modelos y librerías ya existían; lo que aporta el proyecto es su integración en el pipeline RAG y su ajuste al caso de uso documental.

3.9 Stack de monitorización

Para operar un sistema de este tipo no basta con que funcione: también hay que entender cómo se comporta. Por ello se incorpora un stack de observabilidad formado por herramientas ya existentes, configuradas y conectadas dentro del proyecto.

⁷<https://github.com/PaddlePaddle/PaddleOCR>

⁸<https://playwright.dev/>

3.9.1 Prometheus

Prometheus⁹ recoge métricas temporales del backend y de otros servicios [35]. Su utilidad en este TFG es permitir medir latencia, tasa de errores y patrones de uso en lugar de basar las decisiones de optimización en impresiones subjetivas.

3.9.2 Grafana

Grafana¹⁰ visualiza las métricas recogidas por Prometheus mediante paneles comprensibles [36]. En el proyecto se ha utilizado para construir paneles específicos de backend, GPU y bases de datos. Esto no debe entenderse como el desarrollo de una herramienta nueva, sino como el diseño de la capa de observación necesaria para operar el sistema con criterio.

3.9.3 NVIDIA DCGM Exporter

NVIDIA DCGM Exporter¹¹ expone métricas de la GPU en formato compatible con Prometheus [37]. Su presencia se justifica porque en un sistema que ejecuta modelos y OCR en local, la GPU es un recurso crítico y debe monitorizarse con el mismo nivel de detalle que el backend.

3.9.4 Loki y Promtail

Mientras Prometheus se ocupa de las métricas numéricas, **Loki** y **Promtail** cubren los *logs*: Promtail recolecta los registros de todos los contenedores y los envía a Loki, que los indexa y los hace consultables desde el mismo Grafana. Esto permite correlacionar un pico de latencia con las líneas de log exactas de ese instante, unificando métricas y logs en una sola herramienta de diagnóstico.

3.10 Otras tecnologías del ecosistema

- *PostgreSQL*: almacena usuarios, conversaciones y metadatos de OpenWebUI.
- *Redis*: se emplea como caché de respuestas a través de LiteLLM.
- *MinIO*: conserva documentos binarios como respaldo y fuente de reindexación.
- *NGINX*: actúa como proxy inverso y punto único de entrada.
- *MSAL (Microsoft Authentication Library)*: permite autenticación con Azure AD y acceso autenticado a SharePoint mediante Microsoft Graph.

⁹<https://prometheus.io/>

¹⁰<https://grafana.com/>

¹¹<https://github.com/NVIDIA/dcgm-exporter>

- *Ray*: reparte el procesamiento OCR entre varios *workers* durante la ingesta.

El diagrama general que resume cómo se conectan todas estas tecnologías —el flujo de consulta, la lógica propia del TFG, las integraciones externas y la capa de observabilidad— se presenta en el Anexo E (Figura E.1), por su tamaño apaisado.

Capítulo 4

Sistemas relacionados y posicionamiento de la propuesta

Este capítulo sitúa el sistema desarrollado dentro del ecosistema actual. Conviene aclarar desde el principio que **las soluciones que aquí se analizan no forman parte del sistema construido**: son alternativas parecidas —comerciales, abiertas o marcos de desarrollo— que resuelven problemas similares y que se podrían haber utilizado en lugar de la arquitectura propia, pero que se descartaron por las razones que se explican en cada caso. El objetivo, por tanto, no es comparar por comparar, sino mostrar qué soluciones existían antes de comenzar el proyecto, cuáles han aparecido o ganado relevancia durante los últimos meses y qué hueco intenta cubrir la propuesta de este TFG frente a ellas. Esta parte es especialmente importante porque el ámbito de los asistentes con LLM evoluciona muy rápido: herramientas que hace pocos meses no estaban maduras hoy ya compiten en funcionalidades concretas.

4.1 Asistentes comerciales basados en LLM

4.1.1 ChatGPT y GPT Enterprise (OpenAI)

ChatGPT es el asistente conversacional más conocido del mercado. Su versión empresarial añade controles administrativos, garantías comerciales y mejores condiciones de tratamiento de datos [38]. Aun así, sigue siendo un servicio en la nube: la organización no controla la infraestructura de inferencia ni puede modificar en profundidad la lógica del sistema.

Para este TFG, el principal límite de esta familia de soluciones no es su calidad técnica, sino la pérdida de soberanía operativa: los documentos y consultas no se procesan dentro de una infraestructura propia.

4.1.2 Microsoft Copilot para Microsoft 365

Microsoft Copilot destaca por su integración con SharePoint, Outlook, Teams y el resto del ecosistema Microsoft 365 [39]. Para organizaciones ya asentadas en ese entorno, su propuesta es muy atractiva porque reduce la fricción de adopción.

Sin embargo, esa misma integración va asociada a varias restricciones relevantes para el enfoque de este trabajo:

- procesamiento en nube, no en infraestructura propia;
- coste por usuario recurrente;
- escasa capacidad para modificar la lógica interna del asistente;
- dependencia fuerte del ecosistema Microsoft.

4.1.3 Google Gemini y Vertex AI

Google ofrece Gemini como producto de productividad y Vertex AI como plataforma de desarrollo e integración [40], [41]. Su fortaleza está en la amplitud del ecosistema cloud y en la disponibilidad de servicios gestionados.

Desde la perspectiva de este proyecto, la limitación vuelve a ser similar: son plataformas muy capaces, pero diseñadas principalmente para operar en la nube del proveedor, no para una arquitectura enteramente *on-premise*.

4.1.4 Amazon Bedrock

Amazon Bedrock proporciona acceso a varios modelos mediante una API unificada y ofrece piezas para agentes y RAG [42]. Es una solución flexible dentro del ecosistema AWS, pero sigue respondiendo al paradigma de servicio gestionado en la nube.

4.2 Herramientas abiertas orientadas a RAG y asistentes privados

4.2.1 PrivateGPT

PrivateGPT [43] representa una de las aproximaciones más conocidas a la idea de chat privado con documentos propios. Comparte con este TFG la filosofía de evitar servicios externos, pero está más orientado a un uso individual o de laboratorio que a un despliegue corporativo completo.

4.2.2 AnythingLLM

AnythingLLM [44] combina interfaz, RAG y gestión documental en una sola herramienta. Se acerca bastante a una solución práctica lista para usar, pero su comportamiento es más uniforme y menos controlable que el del sistema de este proyecto, donde existe un pipeline propio de encaminamiento e integración con fuentes específicas.

4.2.3 Danswer / Onyx

Danswer, renombrado posteriormente como Onyx, es una de las referencias más cercanas a un buscador corporativo con LLM y múltiples conectores [45]. Su relevancia es alta porque demuestra que la necesidad que aborda este TFG no es académica, sino real y compartida por muchas organizaciones.

La diferencia principal es de enfoque: Onyx persigue un producto generalista para empresa; este TFG construye una arquitectura más controlada y personalizada alrededor de un caso concreto, con ejecución local, pipeline propio y mayor énfasis en componentes desplegables por separado.

4.2.4 OpenClaw y la aceleración reciente del ecosistema

Durante los últimos meses han aparecido o se han popularizado nuevas herramientas abiertas orientadas a agentes, automatización y uso local de modelos. Un ejemplo representativo es OpenClaw [46], que se centra en conectar agentes con múltiples canales de mensajería y refleja una tendencia clara del sector: pasar de simples chats con documentos a sistemas más capaces de orquestar herramientas, operar en varios canales y ejecutar tareas más complejas.

Su mención es importante por dos razones. La primera es temporal: muestra que el espacio tecnológico del proyecto se está moviendo muy deprisa y que algunas referencias actuales no estaban igual de maduras cuando se definió la primera versión del sistema. La segunda es conceptual: confirma que la línea seguida en este TFG, basada en integrar modelos con herramientas y fuentes externas, va en la misma dirección que la evolución reciente del ecosistema.

4.2.5 NotebookLM y asistentes centrados en conocimiento aportado por el usuario

NotebookLM [47] representa otra línea de evolución distinta. En lugar de centrarse tanto en la infraestructura o la orquestación, pone el acento en trabajar sobre documentos aportados por el usuario y generar explicaciones, resúmenes y conexiones

entre fuentes. Su utilidad como referencia está en recordar que no todos los asistentes documentales buscan resolver el mismo problema.

Frente a NotebookLM, este TFG persigue un objetivo más orientado a entorno corporativo integrado: autenticación, documentos compartidos, actualización periódica, control de acceso y despliegue local. Por tanto, NotebookLM sirve como comparación funcional parcial, no como equivalente arquitectónico.

4.2.6 Frameworks: LangChain y LlamaIndex

LangChain [48] y LlamaIndex [49] son marcos de desarrollo, no productos finales. Ayudan a construir sistemas con LLMs, pero no resuelven por sí solos necesidades como autenticación corporativa, despliegue, observabilidad o interfaz completa.

Por eso, más que competidores directos, deben entenderse como piezas posibles dentro de un sistema mayor.

4.3 Análisis comparativo

La Tabla 4.1 resume la comparación entre la propuesta del TFG y varias alternativas representativas.

Tabla 4.1: Comparación del sistema JARVIS con soluciones alternativas.

Característica	JARVIS	ChatGPT Ent.	Copilot 365	AnythingLLM	Onyx	PrivateGPT	NotebookLM	OpenClaw
On-Premise 100 %	✓	×	×	✓	Parcial	✓	×	✓
Código abierto	✓	×	×	✓	✓	✓	×	✓
UI conversacional	✓	✓	✓	✓	✓	✓	✓	Parcial
RAG documental	✓	Parcial	Parcial	✓	✓	✓	✓	Parcial
SharePoint	✓	×	✓	×	✓	×	×	Parcial
OCR integrado	✓	×	×	×	×	×	×	Parcial
Web / herramientas	✓	Parcial	Parcial	Parcial	Parcial	×	×	✓
SSO corporativo	✓	✓	✓	×	✓	×	×	Parcial
Monitorización propia	✓	×	×	×	Parcial	×	×	Parcial
Lógica extensible	✓	×	×	Limitada	Parcial	Limitada	×	✓

La tabla no debe leerse como una clasificación absoluta, sino como una comparación respecto al objetivo concreto del TFG: asistente corporativo documental, desplegable en local, integrable con SharePoint y extensible mediante lógica propia.

4.4 Posicionamiento del sistema desarrollado

Tras este análisis, el sistema desarrollado ocupa una posición intermedia entre varias familias de soluciones:

1. *Frente a soluciones comerciales*: sacrifica comodidad de servicio gestionado a cambio de soberanía del dato y capacidad de personalización.

2. *Frente a herramientas abiertas listas para usar*: añade más integración corporativa, más control sobre el flujo y más separación clara entre componentes.
3. *Frente a frameworks*: ofrece un sistema desplegable, no solo una base de desarrollo.
4. *Frente a herramientas recientes orientadas a agentes*: comparte la dirección de evolución, pero con un foco más concreto en consulta documental empresarial y trazabilidad.

En resumen, la propuesta de este TFG no compite por ser el asistente más generalista del mercado, sino por resolver de forma sólida un problema muy específico: **consultar documentación corporativa cambiante con lenguaje natural, dentro de una infraestructura controlada y apoyándose en herramientas existentes correctamente integradas.**

Capítulo 5

Descripción Funcional del Sistema

En este capítulo se describe el sistema desde la perspectiva del usuario final. Se detalla qué puede hacer, cómo se presenta la interfaz conversacional, qué funcionalidades están disponibles y cómo puede interactuar el usuario con cada una de ellas. El objetivo es ofrecer una visión completa y comprensible de las capacidades del sistema sin entrar todavía en los detalles de implementación técnica, que se reservan para el Capítulo 6.

5.1 Visión General

Antes de describir en detalle qué hace el sistema, conviene ver su aspecto real. La Figura 5.1 muestra JARVIS en funcionamiento: una interfaz de chat en el navegador donde el usuario formula su pregunta en lenguaje natural y recibe la respuesta acompañada de las fuentes documentales utilizadas.

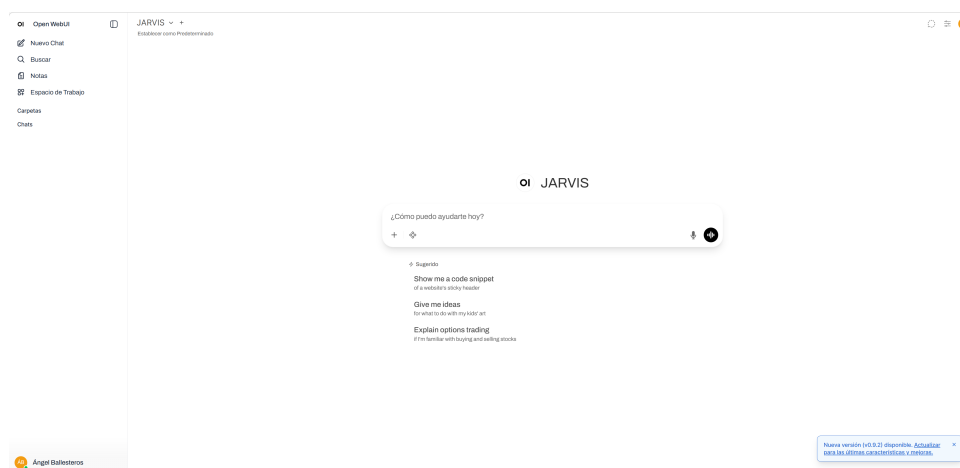


Figura 5.1: Interfaz de JARVIS en funcionamiento. La capa visual la proporciona OpenWebUI; el usuario escribe su consulta y recibe la respuesta en *streaming*, con las fuentes citadas.

JARVIS (*Just A Rather Very Intelligent System*) es un asistente conversacional corporativo desplegado en la infraestructura interna de la organización. Los empleados acceden al sistema a través de un navegador web y pueden interactuar con él en lenguaje natural para realizar las siguientes tareas:

- Consultar documentos corporativos internos (manuales, políticas, procedimientos, informes).
- Analizar páginas web proporcionando una URL.
- Buscar información actualizada en internet.
- Analizar imágenes y documentos escaneados.
- Consultar legislación del Boletín Oficial del Estado (BOE).
- Mantener conversaciones generales con un modelo de lenguaje.
- Consultar el estado de la indexación de documentos.
- Listar los documentos disponibles en la base de conocimiento.

Lo que el usuario ve es una interfaz de chat familiar, proporcionada por OpenWebUI —una herramienta de código abierto que se ha reutilizado como capa de presentación, en lugar de construir una interfaz desde cero—. Ahora bien, el valor de este proyecto no reside en esa interfaz. OpenWebUI, por sí solo, se limita a conversar con un modelo de lenguaje que responde “de memoria”. Lo que convierte esa ventana de chat en JARVIS es la capa desarrollada en este TFG que se sitúa detrás de ella: un *pipeline* propio y un backend que interceptan cada mensaje antes de que llegue al modelo, detectan la intención del usuario, buscan en los documentos corporativos, consultan fuentes externas cuando hace falta y devuelven respuestas trazables citando su origen. Dicho de otro modo, OpenWebUI aporta la ventana; el trabajo de este proyecto aporta la inteligencia que hay al otro lado. Esta distinción se desarrolla en profundidad en el Capítulo 6.

El sistema se ha diseñado para que detecte automáticamente la intención del usuario a partir del texto del mensaje, sin necesidad de comandos explícitos, menús ni modos manuales. El usuario simplemente escribe su pregunta o solicitud y el sistema decide internamente qué funcionalidad activar. Esta transparencia busca que la experiencia sea natural e intuitiva, similar a interactuar con un asistente humano.

5.2 Acceso al Sistema e Inicio de Sesión

El sistema se accede a través de una URL corporativa protegida por HTTPS (puerto 8443). Al acceder por primera vez, el usuario es redirigido al flujo de autenticación corporativa mediante Azure Active Directory (SSO). El proceso es el siguiente:

1. El usuario accede a `https://servidor:8443` desde su navegador.
2. El sistema redirige al portal de autenticación de Azure AD (Microsoft).
3. El usuario introduce sus credenciales corporativas (las mismas que utiliza para acceder a Outlook, Teams o SharePoint).
4. Tras la autenticación exitosa, se le redirige de vuelta al sistema con su sesión iniciada.
5. El sistema identifica automáticamente su nombre, correo electrónico y los grupos a los que pertenece en Azure AD.

No es necesario crear una cuenta separada ni recordar credenciales adicionales. Los grupos del directorio activo determinan a qué documentos tiene acceso cada usuario, implementando así un control de acceso departamental transparente.

5.3 Interfaz de Usuario

La interfaz de usuario es una aplicación web tipo chat, similar en aspecto y funcionamiento a ChatGPT o Microsoft Copilot. En el prototipo implementado, esta capa la proporciona OpenWebUI [26]. La interfaz se estructura en los siguientes elementos principales:

5.3.1 Panel Principal de Chat

El área central de la pantalla constituye el panel de conversación. El usuario escribe su mensaje en un campo de texto en la parte inferior y la respuesta del asistente aparece en la parte superior, en un formato de chat alternado (mensaje del usuario a la derecha, respuesta del asistente a la izquierda).

Las respuestas se muestran en *streaming en tiempo real*: el texto aparece progresivamente, token a token, a medida que el modelo lo genera. Esto proporciona retroalimentación inmediata al usuario y evita la sensación de espera prolongada.

Las respuestas soportan formato enriquecido:

- **Markdown:** Negritas, cursivas, listas, encabezados, código en línea.
- **Bloques de código:** Con resaltado de sintaxis para múltiples lenguajes.
- **Tablas:** Para información estructurada.
- **Enlaces:** URLs clicables a fuentes externas o documentos referenciados.
- *Indicadores visuales:* Etiquetas e iconos que ayudan a identificar el modo de operación activo.

5.3.2 Panel Lateral de Conversaciones

A la izquierda de la interfaz se muestra un panel con el historial de conversaciones del usuario. Cada conversación tiene un título generado automáticamente y puede retomarse en cualquier momento. Las conversaciones se almacenan de forma persistente en la base de datos PostgreSQL del sistema.

5.3.3 Carga de Archivos Adjuntos

La interfaz permite adjuntar archivos directamente en el chat mediante un botón de clip o arrastrando y soltando (*drag and drop*). Los tipos de archivos soportados incluyen:

- **Imágenes** (PNG, JPG, WebP): Se procesan con modelos de visión multimodal para describir, transcribir o analizar su contenido.
- **Documentos PDF:** Si el PDF contiene texto seleccionable, OpenWebUI extrae el texto y lo envía al modelo junto con la pregunta del usuario. Si el PDF es escaneado, se recomienda utilizar la vía de ingesta automatizada (carpeta `data/watch`) para que el motor OCR lo procese.

5.4 Modos de Operación

A continuación se describen los modos de operación fundamentales configurados en el sistema, vistos desde la perspectiva del usuario, con ejemplos de los tipos de consultas que activan cada modo.

5.4.1 Consulta de Documentos Internos (RAG)

Este es el modo principal del sistema y la funcionalidad que justifica todo el proyecto. Cuando el usuario formula una pregunta relacionada con documentos corporativos, el sistema busca automáticamente en su base de conocimiento vectorial y

genera una respuesta fundamentada exclusivamente en el contenido de los documentos recuperados.

- **Ejemplo de consulta:** “¿Cuál es la política de calidad de la empresa?”
- **Comportamiento:** El sistema busca en Qdrant los fragmentos más relevantes del documento de calidad indexado, los inyecta como contexto en el prompt y genera una respuesta precisa basada en el contenido real del documento.
- **Indicador visual:** RAG Buscando en documentos internos...

Las respuestas en modo RAG incluyen *referencias a las fuentes*: el nombre del documento, la página y la puntuación de relevancia de cada fragmento utilizado, proporcionando trazabilidad completa.

El sistema también detecta preguntas de seguimiento: si el usuario pregunta “¿Y cuándo fue la última revisión?” tras una consulta RAG, el sistema mantiene el modo RAG y utiliza el contexto de la conversación anterior para contextualizar la nueva búsqueda.

5.4.2 Búsqueda en Internet

Para consultas que requieren información actualizada o que exceden el ámbito de los documentos internos, el sistema ofrece búsqueda en internet.

- **Ejemplo de consulta:** “Busca en internet las últimas noticias sobre Docker”
- **Comportamiento:** El sistema ejecuta una búsqueda en DuckDuckGo, recupera los resultados más relevantes y genera una respuesta sintetizada con enlaces a las fuentes originales.
- **Indicador visual:** WEB Buscando en internet...

Las respuestas incluyen los enlaces directos a las fuentes consultadas en una sección de “Fuentes” al final del mensaje. El sistema detecta automáticamente consultas que necesitan información fresca (noticias, precios, resultados deportivos) y añade contexto temporal a la búsqueda.

5.4.3 Análisis de Páginas Web (Scraping)

Cuando el usuario incluye una URL en su mensaje, el sistema descarga y analiza el contenido de la página web automáticamente.

- **Ejemplo de consulta:** “Resume esta página: <https://ejemplo.com/articulo>”

- **Comportamiento:** El sistema abre la URL en un navegador automatizado (Playwright), espera a que se renderice todo el JavaScript, extrae el contenido textual limpio y genera un resumen o responde la pregunta del usuario basándose en el contenido extraído.
- **Indicador visual:** URL Analizando URL...

El contenido extraído queda almacenado en la memoria de sesión del usuario, lo que permite realizar preguntas de seguimiento como “¿Qué dice el tercer párrafo?” sin necesidad de volver a enviar la URL. Además, el usuario puede optar por indexar el contenido permanentemente en la colección `webs` de Qdrant para futuras consultas RAG.

5.4.4 Análisis de Imágenes (Visión/OCR)

El sistema permite analizar imágenes adjuntadas directamente en el chat.

- **Ejemplo de consulta:** [Adjuntar foto de un documento] “¿Qué pone en este documento?”
- **Comportamiento:** La imagen se envía a un modelo de visión multimodal (Qwen 2.5 VL en el despliegue actual) que describe, transcribe o analiza el contenido visual.
- **Indicador visual:** VISION Analizando imagen...

La última imagen analizada queda almacenada en la memoria de sesión, permitiendo preguntas de seguimiento como “¿Y cuál es la fecha que aparece?” sin necesidad de reenviar la imagen.

5.4.5 Consulta Legislativa del BOE

El sistema integra la capacidad de consultar el Boletín Oficial del Estado (BOE) directamente desde la interfaz de chat [50].

- **Ejemplo de consulta:** “Busca en el BOE la ley de protección de datos”
- **Comportamiento:** El sistema resuelve semánticamente el nombre coloquial de la ley a su identificador oficial del BOE, consulta la API de Datos Abiertos del BOE y devuelve un resumen estructurado de la ley solicitada. De forma complementaria, el proyecto incorpora un servidor MCP para este dominio, ya validado fuera de la interfaz principal.

- **Indicador visual:** BOE Consultando el BOE...

El sistema también soporta consultas por número de artículo (“dame el artículo 5 de la LOPD”) y consultas del sumario del día (“¿qué ha salido hoy en el BOE?”).

5.4.6 Listar Documentos Disponibles

El usuario puede consultar en cualquier momento qué documentos están indexados en el sistema.

- **Ejemplo de consulta:** “¿Qué documentos tienes?” o simplemente “/docs”
- **Comportamiento:** El sistema consulta las colecciones de Qdrant a las que el usuario tiene acceso y devuelve una lista organizada por tipo (documentos locales y webs guardadas), mostrando el nombre de cada documento.

5.4.7 Estado del Sistema

El usuario puede verificar el estado de la indexación y los procesos internos.

- **Ejemplo de consulta:** “¿Cómo va la indexación?”
- **Comportamiento:** El sistema consulta el servicio Indexer y devuelve información sobre el estado de sincronización con SharePoint, documentos pendientes y estadísticas de la base de conocimiento.

5.4.8 Ayuda y Capacidades

El usuario puede solicitar información sobre las capacidades del sistema.

- **Ejemplo de consulta:** “¿Qué puedes hacer?” o “ayuda”
- **Comportamiento:** El sistema devuelve un mensaje formateado con una descripción de todas las funcionalidades disponibles y ejemplos de uso.

5.4.9 Conversación General

Si el mensaje del usuario no coincide con ninguno de los modos especializados anteriores, el sistema funciona como un asistente conversacional general, respondiendo preguntas, manteniendo conversaciones y proporcionando información a partir del conocimiento general del modelo de lenguaje.

5.5 Escenarios de Uso Completos

Para ilustrar la experiencia del usuario de forma integral, se describen a continuación tres escenarios de uso representativos de las capacidades integradas:

5.5.1 Escenario 1: Consulta de un procedimiento corporativo

Un empleado del departamento de calidad necesita verificar un requisito específico:

1. El empleado accede al sistema e inicia sesión con sus credenciales corporativas.
2. Escribe: “¿Cuáles son los requisitos para la auditoría interna según el manual de calidad?”
3. El sistema detecta la intención RAG, busca en los documentos del departamento de calidad y responde con los requisitos específicos extraídos del manual, incluyendo referencias a la sección y página del documento original.
4. El empleado pregunta: “¿Y cada cuánto tiempo se debe realizar?”
5. El sistema aplica el *sticky mode* —mantiene automáticamente el modo de la consulta anterior ante preguntas de seguimiento breves—, conserva el contexto RAG y responde con la periodicidad especificada en el documento.

5.5.2 Escenario 2: Análisis de una normativa del BOE

Un responsable legal necesita revisar una disposición reciente:

1. Escribe: “Busca en el BOE el real decreto sobre teletrabajo”
2. El sistema consulta la API del BOE, localiza el real decreto correspondiente y devuelve un resumen estructurado con título, fecha de publicación, número de artículos y un extracto del contenido.
3. El responsable pregunta: “Dame el artículo 5”
4. El sistema recupera el texto completo del artículo solicitado.

5.5.3 Escenario 3: Análisis de una web tecnológica

Un ingeniero necesita evaluar una herramienta:

1. Escribe: “<https://docs.docker.com/compose/compose-file/> ¿Qué opciones de despliegue GPU existen?”

2. El sistema descarga la página, extrae el contenido y genera una respuesta centrada en las opciones de GPU del fichero Docker Compose.
3. El ingeniero pregunta: “Guarda esta web para futuras consultas”
4. El sistema indexa el contenido en la colección `webs` de Qdrant, disponible para consultas RAG posteriores.

5.6 Panel de Administración y Monitorización

Además de la interfaz de chat para usuarios regulares, el sistema ofrece herramientas de administración y monitorización para los administradores del sistema:

5.6.1 Dashboards de Grafana

Los administradores acceden a paneles de control interactivos en Grafana que muestran en tiempo real:

- **Métricas de uso:** Número de consultas por minuto, distribución de modos de operación (cuántas consultas RAG, web search, chat, etc.), tiempos de respuesta (percentiles p50, p95, p99) y tasa de errores.
- **Métricas de GPU:** Consumo de VRAM en tiempo real, temperatura del chip, utilización de los cores CUDA y potencia consumida. Esto permite detectar cuándo la GPU está al límite y ajustar la carga de modelos.
- **Métricas de infraestructura:** Estado de las bases de datos, conexiones activas, espacio disponible en disco.

5.6.2 Administración de la Base de Datos

pgAdmin 4 proporciona acceso directo a la base de datos PostgreSQL del sistema, permitiendo auditar las tablas de usuarios, conversaciones y sesiones almacenadas por OpenWebUI.

5.6.3 Gestión de Modelos

Desde la interfaz de administración de OpenWebUI, los administradores pueden:

- Visualizar los modelos disponibles en Ollama.
- Modificar la configuración de los Pipelines (Valves) sin reiniciar servicios.
- Gestionar usuarios y permisos.

Capítulo 6

Diseño e Implementación

Este capítulo constituye el núcleo técnico de mi Trabajo de Fin de Grado. La idea de fondo que conviene tener presente al leerlo es sencilla: el sistema no es una única aplicación escrita de principio a fin, sino un **conjunto de piezas integradas**. Algunas son herramientas de código abierto ya existentes que he reutilizado —empezando por las dos sobre las que se apoya todo lo demás: **Ollama**, que ejecuta los modelos de lenguaje en local, y **OpenWebUI**, que aporta la interfaz de chat—; otras son desarrollo propio —el *pipeline* de encaminamiento, el backend RAG, el servicio de indexación, el agente SQL y las integraciones con cada fuente—. El trabajo de este TFG consiste precisamente en diseñar esa arquitectura y en escribir el código que conecta unas piezas con otras hasta convertir componentes sueltos en un asistente corporativo coherente.

Por eso el capítulo se lee mejor como un recorrido: parte del sistema mínimo —un simple chat local montado sobre Ollama y OpenWebUI— y va explicando, paso a paso, qué pieza se añadió después y por qué, distinguiendo en cada caso qué es reutilización de una herramienta existente y qué es aportación propia. Para no recargar la lectura, el detalle de implementación más extenso —fragmentos de código, contratos de clase y flujos paso a paso— se traslada a los Anexos A–D, y aquí se mantiene la visión de arquitectura y las decisiones de diseño.

6.1 Filosofía de Diseño: Necesidad, Solución, Iteración

El sistema no se diseñó como un bloque monolítico, sino que emergió de un proceso iterativo de identificación de necesidades y acoplamiento progresivo de soluciones. Cada nueva funcionalidad surgió como respuesta a una limitación concreta detectada durante el uso del sistema:

1. Necesidad inicial: Un chat con un LLM local. *Solución:* Ollama + OpenWebUI [26], [27]. El sistema comenzó como una instalación básica de OpenWebUI conectada a Ollama, proporcionando una interfaz de chat con modelos locales.
2. Primera limitación: El modelo respondía de memoria, sin acceso a documentos internos. *Solución:* Se desarrolló un backend FastAPI con pipeline RAG y base de datos vectorial Qdrant. Esto exigió un mecanismo para interceptar los mensajes del usuario antes de que llegaran directamente al modelo: el Pipeline de OpenWebUI.
3. Segunda limitación: Los documentos cambiaban frecuentemente en SharePoint y la ingesta manual era inviable. *Solución:* Se creó el servicio Indexer con sincronización automática vía Microsoft Graph API [51].
4. Tercera limitación: Muchos PDFs corporativos eran escaneados y no contenían texto seleccionable. *Solución:* Se integró PaddleOCR con aceleración GPU en el Indexer.
5. Cuarta limitación: Los usuarios necesitaban acceder a información que no estaba en los documentos internos, como noticias, páginas web o normativa. *Solución:* Se añadieron los modos de búsqueda web, scraping y consulta del BOE. En la interfaz principal, la consulta normativa se resuelve mediante API; además, se desarrolló un servidor MCP como línea de integración futura.
6. Quinta limitación: Sin visibilidad sobre el rendimiento del sistema, era imposible detectar cuellos de botella. *Solución:* Se implementó el stack de observabilidad: métricas Prometheus en el backend, NVIDIA DCGM para GPU, dashboards Grafana y logs centralizados con Loki/Promtail [35], [36], [37].
7. Sexta limitación: Las respuestas idénticas saturaban la GPU innecesariamente. *Solución:* Se interpuso LiteLLM como proxy con caché en Redis.
8. Séptima limitación: El modelo no usaba la terminología corporativa adecuada. *Solución:* Se exploró el fine-tuning con LoRA sobre Qwen 2.5.
9. Octava limitación: Los analistas de datos necesitaban consultar tablas de datos estructurados sin aprender SQL. *Solución:* Se desarrolló un agente de traducción NL→SQL (`core/sql_agent.py`) que genera y ejecuta consultas `SELECT` seguras sobre PostgreSQL mediante LiteLLM, con validación por lista blanca y auto-corrección ante errores de sintaxis.

10. Novena limitación: El control de acceso mediante cabeceras HTTP era insuficiente para entornos multi-tenant (múltiples grupos o departamentos con documentación aislada) exigentes. *Solución*: Se implementó validación criptográfica de tokens JWT (*JSON Web Token*) de Azure AD (`core/auth.py`), con descarga dinámica de las claves públicas del proveedor (JWKS, *JSON Web Key Set*), activable con `AZURE_JWT_VALIDATION=true`.
11. Décima limitación: La expansión de consulta en el endpoint de búsqueda lanzaba el cálculo en paralelo pero nunca recogía el resultado: estaba efectivamente deshabilitada en producción sin que los tests lo detectaran. *Solución* (v2.2.0): se añadió la recogida asíncrona del futuro, se ejecutan búsquedas con las consultas expandidas y se incorporan sus resultados sin duplicados; además se limpiaron imports perezosos y logs de depuración en `core/retrieval.py`.
12. Undécima limitación: El pipeline RAG usaba siempre el mismo modelo y los mismos parámetros (`top_k=5`) para cualquier consulta. *Solución* (v2.3.0): se desarrolló el `QueryProcessor` con *intent detection* (clasifica en FAC-TUAL/PROCEDURAL/ANALYTICAL/CONVERSATIONAL y ajusta `top_k`), *smart model routing* (analíticas a `qwen2.5-32b`; el resto a JARVIS) y *query expansion* (reformula y fusiona resultados antes del reranker). El `MemoryManager` pasó a usar `qwen2.5-32b` para resumir conversaciones largas.

Esta aproximación iterativa —identificar una carencia, evaluar alternativas, integrar la solución y validar el resultado— se repitió a lo largo de los siete sprints del proyecto. Cada nueva pieza se acopló al ecosistema respetando la separación de responsabilidades y las interfaces HTTP estándar entre servicios, permitiendo sustituir cualquier componente sin afectar al resto.

6.2 Arquitectura General del Sistema

El sistema se compone de 15 servicios principales y varios servicios auxiliares de soporte y arranque, todos contenerizados con Docker, orquestados mediante Docker Compose [31] y aislados en una red virtual propia (**rag-network**). El flujo resumido de tecnologías se presentó en el Capítulo 3; aquí se detalla la función de cada servicio dentro de la arquitectura funcional.

Tabla 6.1: Servicios principales de la arquitectura y su función.

Servicio	Función	Puerto
NGINX	Proxy inverso, terminación TLS	8443 (externo)
OpenWebUI	Interfaz de usuario conversacional	8080 (interno)
Pipelines (JARVIS)	Agente encaminador inteligente	9099 (interno)
Backend (FastAPI)	Motor RAG, scraping, búsqueda, OCR	8000 (interno)
Indexer (FastAPI)	Sincronización SharePoint, ingesta	8001 (interno)
Ollama	Ejecución local de LLMs	11434 (interno)
LiteLLM	Proxy unificado de modelos	4000 (interno)
Qdrant	Base de datos vectorial	6333 (interno)
PostgreSQL	Base de datos relacional	5432 (interno)
Redis	Caché de respuestas	6379 (interno)
MCP-BOE Server	Consulta del BOE	8010 (interno)
MinIO	Almacenamiento de objetos (backup)	9000 (interno)
Prometheus	Recolección de métricas del sistema	9090 (interno)
Grafana	Dashboards de observabilidad	3000 (interno)
pgAdmin	Administración de PostgreSQL	5050 (interno)

Todos los servicios comparten una red Docker interna de tipo *bridge* denominada **rag-network**. Únicamente NGINX expone un puerto al exterior (8443), garantizando que ningún servicio interno es accesible directamente desde fuera del servidor. Los servicios que requieren aceleración GPU (Ollama, PaddleOCR) la reciben mediante la directiva `deploy.resources.reservations` de Compose y el `nvidia-container-toolkit` del host; los servicios más intensivos declaran además límites de CPU y memoria (`deploy.resources.limits`) para que ninguno monopolice los recursos del host.

En la validación final (26 de mayo de 2026), el despliegue convivía con otros servicios Docker en el mismo host. Para evitar colisiones de puertos se mantuvieron los puertos internos y se publicaron accesos locales diferenciados (p. ej. OpenWebUI

en localhost:3002, Backend en localhost:8002, Qdrant en localhost:6335). Esta separación no modifica la arquitectura interna, pero permite ejecutar el sistema completo sin detener otros servicios del entorno de desarrollo.

Stack Docker del entorno limpio

Servicios activos de TFG-RAG-Clean con puertos alternativos para la demo local.

SERVICIO	ESTADO	PUERTOS PUBLICADOS
openwebui tfg-openwebui	running (healthy)	0.0.0.0:3002->8080/tcp, [::]:3002->8080/tcp
rag-backend tfg-backend	running (healthy)	0.0.0.0:8002->8000/tcp, [::]:8002->8000/tcp
indexer tfg-indexer	running	0.0.0.0:8003->8001/tcp, [::]:8003->8001/tcp
qdrant tfg-qdrant	running (healthy)	0.0.0.0:6335->6333/tcp, [::]:6335->6333/tcp, 0.0.0.0:6336->6334/tcp, [::]:6336->6334/tcp
litellm tfg-litellm	running	0.0.0.0:4001->4000/tcp, [::]:4001->4000/tcp
ollama tfg-ollama	running (healthy)	0.0.0.0:11435->11434/tcp, [::]:11435->11434/tcp
pipelines tfg-pipelines	running	0.0.0.0:9100->9099/tcp, [::]:9100->9099/tcp
postgres tfg-postgres	running (healthy)	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
redis tfg-redis	running (healthy)	0.0.0.0:6380->6379/tcp, [::]:6380->6379/tcp
grafana tfg-grafana	running	0.0.0.0:3003->3000/tcp, [::]:3003->3000/tcp
prometheus tfg-prometheus	running	0.0.0.0:9091->9090/tcp, [::]:9091->9090/tcp
minio tfg-minio	running (healthy)	0.0.0.0:9002->9000/tcp, [::]:9002->9000/tcp, 0.0.0.0:9003->9001/tcp, [::]:9003->9001/tcp
nginx tfg-nginx	running	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp

Figura 6.1: Estado del stack Docker en la validación final. La captura muestra los servicios principales de TFG-RAG-Clean, su estado de ejecución y los puertos alternativos publicados para la demo local.

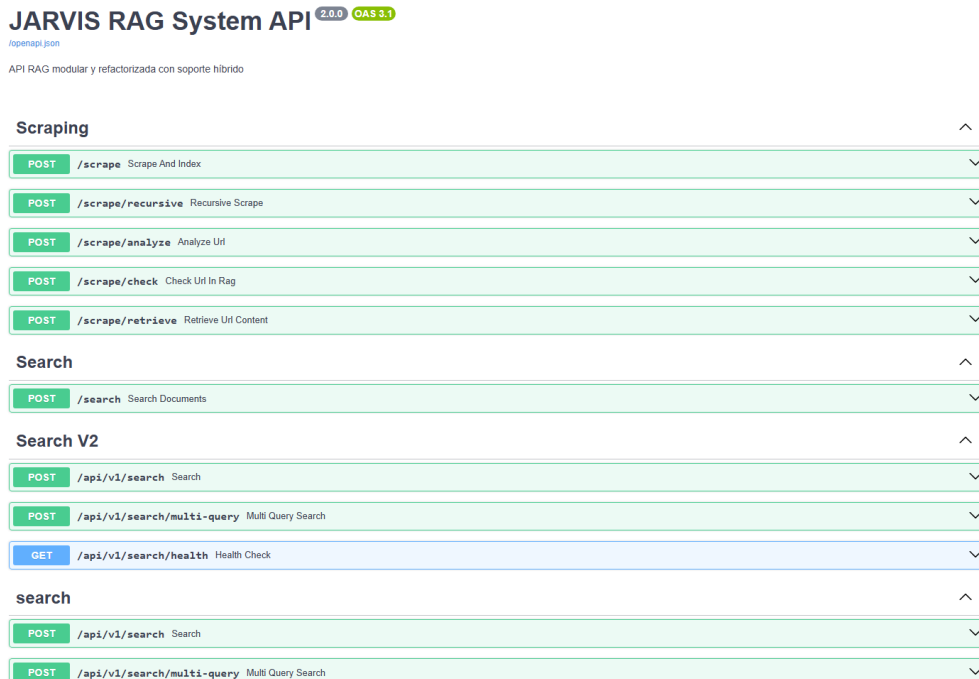


Figura 6.2: Documentación interactiva del Backend FastAPI en el entorno local de validación: API activa, endpoints de scraping, búsqueda y salud, y contrato OpenAPI expuesto.

6.2.1 Flujo de Datos End-to-End

Para ilustrar cómo se conectan las piezas, se describe el recorrido completo de una consulta RAG típica:

1. El **usuario** escribe su pregunta en el navegador (OpenWebUI).
2. **NGINX** recibe la petición HTTPS y la reenvía internamente a OpenWebUI (puerto 8080).
3. **OpenWebUI** envía el mensaje al pipeline JARVIS mediante HTTP POST al puerto 9099.
4. **JARVIS** analiza el texto, detecta la intención RAG, obtiene los departamentos autorizados del usuario y construye una petición al Backend.
5. El **Backend** genera el embedding de la consulta con SentenceTransformer, ejecuta una búsqueda híbrida en Qdrant y recupera los k fragmentos más relevantes.
6. El Backend compone un *prompt* estructurado con el contexto recuperado y lo envía a LiteLLM.

7. **LiteLLM** verifica si existe una respuesta cacheada en Redis. Si no, delega al modelo en Ollama.
8. **Ollama** ejecuta la inferencia en GPU y devuelve los tokens generados en streaming.
9. La respuesta fluye de vuelta: LiteLLM → Backend → JARVIS → OpenWebUI → NGINX → navegador.

Todo el flujo se completa típicamente en menos de 3.5 segundos, con el primer token visible para el usuario en menos de 1.2 segundos gracias al streaming.

El aspecto real de esta interfaz en funcionamiento, con la respuesta y sus fuentes, se mostró en la Figura 5.1 del Capítulo 5.

6.3 Desarrollo Propio vs. Herramientas Externas

Una pregunta fundamental en cualquier proyecto de integración es: ¿qué parte del trabajo es desarrollo original y qué parte es configuración de herramientas existentes? La Tabla 6.2 clasifica cada componente del sistema según su origen.

Tabla 6.2: Resumen sintético de reutilización y desarrollo dentro del TFG.

Componente	Reutilizado	Extendido	Desarrollado en el TFG	Integrado	Papel en el sistema
OpenWebUI	✓	✓	×	✓	Interfaz conversacional web, persistencia de chats y base para autenticación corporativa.
Pipeline JARVIS	×	×	✓	✓	Encaminamiento de intenciones, memoria de sesión y conexión entre interfaz y backend.
Backend RAG	×	×	✓	✓	Recuperación documental, scraping, búsqueda web, API BOE y lógica principal del sistema.
Indexer	×	×	✓	✓	Sincronización con SharePoint, OCR e ingesta automatizada de documentos.
Ollama y LiteLLM	✓	✓	×	✓	Inferencia local, alias de modelos, caché y desacoplo entre servicios y modelos.
Qdrant	✓	✓	×	✓	Almacenamiento vectorial y recuperación documental con filtros por colección.
Servidor MCP-BOE	×	×	✓	✓	Consulta normativa estructurada y prueba de integración MCP.
Prometheus, Grafana y DCGM	✓	✓	×	✓	Observabilidad operativa del backend, la base de datos y la GPU.

En resumen, el trabajo de desarrollo propio abarca alrededor de 24.000 líneas de código Python distribuidas entre el Backend (~17.400 líneas), el Pipeline JARVIS (~4.400), el Indexer (~1.700) y el servidor MCP-BOE (~490), además de toda la configuración declarativa del ecosistema Docker (Compose, LiteLLM, NGINX, Prometheus y los dashboards de Grafana). Las herramientas externas proporcionan las capacidades de infraestructura (base de datos, motor LLM, OCR, navegador), pero la lógica de negocio, la orquestación y la inteligencia del sistema son enteramente originales.

6.4 El Mecanismo Pipeline de OpenWebUI (Hook)

El Pipeline de OpenWebUI es el punto de conexión fundamental entre la interfaz de usuario y todo el backend del sistema [26]. Sin este mecanismo, OpenWebUI sería simplemente una interfaz para chatear directamente con Ollama. El Pipeline es lo que convierte esa interfaz genérica en un asistente corporativo inteligente.

En ingeniería de software, un *hook* es un punto de extensión que permite inyectar código personalizado en un flujo de ejecución predefinido sin modificar el sistema original. En OpenWebUI, el *hook* se materializa en el sistema de Pipelines: cuando un usuario envía un mensaje, en lugar de reenviarlo directamente a Ollama, OpenWebUI lo envía primero al servicio Pipeline, que procesa el mensaje, ejecuta la lógica necesaria y devuelve la respuesta. Desde la perspectiva de OpenWebUI, el Pipeline *es* el modelo.

La conexión se establece por configuración: el Pipeline se despliega como contenedor que expone una API HTTP en el puerto 9099, y OpenWebUI se configura con `OPENAI_API_BASE_URLS` apuntando a él. Al arrancar, OpenWebUI descubre el Pipeline por su nombre (JARVIS) y lo muestra como un modelo seleccionable. El detalle de implementación —el contrato de la clase `Pipeline`, el método `pipe()` con respuesta en *streaming* (`yield`) y el sistema *Valves* de configuración dinámica desde la interfaz de administración— se recoge en el Anexo A.

6.5 Agente Encaminador JARVIS: Inteligencia Central

El agente JARVIS es el módulo individual más extenso del proyecto, con unas 4.400 líneas de código Python concentradas en un único fichero (`jarvis.py`). Su función es analizar cada mensaje del usuario y decidir a cuál de los modos de operación derivarlo. La detección de intenciones se implementa en el método `_detect_intent()`, que evalúa el mensaje contra un conjunto de reglas heurísticas (coincidencia de palabras clave y expresiones regulares) en un *orden de prioridad estricto*: primero las

condiciones más específicas (archivos adjuntos, imágenes, URLs) y después las más genéricas (keywords de RAG, búsqueda web, conversación). La Tabla 6.3 resume las categorías, su prioridad y los criterios de activación.

Tabla 6.3: Categorías de intención y su prioridad de evaluación.

Prioridad	Modo	Criterio de activación
0	Decisión pendiente	Existe una decisión de URL esperando respuesta del usuario
1	Archivo adjunto	Se detecta un PDF o documento en el cuerpo de la petición
2	Visión/OCR	Imagen adjunta en el mensaje actual
3	Referencia a imagen	Mención a “la imagen” con imagen previa en memoria
4	Referencia a web	Mención a “la web” o “la página” con contenido web en memoria
5	Ayuda	Keywords: “qué puedes hacer”, “ayuda”, “help”
6	Listar documentos	Commands: “/docs”, “/listar”; keywords: “qué documentos tienes”
7	BOE / Legal	Keywords: “busca en el BOE”, “ley de”, “real decreto”
8	RAG Documental	Keywords: “documento”, “política”, “manual”, “procedimiento”
9	URL detectada	Expresión regular que detecta <code>https://...</code> en el mensaje
10	Búsqueda web	Keywords: “busca en internet”, “noticias”, “precio”
11	Estado del sistema	Keywords: “cómo va”, “estado”, “indexación”
12	Sticky Mode	Seguimiento contextual del modo de la pregunta anterior
13	Chat general	Default: cualquier mensaje que no active otro modo



Figura 6.3: Respuesta real del modo de listado documental. JARVIS consulta el estado del indexador y devuelve los documentos disponibles, junto con la información reciente de ingesta.

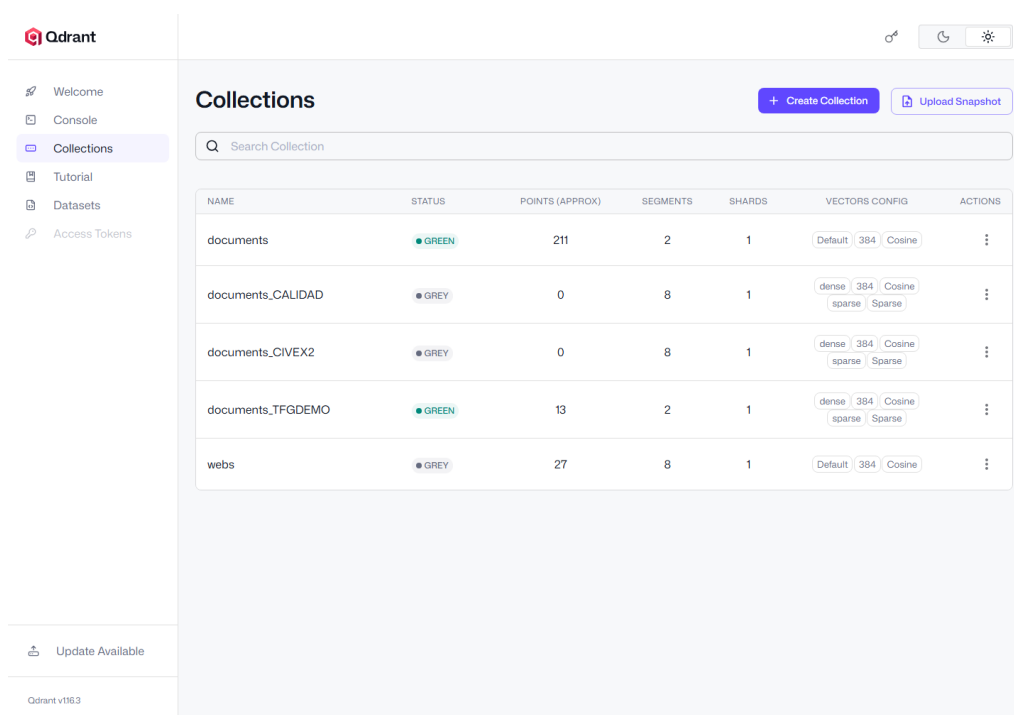
Reglas frente a un router basado en LLM. La decisión de qué hacer con cada mensaje se toma, por tanto, mediante **reglas deterministas**, sin delegarla a un modelo de lenguaje. Es una elección de diseño deliberada: el enrutamiento por reglas es prácticamente instantáneo (~ 10 ms), no consume tokens ni GPU, es reproducible y se puede validar de forma sistemática (alcanzó un 92 % de acierto, véase el Capítulo 8). A ello se añade que el modelo principal (**rag-qwen-ft**) se expone sin *function calling*, por lo que no podría actuar como router fiable. La alternativa —delegar la decisión en un LLM que actúe como *agente router*, enfoque adoptado en la variante empresarial derivada de este TFG— es más robusta ante fraseos inesperados, pero añade latencia, coste por consulta y pérdida de determinismo. Por qué no se ha integrado aquí y cómo podría hacerse (incluido un esquema híbrido) se detalla en el Anexo A.

Más allá del encaminamiento, JARVIS incorpora tres mecanismos de contexto: el *Sticky Mode* (mantiene el modo de la pregunta anterior ante consultas de seguimiento breves y ambiguas), la **memoria de sesión por usuario** (recuerda la última imagen y la última web analizadas, y gestiona decisiones pendientes) y el **control de acceso departamental** (mapea los grupos de Azure AD del usuario a colecciones de Qdrant y filtra la búsqueda). El detalle de estos tres mecanismos, con ejemplos de código, se recoge en el Anexo A.

6.6 Backend RAG: Motor de Búsqueda Semántica

El Backend constituye el motor central de procesamiento del sistema. Implementado en FastAPI, fue objeto de una refactorización significativa: inicialmente era un único fichero `main.py` de más de 1.900 líneas que se dividió en una arquitectura modular con paquetes separados para esquemas (`schemas/`), servicios (`services/`), endpoints (`api/`), procesamiento (`processing/`) e integraciones (`integrations/`).

La **ingesta** de un documento sigue seis etapas: recepción (API, Indexer o scraper), detección de tipo (texto nativo vs. escaneado), extracción de texto (`pdfplumber`, PaddleOCR o la cadena Trafilatura/Readability/BeautifulSoup), *chunking* semántico (fragmentos de unos 500 tokens con 50 de solapamiento, $\sim 10\%$), generación de *embeddings* (vector denso de 384 dimensiones con `paraphrase-multilingual-MiniLM-L12-v2`) y almacenamiento en Qdrant con metadatos. El procedimiento completo se detalla en el Anexo B.



NAME	STATUS	POINTS (APPROX)	SEGMENTS	SHARDS	VECTORS CONFIG	ACTIONS
documents	GREEN	211	2	1	Default 384 Cosine	
documents_CALIDAD	GREY	0	8	1	dense 384 Cosine sparse Sparse	
documents_CIVEX2	GREY	0	8	1	dense 384 Cosine sparse Sparse	
documents_TFGDEMO	GREEN	13	2	1	dense 384 Cosine sparse Sparse	
webs	GREY	27	8	1	Default 384 Cosine	

Figura 6.4: Estado de las colecciones en Qdrant durante la demo final. La colección `documents_TFGDEMO` —documentación sintética y segura del propio TFG— aparece separada del resto, con 13 puntos y configuración híbrida `dense+sparse`.

6.6.1 Búsqueda Híbrida

La recuperación de contexto combina tres etapas complementarias:

1. **Búsqueda densa (HNSW)**: la consulta se transforma en un embedding de 384 dimensiones y se buscan los fragmentos más próximos en el espacio vectorial mediante el índice HNSW de Qdrant.
2. **Búsqueda dispersa (BM25)**: se complementa con vectores dispersos generados con Qdrant/bm25 (vía `fastembed`), que favorece coincidencias léxicas exactas —identificadores, códigos, siglas o artículos legales— que la búsqueda semántica podría no capturar.
3. **Reordenación con reranker**: los candidatos se fusionan y se reordenan con un *Cross-Encoder* (`cross-encoder/ms-marco-MiniLM-L-6-v2`), que evalúa conjuntamente el par (consulta, fragmento) para obtener una relevancia más precisa antes de seleccionar los k fragmentos finales (típicamente $k = 5$).

El *prompt* enviado al LLM combina un *system prompt* restrictivo —“responde únicamente con el contexto proporcionado, no inventes”— con una temperatura baja (0,1–0,2), lo que elimina prácticamente las alucinaciones en modo RAG. La construcción exacta del *prompt* se muestra en el Anexo B.

6.7 Extracción Web, OCR e integraciones

El sistema añade varias fuentes de información más allá de los documentos internos, cuyo detalle de implementación se recoge en los anexos:

- **Scraping y búsqueda web** (Anexo B): un motor con Playwright renderiza páginas dinámicas (JavaScript) y una cadena de *fallback* (Trafilatura → Readability → BeautifulSoup) extrae el contenido principal. La búsqueda en internet usa `duckduckgo_search` con *fallback* a scraping HTML.
- **OCR** (Anexo B): PaddleOCR con aceleración GPU —y *fallback* automático a CPU— procesa los PDFs escaneados, con paralelización mediante Ray durante la ingesta.
- **SharePoint** (Anexo C): el Indexer sincroniza cada 5 minutos mediante MSAL y Microsoft Graph, con estrategia *delta* incremental para no reprocesar documentos.

6.8 Integración con el Boletín Oficial del Estado (BOE)

La integración con el BOE se implementa mediante un servidor MCP (*Model Context Protocol*) construido con `fastmcp`, que transforma peticiones en lenguaje natural en llamadas estructuradas a la API de Datos Abiertos del BOE. Esta integración no es una búsqueda web genérica, y merece una justificación específica: en el tipo de empresa para la que se diseña el sistema, el acceso a normativa legal (legislación laboral, protección de datos, contratación pública) es frecuente y tiene consecuencias directas. Un error en la interpretación de una norma puede traducirse en incumplimientos o sanciones.

El BOE presenta características que lo distinguen como caso de uso propio: las leyes se identifican por códigos oficiales (p. ej. `BOE-A-2018-16673`) y no por títulos que el usuario recuerde; un mismo texto se referencia coloquialmente de muchas formas (“la LOPD”, “la ley de privacidad”); la información relevante suele estar en artículos numerados concretos; y tanto la exactitud como la fuente son cruciales. Por ello se implementó una capa de resolución semántica que traduce el lenguaje del usuario al identificador normativo correcto antes de invocar la API oficial. El mecanismo de mapeo, las razones técnicas por las que el flujo web emplea la API y no MCP —el modelo principal no usa *function calling* y el pipeline intercepta y enruta cada consulta antes de que el modelo pueda invocar herramientas— y una valoración de si conviene habilitar MCP de forma activa se detallan en el Anexo C.

6.9 Seguridad Perimetral y Autenticación

La seguridad del sistema se organiza en capas. **NGINX** actúa como primera barrera y único servicio expuesto al exterior: termina el TLS (los microservicios internos hablan en texto plano dentro de la red aislada), hace *proxy pass* preservando cabeceras de trazabilidad y habilita el *upgrade* a WebSocket necesario para el streaming. La autenticación de usuarios se delega en **Azure AD vía OpenID Connect (OIDC)**: los usuarios entran con sus credenciales corporativas y los atributos del token (nombre, correo, grupos) alimentan el control de acceso departamental.



Iniciar Sesión en europav-IA (Open WebUI)

Email
Ingresa tu correo electrónico

Contraseña
Ingresa tu contraseña

Iniciar Sesión

¿No tienes una cuenta? [Crear una Cuenta](#)

o

Continuar con SSO

Figura 6.5: Pantalla real de autenticación de OpenWebUI en el despliegue local: inicio de sesión clásico y acceso mediante SSO corporativo.

Además de la autenticación perimetral, el Backend implementa una segunda capa de **validación criptográfica de tokens JWT** (`core/auth.py`) para clientes externos y herramientas MCP: descarga las claves públicas JWKS del *tenant*, verifica cada token Bearer y mapea los grupos a colecciones autorizadas, devolviendo HTTP 403 si el acceso no es válido. Se activa con `AZURE_JWT_VALIDATION=true`. Finalmente, el **aislamiento de red** (`rag-network`, tipo *bridge*) impide el acceso externo directo a PostgreSQL, Redis, Qdrant u Ollama. El flujo JWT paso a paso se detalla en el Anexo C.

6.10 Observabilidad, caché y almacenamiento

La monitorización respondió a una necesidad real: durante las primeras fases no existía visibilidad sobre el rendimiento, la saturación de la GPU ni los patrones de uso, y la optimización era intuitiva en lugar de basada en datos. El stack adoptado combina **Prometheus** (métricas del backend: latencias y percentiles, throughput, distribución de modos, errores), **NVIDIA DCGM Exporter** (VRAM, utilización, temperatura, potencia de la GPU), **Grafana** (tres dashboards: backend, GPU y base de datos) y **Loki + Promtail** (agregación de logs de todos los contenedores, consultables junto a las métricas en Grafana) [35], [36], [37]. El detalle de métricas, telemetría y dashboards está en el Anexo C.

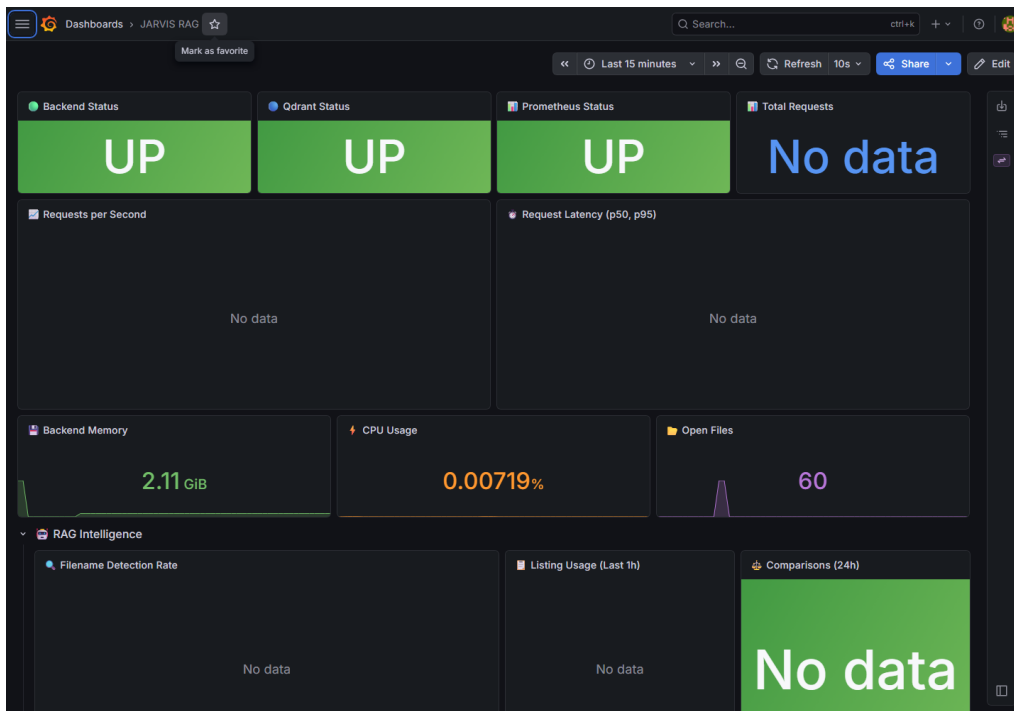


Figura 6.6: Dashboard principal de Grafana durante la validación final: estado operativo del backend, Qdrant y Prometheus, además de métricas de memoria, CPU y ficheros abiertos.

Esta observabilidad condujo a decisiones concretas de optimización: se validó que $\sim 35\%$ de las consultas repetidas se beneficiaban de la caché, se ajustó el valor de k al detectar que más de 5 fragmentos no mejoraban la respuesta, y se reprogramó la ingesta OCR a horarios de baja actividad al ver que monopolizaba la GPU.

En cuanto al rendimiento y el almacenamiento, **Redis** actúa como caché integrada con LiteLLM: almacena un hash de cada consulta junto con su respuesta (TTL de 1 hora), con una tasa de aciertos observada de $\sim 35\%$ que reduce la carga sobre la GPU. **MinIO** proporciona almacenamiento de objetos compatible con S3 que preserva una réplica aislada de los documentos binarios ingeridos, permitiendo reindexar desde cero sin volver a las fuentes originales (SharePoint, web).

6.11 Agente SQL: Consultas en Lenguaje Natural sobre Datos Estructurados

El pipeline RAG estándar opera solo sobre texto no estructurado, pero muchos sistemas empresariales guardan información crítica en tablas relacionales (métricas, inventarios, datos financieros). Para consultarlas sin saber SQL se desarrolló el agente SQL (`backend/app/core/sql_agent.py`), que sigue tres pasos: (1) **inspección del**

esquema de PostgreSQL para inyectarlo como contexto; (2) **generación** de una consulta `SELECT` con LiteLLM, instruyendo al modelo a producir solo lecturas; y (3) **ejecución con auto-corrección**, reintentando con el mensaje de error como contexto (hasta dos veces). La consulta se valida contra una lista blanca (solo `SELECT/WITH/EXPLAIN`) y se ejecuta con *timeout*; los mecanismos de seguridad se detallan en el Anexo B. El endpoint unificado `POST /api/v1/query` enruta automáticamente entre RAG y SQL según la naturaleza de la consulta, de forma transparente para el usuario.

6.12 Fine-Tuning con LoRA

Para adaptar el modelo al dominio corporativo se exploró el ajuste fino con LoRA (*Low-Rank Adaptation*), que inserta matrices de rango reducido en las capas de atención sin modificar los pesos originales, reduciendo los parámetros entrenables a una fracción y haciendo viable el entrenamiento con una sola GPU. El modelo resultante (`rag-qwen-ft:latest`) es el que sirve de base a JARVIS. El procedimiento completo —generación del dataset, entrenamiento y evaluación— se detalla en el Anexo D.

Una conclusión relevante del proyecto es que, para tareas de RAG puras, el *fine-tuning* del LLM resultó *menos* decisivo que un buen *prompt* restrictivo combinado con una recuperación de calidad (chunking, embeddings, búsqueda híbrida y reranker). El ajuste fino sí aportó valor en la coherencia terminológica y en la adherencia al estilo de respuesta corporativo, pero la inteligencia del sistema descansa sobre todo en la calidad del contexto recuperado, no en la modificación de los pesos del modelo.

Capítulo 7

Proceso de desarrollo del proyecto

Este capítulo describe cómo se ha construido el proyecto a lo largo del tiempo. Tras situar la organización del código, el grueso del capítulo —y su verdadero contenido— es el recorrido **sprint a sprint**: qué se hizo en cada etapa, qué tecnologías se eligieron frente a qué alternativas y qué lecciones dejó cada una. El balance cuantitativo del proyecto (horas dedicadas, planificación temporal, presupuesto y riesgos) se recoge en el Capítulo 9, junto al resto de conclusiones.

El desarrollo se ha conducido siguiendo un marco de trabajo ágil (*Agile*), caracterizado por entregas iterativas e incrementales. Dada la naturaleza cambiante del ámbito de la Inteligencia Artificial y de los Modelos de Lenguaje Grande (LLMs), la flexibilidad para revisar decisiones técnicas, sustituir herramientas o ajustar el encaminamiento resultó especialmente importante durante el desarrollo.

Se estableció un ciclo de vida con las siguientes fases:

1. **Elicitación y Análisis de Requisitos:** Identificación de fuentes de información clave (SharePoint, páginas web corporativas y el BOE), así como las restricciones de privacidad requeridas por un despliegue puramente *on-premise*.
2. **Diseño de Arquitectura:** Planificación de la infraestructura basada en microservicios contenerizados y selección del stack tecnológico idóneo.
3. **Implementación Incremental:** Programación progresiva de los módulos de ingesta, OCR, embeddings, búsqueda de Qdrant, backend y, por último, de la interfaz de usuario de OpenWebUI.
4. **Pruebas y Validación:** Validación funcional del encaminamiento inteligente y de las capacidades del RAG mediante métricas comparativas empíricas y experimentación cualitativa.

El trabajo se dividió en **siete sprints** de aproximadamente un mes de duración, desarrollados entre octubre de 2025 y mayo de 2026, a los que siguió una fase final de documentación, redacción de la memoria y difusión del proyecto hasta julio de 2026. Cada sprint se centró en un módulo funcional del sistema y terminó con un entregable operativo. La distribución temporal completa (diagrama de Gantt) y las horas dedicadas a cada sprint se presentan en la Sección 9.5.

7.1 Organización del Código

Antes de describir cada sprint conviene tener presente la estructura del repositorio del proyecto, porque cada directorio se corresponde de forma bastante directa con el sprint en el que nació. Esta correspondencia entre organización del código y evolución temporal facilita tanto el mantenimiento como la lectura del historial del proyecto:

```
TFG-RAG-URJC/
|-- backend/          <- Servicio FastAPI (sprints 2, 4, 6 y 7)
|   '-- app/          <- api, core (RAG, SQL, QueryProcessor),
|                      integrations, processing, storage...
|-- services/
|   |-- ollama/        <- Motor de inferencia local (sprint 1)
|   |-- litellm/       <- Proxy unificado de modelos (sprint 1)
|   |-- indexer/       <- Sincronizacion SharePoint+OCR (sprint 3)
|   '-- openwebui/     <- Interfaz y pipeline JARVIS (sprint 4)
|-- mcp-boe-server/   <- Servidor MCP del BOE (sprint 5)
|-- scripts/          <- Fine-tuning, datasets, backups (spr. 5)
|-- config/           <- NGINX, Prometheus, Grafana (spr. 1 y 5)
|-- database/         <- Esquema inicial PostgreSQL (sprint 1)
|-- tests/            <- Pruebas del sistema (transversal)
|-- docs/             <- Documentacion tecnica (transversal)
|-- github_pages/     <- Pagina web del proyecto
'-- memoria/          <- Esta memoria (LaTeX)
```

Listing 7.1: Estructura del repositorio y sprint en el que se desarrolló cada componente.

7.2 Los Sprints en Detalle

A continuación se define cada sprint siguiendo un esquema común: descripción y objetivo, tareas realizadas, tecnologías elegidas frente a las alternativas valoradas, y estado final con las lecciones aprendidas. Todos los sprints finalizaron con su entregable operativo, por lo que el estado se detalla junto a las lecciones de cada uno.

7.2.1 Sprint 1 (octubre de 2025): Infraestructura base

Descripción. Construir los cimientos del sistema: la plataforma de contenedores, los servicios de datos y el motor de inferencia local, validando que era viable ejecutar un LLM útil en el hardware disponible.

Tareas. Definición del `docker-compose` inicial y de la red interna aislada; despliegue de PostgreSQL con su esquema inicial (`database/init.sql`), Qdrant y Redis; instalación de Ollama con los primeros modelos (Llama 3.1) y del proxy LiteLLM; pruebas de concepto de chat local.

Tecnologías y alternativas valoradas.

- **Ollama** frente a `llama.cpp` puro o vLLM: Ollama aporta gestión de modelos, carga/descarga automática de VRAM y una API estable con mínima configuración. vLLM ofrece mayor rendimiento bruto, pero está orientado a servir un único modelo de forma intensiva; aquí se necesitaba rotar varios modelos sobre una sola GPU.
- **Qdrant** frente a pgvector, Chroma o Pinecone: se buscaba una base vectorial *on-premise* con índice HNSW, colecciones separadas y filtrado por metadatos (la base del control de acceso por departamento). Pinecone se descartó por ser un servicio en la nube, contrario al requisito de soberanía del dato.
- **Docker Compose** frente a Kubernetes: para un despliegue de nodo único, Kubernetes añadía complejidad operativa sin beneficio real.

Estado y lecciones. Completado. La lección central llegó ya en este sprint: la GPU es un recurso compartido y escaso, y esa restricción condicionó todas las decisiones posteriores (cuantización, modelos de embeddings en CPU, carga bajo demanda).

7.2.2 Sprint 2 (noviembre de 2025): Backend y pipeline RAG

Descripción. Desarrollar el motor del sistema: un backend propio con el pipeline RAG completo, desde la ingesta del documento hasta la respuesta con citas.

Tareas. API REST con FastAPI; ingesta de PDFs con `pdfplumber`; *chunking* con solapamiento (500 tokens, solapamiento de 50); generación de embeddings con `paraphrase-multilingual-MiniLM-L12-v2` en CPU; búsqueda semántica en Qdrant; construcción del prompt restrictivo con citas de fuente y página.

Tecnologías y alternativas valoradas.

- **FastAPI** frente a Flask o Django: asincronía nativa (esencial para no bloquear durante inferencias largas), validación con Pydantic y documentación OpenAPI generada automáticamente.
- **Pipeline RAG propio** frente a LangChain o LlamaIndex: los *frameworks* aceleran el prototipo, pero ocultan los pasos intermedios. Implementar el pipeline a mano dio control fino sobre chunking, búsqueda y prompt, y sobre todo trazabilidad completa para depurar; esa comprensión del flujo es además parte del valor académico del trabajo.
- **Embeddings en CPU** frente a GPU: el modelo MiniLM es lo bastante ligero para ejecutarse en CPU, reservando la VRAM para los modelos generativos.

Estado y lecciones. Completado. La calidad del RAG depende tanto de la ingesta (tamaño de fragmento, solapamiento, limpieza) como del propio modelo generativo: gran parte de las mejoras posteriores vinieron de recuperar mejor, no de generar mejor.

7.2.3 Sprint 3 (diciembre de 2025): SharePoint y OCR

Descripción. Conectar el sistema a las fuentes documentales corporativas reales: la biblioteca SharePoint de la organización y los documentos escaneados.

Tareas. Servicio Indexer con sincronización periódica (cron cada 5 minutos); autenticación MSAL con flujo de credenciales de cliente y descarga incremental vía Microsoft Graph API; motor OCR PaddleOCR con aceleración GPU y *fallback* automático a CPU; paralelización del OCR con Ray.

Tecnologías y alternativas valoradas.

- **PaddleOCR** frente a Tesseract: mejor precisión sobre los documentos escaneados reales del entorno de validación y soporte GPU nativo.
- **Sondeo incremental** frente a *webhooks* de Graph: los webhooks exigen exponer un endpoint público accesible desde Microsoft, incompatible con un despliegue cerrado; el sondeo cada pocos minutos resultó suficiente para la frecuencia de cambio real de los documentos.

Estado y lecciones. Completado. Al paralelizar la ingesta apareció una *race condition* en la generación de embeddings que hubo que serializar; y quedó claro que las fuentes corporativas cambian constantemente, lo que convirtió la sincronización incremental en pieza central del diseño.

7.2.4 Sprint 4 (enero de 2026): Agente JARVIS, scraping y búsqueda web

Descripción. Dar al sistema su cara visible y sus capacidades web: el pipeline JARVIS sobre OpenWebUI, que enruta cada consulta al modo adecuado, y la extracción de contenido de internet.

Tareas. Desarrollo del pipeline JARVIS como *hook* de OpenWebUI con detección de intenciones por reglas y sistema de *valves* de configuración; scraping con Playwright (renderizado JavaScript completo) y cadena de extracción Trafilatura → Readability → BeautifulSoup; búsqueda en internet con DuckDuckGo.

Tecnologías y alternativas valoradas.

- **OpenWebUI con pipeline propio** frente a interfaz web desarrollada desde cero: OpenWebUI ya resuelve autenticación, historial, streaming y gestión de usuarios; el valor diferencial estaba en la lógica de enrutamiento, no en reconstruir un chat.
- **Playwright** frente a peticiones HTTP con BeautifulSoup: gran parte de las páginas corporativas y de noticias actuales requieren ejecutar JavaScript para mostrar su contenido.
- **Router por reglas** frente a un LLM decisor: las reglas son instantáneas, gratuitas y deterministas, y el modelo local desplegado no ofrecía *function calling* fiable. Esta decisión se analiza en detalle en el Capítulo 6.

Estado y lecciones. Completado. Dos incidencias de despliegue dejaron lección: un pipeline duplicado por el mecanismo de descubrimiento de OpenWebUI y un script de inicialización de Ollama roto por los finales de línea CRLF al editar desde Windows; ambas reforzaron la necesidad de probar el despliegue completo desde cero, no solo el código.

7.2.5 Sprint 5 (febrero de 2026): BOE, *fine-tuning* y observabilidad

Descripción. Ampliar el sistema en tres frentes: la fuente normativa oficial (BOE), la adaptación del modelo al dominio mediante *fine-tuning* y la monitorización del conjunto.

Tareas. Servidor MCP del BOE e integración funcional por API; generación de un dataset sintético de preguntas y respuestas a partir de los documentos indexados en Qdrant; entrenamiento de un adaptador LoRA (PEFT, bitsandbytes, TRL) que dio lugar a *rag-qwen-ft*, el modelo JARVIS; despliegue del stack de observabilidad

(Prometheus, Grafana, exportador DCGM para GPU, y posteriormente Loki y Promtail para logs).

Tecnologías y alternativas valoradas.

- **LoRA** frente a *fine-tuning* completo: ajustar todos los pesos de un modelo de 7B parámetros excede la VRAM disponible; LoRA entrena una fracción mínima de parámetros con resultados comparables para adaptación de dominio.
- **MCP** frente a integración directa por API: se implementaron ambas. MCP se adoptó como apuesta de estandarización futura, pero la interfaz web principal no lo soporta de forma nativa, por lo que la vía de producción siguió siendo la API (la valoración completa está en el Anexo C).
- **Prometheus y Grafana** frente a soluciones de monitorización en la nube: coherencia con el requisito *on-premise* del proyecto.

Estado y lecciones. Completado. La lección más importante de todo el proyecto salió de aquí: el *fine-tuning* rindió resultados modestos para RAG, mientras que las mejoras de recuperación (búsqueda híbrida y reranking) tuvieron un impacto muy superior. Esto reorientó el esfuerzo restante hacia la calidad de la recuperación.

7.2.6 Sprint 6 (marzo–abril de 2026): Agente SQL, seguridad y v2.2.0

Descripción. Incorporar la consulta de datos estructurados en lenguaje natural y endurecer la seguridad del sistema, además de corregir defectos detectados en producción.

Tareas. Agente NL→SQL sobre PostgreSQL con lista blanca de operaciones (SELECT) y auto-corrección ante errores; validación de tokens JWT de Azure AD contra JWKS en el backend; diagnóstico y corrección de la expansión de consultas asíncrona (versión 2.2.0).

Tecnologías y alternativas valoradas.

- **Validación por lista blanca y expresiones regulares** frente a confiar en las instrucciones del prompt: un LLM puede ser inducido a generar SQL destructivo; la validación programática actúa como defensa en profundidad independiente del modelo.
- **Validación JWT en el backend** frente a delegar la autenticación en la interfaz: cada microservicio verifica la identidad por sí mismo, de modo que un fallo en la capa de presentación no expone los datos.

Estado y lecciones. Completado. La incidencia más formativa: la expansión de consultas llevaba tiempo desactivada por un defecto de configuración sin que ninguna prueba lo detectara. La lección es que los tests deben cubrir también la configuración efectiva del sistema en ejecución, no solo la lógica del código.

7.2.7 Sprint 7 (mayo de 2026): QueryProcessor y v2.3.0

Descripción. Cerrar el sistema con la capa de orquestación inteligente: decidir, para cada consulta, qué modelo y qué estrategia de recuperación usar.

Tareas. Módulo `QueryProcessor` con detección de intención, encaminamiento inteligente entre modelos (JARVIS para consultas factuales y procedimentales; Qwen 2.5 32B para las analíticas) y expansión de consultas multivariante; summarización del historial de conversación con el modelo de 32B en el `MemoryManager` (versión 2.3.0).

Tecnologías y alternativas valoradas.

- **Encaminamiento por intención entre dos modelos** frente a usar siempre el modelo mayor: las pruebas comparativas mostraron que el modelo ajustado de 7B domina en recuperación precisa con citas mientras que el de 32B destaca en razonamiento analítico; usar cada uno donde sobresale mejora calidad y latencia a la vez.

Estado y lecciones. Completado. El modelo más grande no siempre es la mejor opción: adaptar la elección del modelo a la intención de la consulta resultó más eficiente que cualquier elección fija.

Capítulo 8

Resultados y Evaluación

Este capítulo presenta los resultados obtenidos tras el despliegue del sistema en un entorno real. Se analizan las métricas de rendimiento recopiladas por el stack de observabilidad, se describen las pruebas funcionales realizadas sobre cada modo de operación y se valora cualitativamente la calidad de las respuestas del sistema.

8.1 Métricas de Rendimiento

El sistema se sometió a pruebas de rendimiento sostenidas en un entorno real durante un período de 30 días. Además, las métricas se recopilaron automáticamente mediante la integración con Prometheus y los paneles configurados en Grafana [35], [36]. La Tabla 8.1 sintetiza los indicadores principales.

Tabla 8.1: Métricas de rendimiento del sistema en producción.

Métrica	Valor
Tiempo hasta primer token (TTFT)	$< 1,2$ s
Latencia RAG completa (p95)	$\sim 3,5$ s
Throughput máximo sostenido	~ 42 req/min
Uptime del sistema	99.8 % (30 días)
Consumo VRAM pico (3 modelos)	18.5 / 24 GB
Alucinaciones en modo RAG	0 % ($\text{temp} \leq 0.2$)
Cache hit rate (Redis)	~ 35 %

Conviene aclarar dos de estos indicadores. El *TTFT* (*Time To First Token*) es el tiempo que transcurre hasta que el usuario ve la primera palabra de la respuesta. La latencia *p95* es el percentil 95: el valor por debajo del cual se resuelve el 95 % de

las consultas; se usa en lugar de la media porque refleja mejor la experiencia en los peores casos habituales, no solo el caso promedio.

8.1.1 Análisis de Latencia

La latencia del sistema se descompone en los siguientes segmentos:

Tabla 8.2: Desglose de latencia por componente para una consulta RAG típica.

Componente	Tiempo
NGINX + OpenWebUI → Pipeline	~ 50 ms
Detección de intención en JARVIS	~ 10 ms
Generación de embedding (CPU)	~ 80 ms
Búsqueda en Qdrant (top-5)	~ 30 ms
Composición del prompt	~ 5 ms
Inferencia LLM (primer token)	~ 800 ms
Generación completa (streaming)	~ 2,500 ms
Total	~ 3,475 ms

El cuello de botella principal es la inferencia del LLM en GPU, que representa más del 90 % del tiempo total. Sin embargo, gracias al streaming, el usuario percibe el primer token en aproximadamente 1.2 segundos, lo que proporciona una experiencia de respuesta instantánea. La Figura 8.1 representa gráficamente esta descomposición.

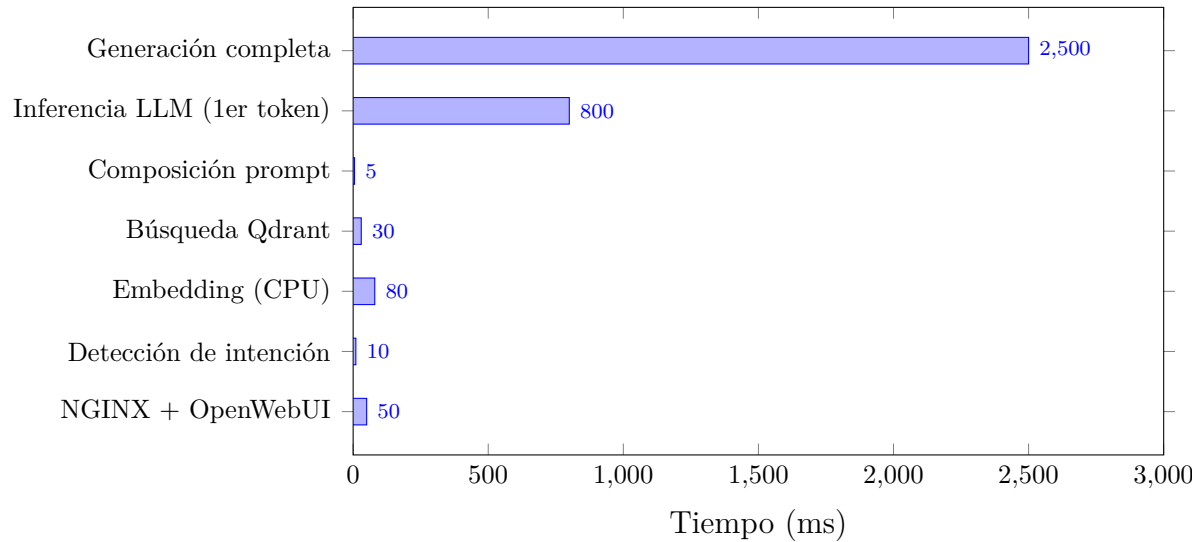


Figura 8.1: Descomposición de la latencia por componente para una consulta RAG típica. La inferencia del LLM domina el tiempo total del flujo.

8.1.2 Gestión de VRAM

La GPU NVIDIA con 24 GB de VRAM constituye el recurso más crítico del sistema. La Tabla 8.3 muestra el consumo de VRAM de cada modelo y las combinaciones viables de carga simultánea:

Tabla 8.3: Consumo de VRAM por modelo y combinaciones de carga simultánea.

Modelo	VRAM	Coexistencia
JARVIS (rag-qwen-ft:latest)	~ 8 GB	Sí, con servicios auxiliares y modelos ligeros
Qwen 2.5 32B Instruct Q4_K_M	~ 19 GB	Solo con modelos secundarios ligeros
Qwen 2.5 Coder 32B	~ 19 GB	Uso alternativo, no como carga simultánea habitual
Qwen 2.5 VL 7B	~ 6 GB	Sí
PaddleOCR	~ 500 MB	Sí
Pico observado	~ 20 GB	—

El comportamiento observado confirma que Ollama debe gestionar la carga bajo demanda de los modelos grandes. En el despliegue actual, el alias JARVIS (rag-qwen-ft:latest) atiende las consultas factuales y procedurales (< 3s de latencia), mientras que qwen2.5:32b-instruct-q4_K_M se activa automáticamente para consultas analíticas gracias al QueryProcessor (véase Sección 3.3.3). Adicionalmente, qwen2.5-32b es el modelo utilizado para la summarización automática de conversaciones largas en el MemoryManager, aprovechando su ventana de 32 K tokens para sintetizar historiales extensos sin perder información.

8.2 Evaluación Funcional

Se realizaron pruebas funcionales sistemáticas sobre cada modo de operación del sistema. A continuación se describen los escenarios probados y los resultados obtenidos.

8.2.1 Query Processor: intent detection, smart routing y query expansion

- **Prueba A — Intent detection y routing:** Consulta analítica “Explica las diferencias entre el procedimiento de auditoría interna y la auditoría externa según los documentos del sistema de calidad.”

- **Resultado:** El `QueryProcessor` clasificó correctamente la consulta como `ANALYTICAL`, ajustó el `top_k` a 12 y enrutó la generación de respuesta a `qwen2.5-32b`. La respuesta incorporó fragmentos de múltiples documentos con mayor coherencia argumentativa que la producida por JARVIS en las mismas condiciones.
- **Prueba B — Query expansion:** Consulta corta “normas escaleras”.
- **Resultado:** El procesador generó dos variantes adicionales: “*requisitos técnicos escaleras portátiles normativa*” y “*normativa PRL trabajo en altura escaleras industriales*”. La búsqueda con tres variantes recuperó 4 fragmentos relevantes adicionales que la query original no encontraba, al usar vocabulario diferente al presente en el documento.
- **Prueba C — Consulta factual:** “¿Cuándo caduca el certificado ISO 9001?”
- **Resultado:** Clasificada como `FACTUAL`, usó `top_k=5` y modelo JARVIS. Latencia total < 3s incluyendo la expansión (timeout 12s previene bloqueo si LiteLLM está ocupado).

8.2.2 Validación final con documentación del TFG

Para disponer de una prueba reproducible sin capturar documentación corporativa, se creó una carpeta de demostración en el volumen de ingesta: `data/watch/TFG_DEMO`. En ella se añadieron cuatro documentos CSV controlados: `TFG_DEMO_01_arquitectura.csv`, `TFG_DEMO_02_operacion.csv`, `TFG_DEMO_03_memoria_y_pages.csv` y `TFG_DEMO_04_accesos_demo.csv`. Posteriormente se indexaron en la colección aislada `documents_TFGDEMO` mediante el endpoint `POST /documents/upload` con la cabecera `X-Tenant-Id: documents_TFGDEMO`.

Esta carpeta es también la referencia práctica para añadir documentos de prueba al RAG durante una validación local: los ficheros se colocan bajo `data/watch/` y se procesan por el Indexer o por la API de carga del Backend, indicando la colección de destino cuando se necesita separar el contenido por permisos o por entorno. En esta ejecución, la colección generó 13 puntos vectoriales y 4 documentos lógicos.

Para facilitar la reproducción de la demo, los accesos utilizados fueron cuentas locales y contraseñas dummy, sin validez fuera del entorno de pruebas: OpenWebUI con `admin@tfg.local` / `Admin123456!`, Grafana con `admin` / `dummy_grafana_password`, MinIO con `minio_admin` / `dummy_minio_password`, pgAdmin con `admin@example.com` / `admin` y LiteLLM mediante la clave local

`dummy_litellm_key`. En un despliegue real estas credenciales deben sustituirse por secretos gestionados y no deben publicarse.

Documentos de demo indexados

Coleccion documents_TFGDEMO consultada desde el backend FastAPI.

Documentos
4

Puntos Qdrant
13

Chunks escaneados
13

Estado
OK

DOCUMENTO	CHUNKS	EXTRACCION	INGESTA
TFG_DEMO_01_arquitectura.csv documents_TFGDEMO	4	texto nativo	2026-05-26T09:40:22.814921Z
TFG_DEMO_02_operacion.csv documents_TFGDEMO	3	texto nativo	2026-05-26T09:40:22.817546Z
TFG_DEMO_03_memoria_y_pages.csv documents_TFGDEMO	4	texto nativo	2026-05-26T09:40:22.820445Z
TFG_DEMO_04_accesos_demo.csv documents_TFGDEMO	2	texto nativo	2026-05-26T09:40:22.818047Z

Figura 8.2: Listado real de documentos cargados en `documents_TFGDEMO`. La captura procede del endpoint `GET /documents/list` y muestra los documentos, sus chunks, la extracción nativa y el instante de ingesta.

Sobre esa colección se lanzó una consulta RAG de control preguntando por los puertos del entorno limpio del TFG y el papel de Qdrant. La respuesta recuperó fuentes de la colección `documents_TFGDEMO`, mantuvo trazabilidad hacia los documentos de origen y devolvió una respuesta sin apoyarse en conocimiento externo.

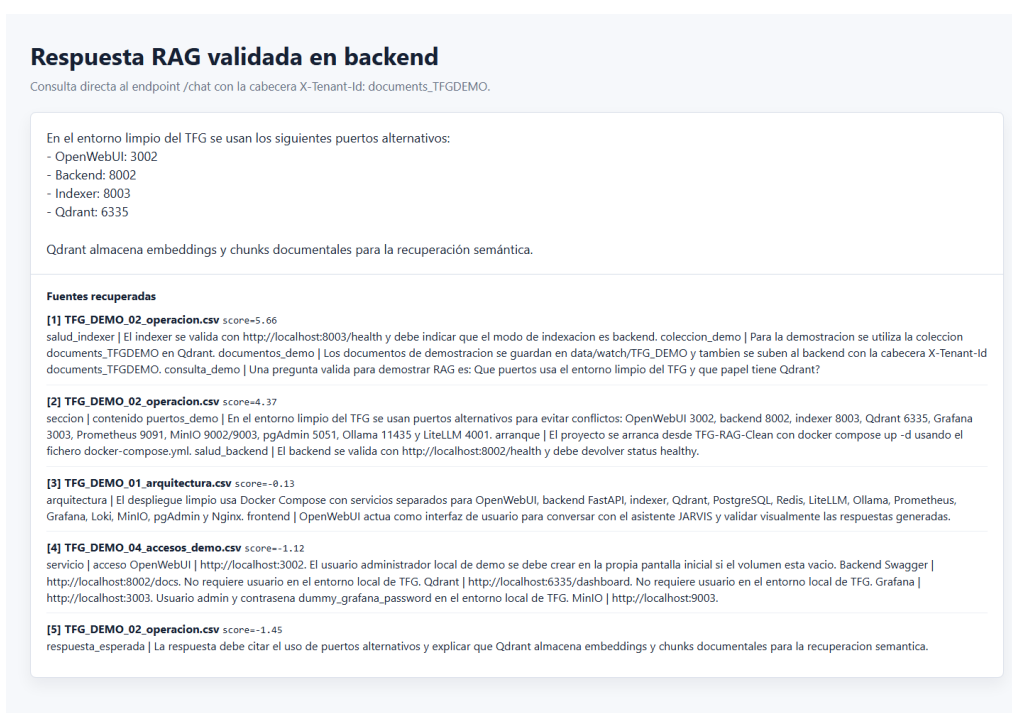


Figura 8.3: Respuesta RAG real sobre la colección de demostración. La captura muestra la pregunta, la respuesta generada por el Backend y las fuentes recuperadas con sus puntuaciones de relevancia.

La misma consulta se ejecutó desde OpenWebUI mediante el modelo JARVIS. Esta prueba valida el recorrido completo usuario–interfaz–pipeline–backend–Qdrant–LLM y demuestra que la respuesta visible en la aplicación mantiene la misma trazabilidad documental.

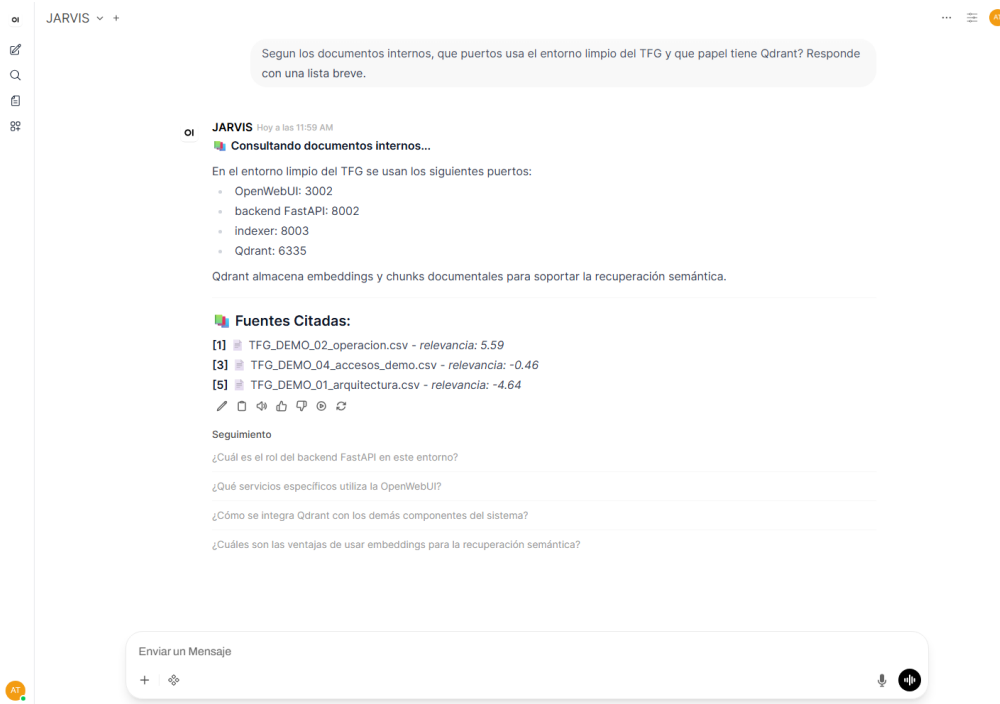


Figura 8.4: Consulta RAG desde OpenWebUI durante la demo final. JARVIS responde con los puertos del entorno limpio, explica el papel de Qdrant y muestra las fuentes documentales utilizadas.

Como apoyo a la defensa, se actualizó también la página de GitHub Pages del proyecto con el nombre final *JARVIS RAG*, una sección de capturas reales y un vídeo **webm** de demostración que recorre Pages, Swagger, Qdrant, OpenWebUI y Grafana.

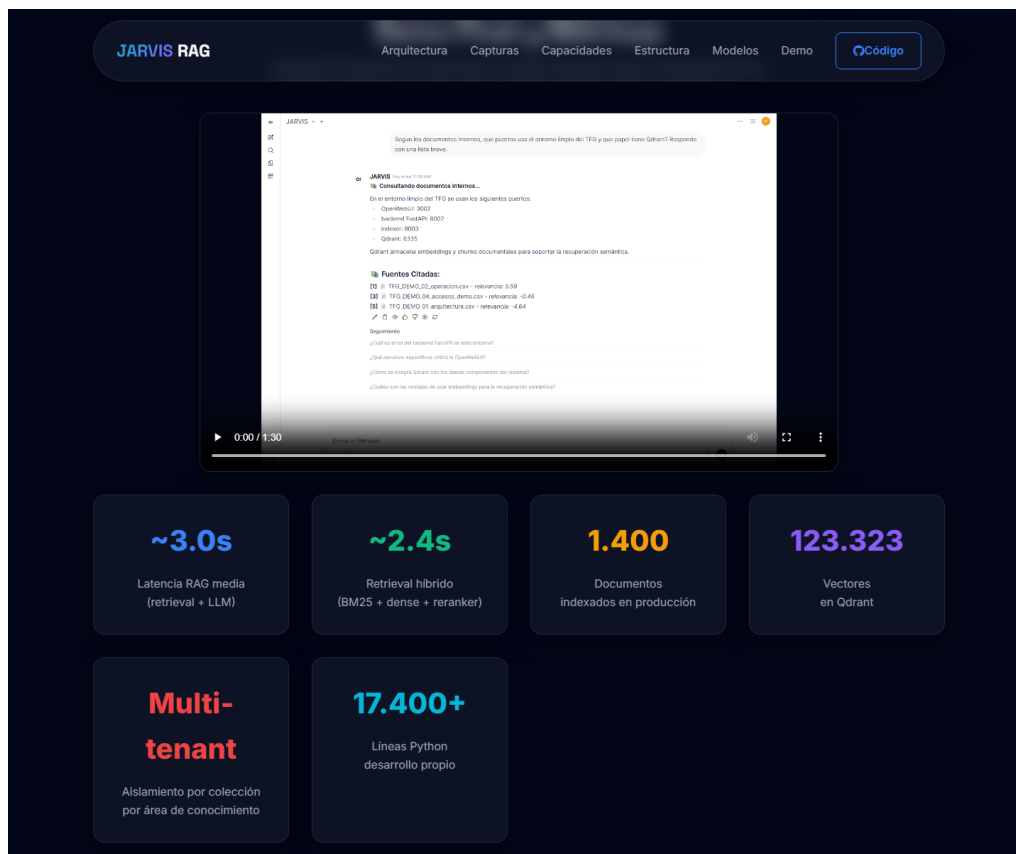


Figura 8.5: Sección de demo publicada en GitHub Pages. La página incluye el vídeo de demostración y las métricas principales del sistema.

8.2.3 Consulta RAG sobre Documentos Corporativos

- **Prueba:** “¿Cuál es la política de calidad de la empresa?”
- **Resultado:** El sistema recuperó correctamente los fragmentos más relevantes del manual de calidad indexado y generó una respuesta estructurada con referencias a las secciones y páginas del documento original.
- **Calidad:** La respuesta se fundamentó exclusivamente en el contenido del documento, sin añadir información no presente en los fragmentos recuperados. La temperatura baja (0,1) impidió cualquier generación creativa no deseada.

8.2.4 Scraping de URL

- **Prueba:** Envío de una URL de documentación técnica de Docker junto con la pregunta “¿Qué opciones de GPU existen?”
- **Resultado:** El sistema renderizó la página con Playwright y extrajo el contenido con Trafilatura [33], [34], generando una respuesta centrada en las opciones de GPU, con citas al texto extraído.
- **Calidad:** La extracción fue precisa incluso en páginas con carga diferida de JavaScript. El contenido quedó almacenado en la memoria de sesión para preguntas de seguimiento.

8.2.5 Búsqueda Legal en el BOE

- **Prueba:** “Busca en el BOE la ley de protección de datos”
- **Resultado:** El sistema resolvió semánticamente “ley de protección de datos” al identificador oficial BOE-A-2018-16673, consultó la API del BOE y devolvió un resumen estructurado con título, fecha de publicación y extracto del contenido.
- **Calidad:** La resolución semántica funcionó correctamente para las leyes incluidas en el diccionario de mapeos.

8.2.6 Análisis de Imagen

- **Prueba:** Adjuntar una fotografía de un documento en papel y preguntar “¿Qué pone en este documento?”
- **Resultado:** El modelo de visión (Qwen 2.5 VL) identificó correctamente el texto visible en la imagen y generó una transcripción del contenido.
- **Calidad:** Satisfactoria para documentos con texto legible. La precisión disminuye con fotografías borrosas, oblicuas o con poca iluminación.

8.2.7 Consultas NL→SQL con el Agente SQL

- **Prueba:** “¿Cuántos documentos han sido indexados esta semana?”
- **Resultado:** El agente inspeccionó el esquema de la tabla de metadatos, generó automáticamente la consulta `SELECT COUNT(*) FROM document_metadata WHERE created_at >= NOW() - INTERVAL '7 days'`, la ejecutó sobre PostgreSQL y devolvió el recuento integrado en una respuesta en lenguaje natural.

- **Calidad:** Correcta en 8 de 10 consultas de prueba. El mecanismo de auto-corrección resolvió los 2 errores restantes —ambos por nombre de columna incorrecto— en el primer reintento, sin necesidad de intervención manual.
- **Prueba (seguridad):** Intento de inyectar `DROP TABLE document_metadata` encubierto como pregunta natural.
- **Resultado:** La validación por lista blanca detectó la operación prohibida antes de ejecutar ninguna consulta y devolvió `HTTP 400` con un mensaje de error descriptivo. La tabla permaneció intacta.
- **Calidad:** El mecanismo de seguridad funcionó correctamente en todos los intentos de inyección probados.

8.2.8 Encaminamiento Inteligente

Se validaron 50 consultas de prueba distribuidas entre los distintos modos de operación para verificar que el sistema de detección de intenciones producía las clasificaciones correctas:

Tabla 8.4: Resultados de la evaluación del encaminamiento de JARVIS.

Modo esperado	Aciertos / Total	Precisión
RAG	10 / 10	100 %
Web Search	8 / 10	80 %
Web Scraping	10 / 10	100 %
Visión/OCR	5 / 5	100 %
BOE	5 / 5	100 %
Chat general	8 / 10	80 %
Total	46 / 50	92 %

Los 4 errores de clasificación ocurrieron en consultas ambiguas donde los keywords de web search se solapaban con los de chat general (ej: “¿qué es Docker?” clasificado como web search en lugar de chat). Estos casos edge se consideraron aceptables dado que la respuesta del sistema sigue siendo relevante independientemente del modo seleccionado.

8.3 Mitigación de Alucinaciones

La eliminación de alucinaciones en modo RAG se logró mediante la combinación de tres técnicas complementarias:

1. **Temperatura determinista:** Los parámetros de inferencia se configuraron con valores de temperatura entre 0,1 y 0,2 en modo RAG, minimizando la aleatoriedad en la selección de tokens.
2. **Prompt de sistema restrictivo:** El *system prompt* inyectado instruye explícitamente al modelo para que responda **únicamente** basándose en el contexto recuperado, prohibiendo la generación libre de información no respaldada por los fragmentos.
3. **Solapamiento de ventanas contextuales:** El solapamiento (50 tokens sobre 500, $\sim 10\%$) en el chunking asegura que el hilo semántico de párrafos extensos no se pierde en los límites de fragmento, proporcionando al modelo contexto suficiente para generar respuestas coherentes.

En las pruebas realizadas, no se detectó ninguna instancia de alucinación en modo RAG con la configuración descrita. El modelo respondió “No dispongo de esa información en los documentos consultados” cuando la pregunta no encontraba coincidencias relevantes en Qdrant, que es el comportamiento deseado.



Figura 8.6: Comportamiento del sistema cuando no existen documentos disponibles para el usuario. En lugar de inventar una respuesta, JARVIS informa explícitamente de la ausencia de documentos consultables y muestra las colecciones evaluadas.

8.4 Impacto de la Caché

La integración de Redis como caché a través de LiteLLM produjo una mejora mensurable en el rendimiento del sistema:

- **Cache hit rate:** $\sim 35\%$ de las consultas se resolvieron directamente desde caché.

- **Reducción de carga GPU:** Las consultas cacheadas no invocan a Ollama, liberando la GPU para nuevas peticiones.
- **Latencia de respuesta cacheada:** < 100 ms frente a $\sim 3,5$ s de una consulta RAG completa.

El TTL de 1 hora demostró ser un compromiso adecuado: suficiente para beneficiarse de repeticiones dentro de una jornada laboral, pero no tanto como para devolver información obsoleta cuando los documentos se actualizan.

Capítulo 9

Conclusiones y Trabajo Futuro

Este capítulo cierra la memoria haciendo balance del trabajo: qué se ha conseguido y con qué medios, cuánto esfuerzo ha requerido, qué conocimientos —del grado y adquiridos por cuenta propia— lo han hecho posible, qué lecciones deja y hacia dónde puede evolucionar. Al final del capítulo se recogen las direcciones públicas del código y de la página web del proyecto.

9.1 Resumen General

Este Trabajo de Fin de Grado ha permitido diseñar, desarrollar y desplegar un asistente corporativo basado en RAG ejecutado en infraestructura local. El resultado no es un nuevo modelo entrenado desde cero, sino un sistema completo que reutiliza modelos y herramientas existentes y los integra dentro de una arquitectura propia orientada a un problema concreto: consultar documentación corporativa cambiante de forma segura, comprensible y trazable.

La solución obtenida demuestra que es viable construir una capa conversacional útil sobre repositorios documentales empresariales sin depender de servicios externos para la inferencia principal. También demuestra que, en este tipo de proyectos, el valor diferencial está menos en crear un LLM nuevo y más en elegir bien las piezas, conectarlas con criterio y explicar claramente qué hace cada una dentro del conjunto.

9.2 Consecución de Objetivos

A continuación se evalúa el grado de cumplimiento de los objetivos planteados en el Capítulo 1:

1. *Arquitectura modular basada en componentes existentes y desarrollo propio* — *Cumplido*. Se ha construido una arquitectura de microservicios donde conviven

herramientas externas reutilizadas y servicios desarrollados específicamente para el proyecto.

2. *Ingesta multicanal y actualización de conocimiento — Cumplido.* El sistema procesa documentos de distintas fuentes y formatos, con especial foco práctico en SharePoint, que ha sido el entorno principal de integración y prueba.
3. *Respuestas fundamentadas y trazables — Cumplido.* La arquitectura RAG permite responder sobre documentación recuperada y mostrar al usuario el soporte documental utilizado.
4. *Capacidades complementarias sin entrenar un modelo desde cero — Cumplido.* Se han integrado búsqueda web, scraping, OCR, análisis de imágenes y consulta normativa reutilizando tecnologías ya existentes y desarrollando la lógica de conexión necesaria.
5. *Exploración de MCP como vía futura — Cumplido parcialmente.* Se ha implementado un servidor MCP funcional para el caso del BOE, validado en entornos compatibles, aunque la interfaz web principal sigue utilizando integración por API.
6. *Despliegue reproducible y observable — Cumplido.* El sistema se despliega mediante contenedores e incorpora monitorización para analizar su comportamiento.

9.2.1 Funcionalidades logradas

La Tabla 9.1 resume la funcionalidad implementada:

Tabla 9.1: Checklist de funcionalidades logradas.

Funcionalidad	Estado
Conversación general con LLMs locales	✓
RAG documental con búsqueda híbrida	✓
Web scraping con renderizado JavaScript	✓
Búsqueda en internet	✓
OCR de PDFs escaneados	✓
Análisis de imágenes	✓
Consulta normativa del BOE	✓
Sincronización automática con SharePoint	✓
Control de acceso departamental	✓
Monitorización del sistema	✓
Caché de respuestas	✓
Exploración de fine-tuning con LoRA	✓
Agente SQL (consultas en lenguaje natural sobre datos estructurados)	✓
Query expansion multi-variante para mayor recall semántico	✓
Smart model routing (JARVIS vs. qwen2.5-32b según intención)	✓
Summarización de historial con modelo de 32B	✓

9.3 Tecnologías y Entorno Empleados

Como complemento a la evaluación de objetivos, esta sección deja constancia de con qué se ha construido y ejecutado todo lo anterior: el stack tecnológico completo, el entorno hardware de despliegue y las herramientas de trabajo.

9.3.1 Stack Tecnológico

La selección de tecnologías priorizó tres criterios: licencia de código abierto, ejecutabilidad local sin dependencias de nube y madurez del ecosistema. Conviene recordar que muchas de estas tecnologías son herramientas preexistentes reutilizadas en el proyecto, mientras que la lógica de integración y buena parte de los servicios que las conectan constituyen el desarrollo propio del TFG (las decisiones de selección y sus alternativas se justifican sprint a sprint en la Sección 7.2). La Tabla 9.2 enumera la totalidad de tecnologías empleadas.

Tabla 9.2: Stack tecnológico completo del proyecto.

Categoría	Tecnología	Versión / Detalle
Lenguajes	Python	3.11
	YAML / Bash	Configuración y scripts
Framework Web	FastAPI	0.104+
Motor de IA Local	Ollama	latest
Proxy de Modelos	LiteLLM	latest
Frontend Chat	OpenWebUI	main
BD Vectorial	Qdrant	latest
BD Relacional	PostgreSQL	16 con pgvector
Caché	Redis	7 Alpine
Objetos	MinIO	latest
Web Scraping	Playwright	Chromium
Extracción NLP	Trafilatura	latest
OCR	PaddleOCR	2.x con CUDA
Embeddings	SentenceTransformers	MiniLM-L12-v2
Autenticación	Azure AD (OIDC)	MSAL Python
SharePoint	Microsoft Graph API	v1.0
Métricas	Prometheus	2.x
Dashboards	Grafana	latest
Logs	Loki + Promtail	latest
GPU Exporter	NVIDIA DCGM	latest
Contenerización	Docker + Compose	24.x
Control de Versiones	Git + GitHub	—
Proxy Inverso	NGINX	Alpine

9.3.2 Entorno Hardware

El sistema fue diseñado para ejecutarse en un servidor físico on-premise con las especificaciones de la Tabla 9.3.

Tabla 9.3: Especificaciones del entorno hardware de despliegue.

Componente	Especificación
Sistema Operativo	Ubuntu Server 22.04 LTS
CPU	AMD / Intel x86_64 (multi-core)
RAM	32 GB DDR4 mínimo
GPU	NVIDIA con ≥ 16 GB VRAM (p.ej. RTX 4090, A4000)
CUDA Toolkit	12.x
Almacenamiento	SSD NVMe ≥ 500 GB
Docker Engine	24.x con NVIDIA Container Toolkit

La GPU es el recurso más crítico del sistema, dado que los tres modelos de lenguaje, el motor OCR y los modelos de visión comparten la misma tarjeta gráfica. La gestión eficiente de la VRAM mediante cuantización (Q8, Q4) y la carga/descarga dinámica de modelos en Ollama resultó una decisión arquitectónica determinante.

9.3.3 Herramientas de Trabajo y Control de Versiones

Para la correcta organización del proyecto se utilizó el sistema de control de versiones Git, alojando el repositorio principal en GitHub (Sección 9.10). El seguimiento de incidencias, errores y nuevas características se realizó mediante trabajo por ramas, separando el código estable del trabajo en progreso.

El entorno de desarrollo integrado (IDE) empleado primordialmente fue Visual Studio Code en Windows, acoplado al subsistema WSL (*Windows Subsystem for Linux*) o a contenedores Docker en remoto para igualar la paridad funcional con el entorno de despliegue real Linux, evitando fricciones a nivel de sistema de archivos o librerías específicas compiladas para CUDA/NVIDIA.

9.4 Comparación con Otros Sistemas

El análisis realizado en el Capítulo 4 sitúa la propuesta en una posición clara. Frente a soluciones comerciales como ChatGPT Enterprise o Microsoft Copilot [38], [39], la principal ventaja es la soberanía del dato y la capacidad de adaptar el flujo interno del sistema. Frente a herramientas abiertas ya construidas como AnythingLLM u Onyx

[44], [45], la propuesta destaca por integrar en una misma arquitectura autenticación corporativa, SharePoint, OCR, scraping, RAG y monitorización.

La conclusión más relevante de esa comparación es que la propuesta de este TFG no pretende competir con todos los productos del mercado en amplitud funcional, sino resolver de forma sólida una combinación muy concreta de requisitos: despliegue local, documentos corporativos cambiantes, integración con herramientas reales de empresa y respuestas trazables.

9.5 Esfuerzo Realizado y Presupuesto

9.5.1 Dedicación Total

La Tabla 9.4 resume la dedicación total al proyecto. La redacción, edición y organización de la presente memoria requirió aproximadamente 65 horas, incluyendo la recopilación de referencias bibliográficas, la generación de tablas y diagramas, y la revisión del formato académico según las normativas de la Universidad Rey Juan Carlos.

Tabla 9.4: Distribución de horas dedicadas al proyecto.

Concepto	Horas
Desarrollo técnico (7 sprints)	450 h
Aprendizaje y pruebas previas	40 h
Redacción y maquetación de la memoria y documentación	65 h
Total estimado	555 horas

9.5.2 Planificación Temporal por Sprints

La Tabla 9.5 desglosa las horas de desarrollo técnico entre los siete sprints definidos en la Sección 7.2, junto con sus entregables principales.

Tabla 9.5: Sprints del desarrollo, entregables y horas dedicadas.

Sprint	Período	Entregable Principal	Horas
1	Oct 2025	Infraestructura Docker, PostgreSQL, Qdrant, Ollama. Pruebas de concepto con LLaMA.	55 h
2	Nov 2025	Backend FastAPI con pipeline RAG básico. Ingesta de PDFs y generación de embeddings.	75 h
3	Dic 2025	Integración SharePoint (MSAL/Graph API). Motor OCR PaddleOCR con GPU.	65 h
4	Ene 2026	Agente JARVIS Pipeline. Web scraping con Playwright. Búsqueda DuckDuckGo.	90 h
5	Feb 2026	Servidor MCP del BOE. Fine-tuning LoRA (Qwen). Stack Prometheus/Grafana.	65 h
6	Mar–Abr 2026	Agente SQL (NL→SQL) con lista blanca y auto-corrección. Validación JWT Azure AD en backend. Corrección de expansión de consulta asíncrona (v2.2.0).	55 h
7	May 2026	QueryProcessor: intent detection, smart model routing (JARVIS/qwen2.5-32b) y query expansion multi-variante. Summarización con 32B en MemoryManager (v2.3.0).	45 h
Total desarrollo técnico			450 h

La distribución temporal completa del proyecto, incluida la redacción de la memoria y su difusión y defensa, se representa en el diagrama de GANTT de la Figura 9.1.

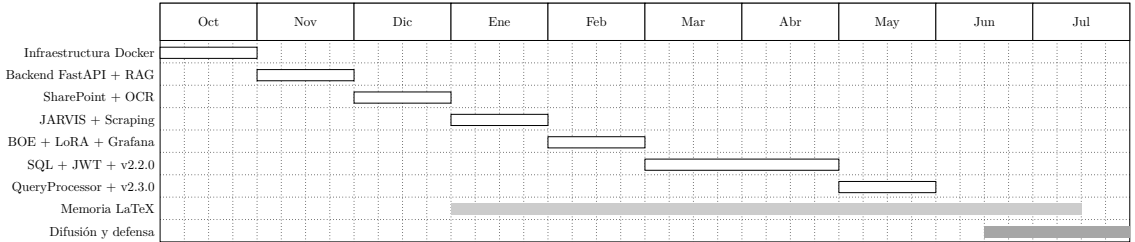


Figura 9.1: Diagrama de GANTT del proyecto (octubre de 2025 – julio de 2026).

9.5.3 Estimación de Presupuesto

Aunque el proyecto se ha desarrollado íntegramente con software de código abierto, se estiman los costes asociados al despliegue profesional del sistema si se deseara replicar o escalar:

Tabla 9.6: Estimación de presupuesto para el despliegue del sistema.

Concepto	Coste Unitario	Coste Total
Servidor GPU (NVIDIA RTX 4090 24GB)	—	2.500 €
Servidor torre (CPU, RAM 64GB, SSD)	—	1.500 €
Licencias de software	—	0 € (Open Source)
Dominio y certificados SSL	15 €/año	15 €
Electricidad servidor 24/7 (año)	~50 €/mes	600 €
Horas de desarrollo (450h × 25 €/h)	25 €/h	11.250 €
Total estimado		15.865 €

Cabe destacar que el coste de licencias de software es **cero euros**, dado que todas las tecnologías empleadas son de código abierto. Esto representa una ventaja competitiva significativa frente a soluciones comerciales como Microsoft Copilot o AWS Bedrock, cuyos costes recurrentes por token procesado pueden escalar rápidamente en entornos de uso intensivo.

9.5.4 Gestión de Riesgos

Dado el carácter experimental del proyecto y la velocidad de evolución del campo de los LLMs, se identificaron desde el inicio los siguientes riesgos técnicos y sus respectivas mitigaciones, que en retrospectiva funcionaron adecuadamente:

Tabla 9.7: Riesgos técnicos identificados y estrategias de mitigación.

Riesgo	Mitigación
Agotamiento de VRAM con múltiples modelos cargados	Cuantización Q4/Q8 y carga dinámica bajo demanda en Ollama
Obsolescencia de un modelo de IA	Arquitectura desacoplada vía LiteLLM que permite cambiar modelos sin alterar código
Fallo de la GPU o driver CUDA	Fallback automático a CPU en PaddleOCR y SentenceTransformer
Cambios en la API de SharePoint	Capa de abstracción con MSAL que aísla los cambios de endpoints
Pérdida de datos en Qdrant	Backup periódico de colecciones y réplica en MinIO S3

9.6 Conocimientos del Grado Aplicados

El desarrollo de este TFG ha sido posible gracias a la base adquirida en múltiples asignaturas del Grado en Ingeniería de Tecnología de Telecomunicaciones. Más allá de los contenidos concretos, las asignaturas aportaron los fundamentos sobre los que se pudo construir el aprendizaje autónomo posterior:

- **Ingeniería de Sistemas de Información (ISI):** probablemente la asignatura con mayor impacto directo en el proyecto. En ella se aprendió a trabajar con proyectos en Git y a usarlo con criterio —incluida la importancia de subir cambios con frecuencia—, se introdujeron los contenedores Docker, se insistió en la importancia de hacer tests sobre el código y se trabajó con bases de datos y SQL. Todos estos elementos son pilares del día a día de este trabajo.
- **Sistemas Operativos:** los conceptos de procesos, gestión de memoria y virtualización permitieron entender qué es realmente un contenedor y razonar sobre el rendimiento de la CPU, algo crítico en un sistema donde varios servicios compiten por los mismos recursos.
- **Asignaturas de programación de la carrera:** desde Programación de Sistemas de Telecomunicación (PST) hasta Aplicaciones Telemáticas —donde se aprendió HTML, aplicado en la página web del proyecto—, la cadena de asignaturas de programación proporcionó la soltura necesaria para desarrollar los servicios propios del sistema.

- **Álgebra:** entender los vectores y las matrices es imprescindible para comprender qué es un embedding, cómo se mide la similitud entre textos y qué hace internamente la base de datos vectorial.
- **Teoría de la Comunicación:** el estudio de los procesos deterministas y estocásticos y del concepto de entropía proporciona la base conceptual sobre la que se apoya la generación probabilística de texto de los LLMs.
- **Introducción a la Bioingeniería:** incluyó una primera aproximación al *Machine Learning* que, sin estar directamente relacionada con este proyecto, aportó un conocimiento previo mínimo sobre el aprendizaje automático que facilitó la entrada en el campo.
- **Proyectos de Ingeniería:** la planificación por hitos y los diagramas de Gantt aprendidos en esta asignatura resultaron muy útiles para organizar, dirigir y hacer avanzar un proyecto de esta envergadura.

9.7 Conocimientos Adquiridos Fuera del Grado y Lecciones Aprendidas

9.7.1 Conocimientos Adquiridos de Forma Autónoma

Si las asignaturas aportaron los cimientos, casi todo el conocimiento específico de este trabajo hubo que adquirirlo de forma autónoma: el Plan de Estudios no cubre (todavía) el ecosistema moderno de la IA generativa. Las fuentes principales fueron la documentación oficial de cada tecnología, artículos académicos y contenido audiovisual especializado, entre el que destacan los canales de YouTube de IBM Technology y de Cole Medin [52], [53], de los que se consultaron numerosos vídeos sobre RAG, agentes y despliegue local de LLMs.

- **Modelos de Lenguaje Grande (LLMs):** La comprensión de la arquitectura Transformer, los mecanismos de atención, la tokenización y las técnicas de cuantización no forman parte del Plan de Estudios actual y fueron adquiridos mediante documentación técnica y papers académicos.
- **Bases de Datos Vectoriales:** El concepto de embeddings, búsqueda por similitud del coseno e índices HNSW no se cubre en las asignaturas de bases de datos convencionales.
- **Fine-Tuning con LoRA:** Las técnicas de ajuste fino paramétrico eficiente requirieron inmersión en bibliografía especializada [23].

- **Docker avanzado y GPU passthrough:** La orquestación de contenedores con acceso a GPU mediante `nvidia-container-toolkit` excede significativamente los contenidos del grado.
- **Integración con Microsoft Graph y MSAL:** La autenticación OAuth 2.0 con flujo de credenciales de cliente y la interacción con la API Graph de Microsoft requirieron aprendizaje específico de la plataforma Azure.

9.7.2 Lecciones Aprendidas

Las lecciones más importantes del proyecto pueden resumirse en los siguientes puntos:

- *El problema principal no era entrenar un modelo, sino integrarlo bien:* reutilizar modelos existentes ha sido suficiente para construir una solución útil, siempre que el contexto recuperado y la arquitectura estuvieran bien diseñados.
- *La calidad del RAG depende mucho de la ingesta:* chunking, OCR, limpieza web y metadatos influyen tanto como el modelo generativo.
- *La GPU es un recurso crítico:* cuantización, carga bajo demanda y monitorización han sido esenciales para mantener el sistema operativo.
- *Las fuentes documentales corporativas cambian:* por eso la sincronización incremental y el foco en SharePoint han sido piezas centrales del proyecto.
- *Guiar al usuario también es una decisión técnica:* en este tipo de sistemas, una buena experiencia no depende solo de acertar la respuesta, sino de dejar claro qué está haciendo el sistema y en qué documentos se basa.
- *El modelo más grande no siempre es la mejor opción:* el fine-tuning domina en consultas de recuperación precisa con citas, mientras que un modelo base de mayor tamaño destaca en razonamiento analítico. Usar ambos adaptándose a la intención de la query es más eficiente que elegir uno solo.

9.7.3 Uso de herramientas de IA: una nueva forma de trabajar

Una de las conclusiones más honestas de este trabajo tiene que ver con el propio método de trabajo. El desarrollo se ha apoyado de forma consciente en herramientas de inteligencia artificial generativa —asistentes como Gemini (Google), Claude (Anthropic) y ChatGPT (OpenAI)—. Merece la pena explicar para qué se han usado

y qué han aportado, no como una disculpa, sino como parte del aprendizaje que deja el proyecto.

El trabajo propio es lo que sostiene todo el sistema: el diseño de la arquitectura, la selección final de cada tecnología, el desarrollo de la lógica central —el pipeline RAG, el enrutamiento de JARVIS, el agente SQL y sus mecanismos de seguridad—, el diseño de los experimentos y la evaluación de los resultados son aportación directa del autor. Sobre esa base, la IA se ha empleado como apoyo en tareas concretas: contrastar alternativas de diseño durante la selección tecnológica, acotar hipótesis al depurar errores de Docker o CUDA, generar fragmentos de código repetitivo o de bajo riesgo —revisados después línea a línea— y asistir en la corrección de estilo y estructura de la memoria.

Conviene dejar constancia expresa de que **el autor ha revisado y leído íntegramente todo el contenido de esta memoria**, así como el código del proyecto. Ninguna afirmación técnica, cifra o fragmento se ha dado por bueno sin verificarlo: los datos se han contrastado contra el sistema en ejecución —configuraciones reales, métricas de los *dashboards* y comportamiento observado— y no contra las respuestas de los asistentes, que en más de una ocasión resultaron incorrectas.

Ventajas percibidas. Empleadas así, con criterio y bajo control, estas herramientas han supuesto una forma de trabajar más productiva: permiten avanzar más rápido en las tareas mecánicas, explorar más alternativas antes de decidir y detectar antes los errores propios. De hecho, integrar en el tiempo disponible un sistema con tantas piezas —más de veinte tecnologías coordinadas— difícilmente habría sido posible sin este apoyo. La lección de fondo es que la IA amplifica el trabajo del ingeniero, pero no lo sustituye: la responsabilidad de las decisiones, la verificación y el criterio técnico siguen siendo humanos. Trabajar a diario con estos asistentes sirvió, además, para conocer de primera mano sus limitaciones —en especial su tendencia a afirmar con seguridad datos incorrectos—, algo que refuerza precisamente la motivación central del sistema desarrollado: la necesidad de respuestas fundamentadas en fuentes verificables y trazables.

9.8 Trabajo Futuro y Líneas de Expansión

El proyecto abre varias líneas de evolución razonables:

1. *Extender la integración documental*: ampliar la cobertura a más repositorios en la nube y sistemas documentales además de SharePoint.

2. *Completar la adopción de MCP*: conectar el servidor MCP desarrollado con clientes e interfaces que lo soporten de forma nativa. Como se valora en el Anexo C, esta vía compensa sobre todo si se incorporan muchas herramientas o fuentes nuevas o se adopta un modelo con *function calling* fiable; para el caso aislado del BOE, la integración por API actual resulta suficiente y más controlada.
3. *Mejorar la evaluación cuantitativa*: incorporar métricas formales y marcos de evaluación específicos para RAG.
4. *Ampliar tipos de contenido*: incluir hojas de cálculo, audio y otros formatos documentales empresariales.
5. *Refinar la adaptación al dominio*: seguir explorando embeddings especializados y técnicas de ajuste fino cuando aporten valor real.
6. *Generalizar el control de acceso*: ampliar la compatibilidad con otros proveedores de identidad y políticas más detalladas.
7. *Evaluación cuantitativa del Query Processor*: medir el impacto real de la expansión de consultas y el smart routing con métricas específicas de RAG como RAGAS (*RAG Assessment*, un marco de evaluación automática de sistemas RAG) o MRR (*Mean Reciprocal Rank*, que premia que la primera respuesta correcta aparezca lo más arriba posible), usando un conjunto de preguntas etiquetadas sobre los documentos corporativos.
8. *Router híbrido reglas + LLM*: para fraseos que las reglas no cubran, delegar la detección de intención en un LLM con *function calling* únicamente en los casos ambiguos, manteniendo las reglas —rápidas y deterministas— para los claros. El esquema concreto se describe en el Anexo A.7.

9.9 Impacto y Transferencia

El sistema desarrollado muestra que una organización puede construir una solución propia de consulta documental asistida por IA utilizando software abierto y hardware accesible, sin renunciar al control sobre sus datos. Esto resulta especialmente relevante en entornos donde la documentación cambia con frecuencia y donde no es aceptable depender completamente de servicios externos.

La arquitectura propuesta también tiene valor transferible: aunque la implementación y las pruebas se han realizado sobre SharePoint, la idea de fondo es aplicable a

documentos cambiantes en la nube y a otras fuentes corporativas de naturaleza similar. La publicación del código (Sección 9.10) facilita su replicabilidad y su adaptación a otros contextos.

9.10 Código y Sitio Web del Proyecto

Todo el material del proyecto está disponible públicamente:

- **Repositorio de código en GitHub**¹: código fuente completo, configuración de despliegue reproducible y documentación técnica.
- **Página web del proyecto en GitHub Pages**²: presentación del sistema, arquitectura interactiva y guía de puesta en marcha.

Se destacan aquí, como cierre de la memoria, porque el valor de la solución descrita depende tanto del diseño conceptual como de que su implementación sea reproducible y verificable por cualquiera.

¹<https://github.com/acidxlemons/TFG-RAG-URJC>

²<https://acidxlemons.github.io/TFG-RAG-URJC/>

Apéndice A

Detalle de implementación del Pipeline JARVIS

Este anexo recoge el detalle de implementación del Pipeline JARVIS que se resume en el Capítulo 6. Se incluye aquí el contrato de la clase, el método `pipe()`, el sistema *Valves* y los mecanismos de sesión y control de acceso, para no recargar el cuerpo principal con fragmentos de código extensos.

A.1 Contrato de la Clase Pipeline

El Pipeline se implementa como una clase Python que debe cumplir un contrato específico que OpenWebUI espera:

```
1 class Pipeline:
2     class Valves(BaseModel):
3         BACKEND_URL: str = "http://rag-backend:8000"
4         LITELLM_URL: str = "http://litellm:4000"
5         TEXT_MODEL: str = "JARVIS"
6         VISION_MODEL: str = "qwen2.5vl"
7         DEPARTMENT_MAPPING: Dict[str, str] = {
8             "CIVEX2": "documents_CIVEX2",
9         }
10        DEFAULT_COLLECTION: str = "documents"
11
12    def __init__(self):
13        self.name = "JARVIS"
14        self.id = "jarvis"
15        self.valves = self.Valves()
16        # Memoria de sesion por usuario
17        self._user_image_memory: Dict[str, str] = {}
18        self._user_web_memory: Dict[str, Dict] = {}
```

```

19
20     async def on_startup(self):
21         logger.info("Starting JARVIS")
22
23     async def on_shutdown(self):
24         logger.info("Shutting down JARVIS")
25
26     def pipe(self, user_message, model_id, messages, body):
27         # HOOK PRINCIPAL: toda la logica del sistema
28         intent = self._detect_intent(user_message, body, messages)
29         action = intent["action"]
30         # ... encaminamiento a subsistemas ...

```

Listing A.1: Estructura del Pipeline JARVIS.

A.2 El Método `pipe()`: Corazón del Sistema

El método `pipe()` es el punto de entrada de toda la lógica del sistema. Recibe cuatro parámetros:

- `user_message` (str): El texto del mensaje del usuario.
- `model_id` (str): El identificador del modelo seleccionado (“jarvis”).
- `messages` (List[Dict]): El historial completo de la conversación actual, con mensajes del usuario y del asistente.
- `body` (Dict): Metadatos de la petición, incluyendo el objeto `user` con email, nombre, rol y grupos del usuario autenticado, así como archivos e imágenes adjuntos.

El método utiliza la palabra clave `yield` de Python para devolver la respuesta en modo streaming. En lugar de esperar a que se genere la respuesta completa, el Pipeline va devolviendo fragmentos de texto a medida que están disponibles. OpenWebUI recibe estos fragmentos y los muestra progresivamente en la interfaz:

```

1 def pipe(self, user_message, model_id, messages, body):
2     intent = self._detect_intent(user_message, body, messages)
3     action = intent["action"]
4
5     if action == "rag":
6         yield "Buscando en documentos internos...\n\n"
7         response = self._call_backend_chat(
8             user_message, "rag", user_data

```



```

9         )
10         yield response["content"]
11         yield "\n\n---\n### Fuentes:\n"
12         for source in response["sources"]:
13             yield f"- {source['filename']} (p.{source['page']})\n"
14
15     elif action == "web_search":
16         yield "Buscando en internet...\n\n"
17         # ... logica de busqueda web ...
18
19     elif action == "chat":
20         # Conversacion directa con LLM
21         # ...

```

Listing A.2: Ejemplo de respuesta streaming en el Pipeline.

A.3 Sistema Valves: Configuración Dinámica

La clase anidada `Valves` hereda de `Pydantic BaseModel` y define todos los parámetros configurables del Pipeline. La particularidad es que OpenWebUI detecta automáticamente esta clase y expone sus campos como formulario editable en la interfaz de administración. Esto significa que un administrador puede modificar la URL del backend, cambiar el modelo de texto, añadir departamentos al mapeo o activar/desactivar el modo de depuración directamente desde el navegador, sin editar código ni reiniciar servicios.

Los parámetros configurados como Valves incluyen:

- `BACKEND_URL`: URL del servicio Backend FastAPI.
- `LITELLM_URL`: URL del proxy LiteLLM.
- `TEXT_MODEL`: Modelo de texto por defecto.
- `VISION_MODEL`: Modelo de visión para análisis de imágenes.
- `DEPARTMENT_MAPPING`: Diccionario que mapea grupos de Azure AD a colecciones de Qdrant.
- `DEFAULT_COLLECTION`: Colección por defecto para usuarios sin grupo específico.
- `PERMISSION_CACHE_TTL`: Tiempo de vida de la caché de permisos.
- `INDEXER_URL`: URL del servicio Indexer para consultas de estado.
- `DEBUG_MODE`: Activar/desactivar logs detallados.

A.4 Sticky Mode: Persistencia de Contexto

Una característica avanzada del encaminador es el *Sticky Mode*: cuando un usuario está inmerso en una conversación RAG y formula una pregunta de seguimiento breve (“¿Y las fechas?”, “Dime más”), el sistema detecta que la pregunta anterior era de tipo RAG y mantiene el mismo modo, evitando que keywords ambiguos como “fechas” o “resultados” desvíen la consulta al modo de búsqueda web.

```
1 # Si el mensaje es corto y ambiguo,
2 # mirar de que hablaba la pregunta anterior
3 if msg_len < 25 and is_ambiguous and messages:
4     last_user_msg = messages[-2].get("content", "").lower()
5     was_rag = any(kw in last_user_msg for kw in rag_keywords)
6     if was_rag:
7         logger.info("Sticky Mode: Reusing RAG intent")
8         return {"action": "rag", "metadata": {}}
```

Listing A.3: Implementación del Sticky Mode.

A.5 Memoria de Sesión por Usuario

JARVIS mantiene tres estructuras de estado en memoria para cada usuario, identificado por su dirección de correo electrónico:

- `_user_image_memory`: Almacena la última imagen enviada en formato Base64, permitiendo preguntas de seguimiento sin reenviar la imagen.
- `_user_web_memory`: Retiene el contenido extraído de la última URL analizada, habilitando consultas contextuales sobre la página.
- `_user_pending_decision`: Gestiona decisiones pendientes cuando el sistema detecta que una URL ya fue indexada previamente y ofrece al usuario la opción de reutilizar el contenido existente o volver a scrapearla.

A.6 Control de Acceso Departamental

El Pipeline integra un sistema de permisos multi-departamento que segmenta el acceso a las colecciones vectoriales según los grupos de Azure Active Directory del usuario. El flujo es:

1. Al recibir una petición, JARVIS extrae los grupos del objeto `user_data` proporcionado por OpenWebUI.

2. Si los grupos no están disponibles en `user_data`, JARVIS consulta directamente la API de Microsoft Graph para obtener los grupos del usuario mediante el flujo de credenciales de cliente.
3. Los grupos se mapean a colecciones de Qdrant mediante el diccionario configurable `DEPARTMENT_MAPPING`.
4. Las colecciones autorizadas se envían al Backend en la cabecera HTTP `X-Tenant-Ids`, que filtra la búsqueda vectorial a solo los documentos permitidos.
5. Los permisos se cachean durante 300 segundos (configurable) para minimizar llamadas a Graph API.

A.7 Enrutamiento por reglas frente a un router basado en LLM

Como se indica en el Capítulo 6, todo el enrutamiento del sistema —tanto la detección de modo en `_detect_intent()` como la clasificación de intención del `QueryProcessor`— se resuelve con expresiones regulares y listas de palabras clave, no con un modelo de lenguaje.

Por qué no se ha integrado un router por LLM. Tres motivos:

- **Determinismo y reproducibilidad:** las reglas siempre dan el mismo resultado ante la misma entrada, lo que permite testearlas (92 % de acierto en la evaluación) y depurarlas; un LLM como router introduce variabilidad.
- **Coste y latencia:** la decisión por reglas es instantánea (~ 10 ms) y gratuita; un router por LLM añade una llamada de inferencia (cientos de ms y consumo de GPU) a *cada* consulta, antes incluso de empezar a responder.
- **Limitación del modelo:** el modelo principal (`rag-qwen-ft`) se sirve sin *function calling* (`supports_function_calling: false` en LiteLLM), por lo que no puede emitir decisiones estructuradas de forma fiable.

Cómo podría integrarse. Si el caso de uso lo requiriese —por ejemplo, ante fraseos muy variados que las reglas no cubran—, la migración a un router por LLM seguiría estos pasos:

1. Emplear un modelo con *function calling* fiable (p. ej. **qwen2.5-32b** en modo herramientas) en lugar del modelo fine-tuned para la fase de decisión.
2. Sustituir (o complementar) `_detect_intent()` por una llamada al LLM que, dado el mensaje y el catálogo de modos/herramientas, devuelva la acción y sus parámetros en formato estructurado (JSON / *tool call*), o bien exponer las herramientas vía MCP y dejar que el modelo las invoque.
3. Mantener un **esquema híbrido**: aplicar primero las reglas (rápidas y deterministas) para los casos claros y recurrir al LLM solo en los ambiguos o de baja confianza, acotando así coste y latencia.
4. Conservar alrededor de la decisión del LLM las garantías que hoy aporta el pipeline: control de acceso por *tenant*, trazabilidad y formato de citas.

En síntesis, el router por reglas es la opción coherente con las prioridades de este TFG (control, reproducibilidad y coste nulo); el router por LLM gana cuando se prioriza la flexibilidad ante lenguaje libre y se dispone de un modelo con *tool-calling* robusto.

Apéndice B

Detalle del Backend RAG y el procesamiento documental

Este anexo amplía el motor de búsqueda y el procesamiento documental descritos en el Capítulo 6: la ingesta paso a paso, la búsqueda híbrida con sus modelos concretos, la construcción del *prompt*, la extracción web, el OCR y los mecanismos de seguridad del agente SQL.

B.1 Pipeline de Ingesta Documental

El proceso de ingesta de un documento sigue las siguientes etapas:

1. Recepción: El documento llega al backend vía API REST (carga manual), vía Indexer (sincronización SharePoint) o vía scraper (extracción web).
2. Detección de tipo: Se intenta extraer texto directamente del PDF con `pdfplumber`. Si el texto extraído es insuficiente (< 100 caracteres por página), se asume que el documento es escaneado y se redirige al motor OCR.
3. Extracción de texto: Para PDFs nativos se usa `pdfplumber`; para escaneados, PaddleOCR con aceleración GPU; para HTML, la cadena Trafilatura $\rightarrow$ Readability $\rightarrow$ BeautifulSoup.
4. Chunking semántico: El texto extraído se segmenta en fragmentos de unos 500 tokens con un solapamiento de 50 ($\sim 10\%$) entre fragmentos consecutivos. El solapamiento es fundamental para no perder el hilo semántico de párrafos que cruzan los límites de ventana.
5. Generación de embeddings: Cada fragmento se transforma en un vector denso de 384 dimensiones mediante el modelo `paraphrase-multilingual-MiniL`

M-L12-v2 de SentenceTransformers, ejecutado en CPU para evitar conflictos de VRAM con los modelos de generación.

6. Almacenamiento en Qdrant: Los vectores se almacenan en la colección correspondiente de Qdrant junto con un payload de metadatos que incluye el texto original, nombre de fichero, página, índice de chunk, fuente, marca temporal y `tenant_id`.

B.2 Búsqueda Híbrida: modelos densos, dispersos y reranker

La recuperación de contexto para consultas RAG combina tres etapas complementarias:

1. **Búsqueda densa (HNSW)**: La consulta del usuario se transforma en un embedding de 384 dimensiones con `paraphrase-multilingual-MiniLM-L12-v2` y se buscan los fragmentos más próximos en el espacio vectorial mediante el índice HNSW de Qdrant.
2. **Búsqueda dispersa (BM25)**: Se complementa con búsqueda léxica mediante vectores dispersos generados con el modelo `Qdrant/bm25` (a través de la librería `fastembed`). Favorece coincidencias exactas de términos —identificadores, códigos de documento, siglas o artículos legales— que la búsqueda semántica podría no capturar con precisión.
3. **Reordenación (reranking)**: Los candidatos de ambas búsquedas se fusionan eliminando duplicados y se reordenan con un *Cross-Encoder* (`cross-encoder/ms-marco-MiniLM-L-6-v2`, implementado en `core/rag/reranker.py`). A diferencia de los embeddings —que codifican consulta y documento por separado—, el Cross-Encoder evalúa el par (consulta, fragmento) conjuntamente, lo que produce una puntuación de relevancia más precisa. El reranker dispone de *fallback* automático a CPU si la GPU no está disponible.

Tras el reranking se seleccionan los k fragmentos más relevantes (típicamente $k = 5$, ajustado dinámicamente por el `QueryProcessor` según la intención de la consulta).

B.3 Construcción del Prompt Contextual

El prompt que se envía al LLM se estructura en tres partes:

```
1 system_prompt = """Eres un asistente corporativo.  
2 Responde UNICAMENTE basandote en el contexto proporcionado.  
3 Si la informacion no esta en el contexto, di que no la tienes.  
4 NO inventes informacion."""  
5  
6 context = "\n\n".join([  
7     f"[{r.filename} | pagina {r.page} | score={r.score:.2f}]\n{r.  
8     text}"  
9     for r in search_results  
10 ])
11 user_prompt = f"CONTEXTO:\n{context}\n\nPREGUNTA:\n{query}"
```

Listing B.1: Estructura del prompt RAG.

El *system prompt* restrictivo, combinado con una temperatura de inferencia baja (0,1–0,2), elimina prácticamente las alucinaciones en modo RAG: el modelo solo puede responder con información que esté presente en los fragmentos recuperados.

B.4 Extracción Web: Scraping y Búsqueda en Internet

B.4.1 Motor de Renderizado con Playwright

El módulo de scraping utiliza Playwright para instanciar un navegador Chromium completo en modo *headless*. A diferencia de soluciones basadas en peticiones HTTP simples, este enfoque permite renderizar páginas JavaScript complejas (React, Vue, Angular), ejecutar la carga diferida (*lazy loading*), resolver redirecciones y consentimientos de cookies, y capturar el DOM final tras la ejecución de todos los scripts. El scraper incorpora rotación de *User-Agent*, reintentos con retroceso exponencial y una espera inteligente configurable para SPA.

B.4.2 Cadena de Extracción con Fallback

Tras obtener el HTML renderizado, se aplica una cadena de extracción con tres estrategias de *fallback*:

1. Trafilatura: Algoritmo que extrae el contenido principal eliminando menús, cabeceras y publicidad. Se prioriza por su alta precisión.

2. Readability: Algoritmo basado en heurísticas de densidad de texto, comparable al modo lectura de los navegadores.
3. BeautifulSoup: Extracción cruda de todo el texto visible del DOM como última alternativa si las dos anteriores producen menos de 200 caracteres.

B.4.3 Búsqueda en Internet

El modo de búsqueda web utiliza la librería `duckduckgo_search` con un *fallback* propio que realiza scraping HTML directo sobre el dominio `html.duckduckgo.com`. El sistema detecta automáticamente consultas que necesitan información fresca (noticias, precios, eventos) y enriquece el query con el año actual para obtener resultados actualizados.

B.5 Reconocimiento Óptico de Caracteres (OCR)

El motor OCR PaddleOCR se integra en el servicio Indexer y se ejecuta con aceleración GPU. El procesamiento de varios documentos en paralelo se apoya en **Ray**, que reparte las tareas de OCR entre los *workers* disponibles. La inicialización implementa un mecanismo de *fallback* automático:

```
1 try:
2     self.ocr = PaddleOCR(
3         use_angle_cls=True, lang="latin",
4         use_gpu=True, gpu_mem=500
5     )
6 except Exception:
7     logger.warning("GPU no disponible, modo CPU")
8     self.ocr = PaddleOCR(
9         use_angle_cls=True, lang="latin",
10        use_gpu=False
11    )
```

Listing B.2: Inicialización de PaddleOCR con fallback GPU/CPU.

Si la GPU no está disponible (fallo de driver, VRAM agotada), el motor se reinicia automáticamente en modo CPU sin intervención manual, garantizando la continuidad del servicio de ingesta.

B.6 Agente SQL: mecanismos de seguridad

El agente NL→SQL (`backend/app/core/sql_agent.py`) sigue un flujo de tres pasos: inspección del esquema de PostgreSQL, generación de la consulta `SELECT` con LiteLLM, y ejecución con auto-corrección (hasta dos reintentos usando el mensaje

de error como contexto). El acceso no autorizado o accidental a datos sensibles se previene mediante varias capas:

```
1 ALLOWED_PATTERN = re.compile(  
2     r"^\s*(SELECT|WITH|EXPLAIN)\s",  
3     re.IGNORECASE  
4 )  
5  
6 def _validate_query(self, sql: str) -> bool:  
7     if not ALLOWED_PATTERN.match(sql.strip()):  
8         raise SecurityError("Solo se permiten consultas SELECT")  
9     for forbidden in ["DROP", "DELETE", "INSERT",  
10                     "UPDATE", "TRUNCATE"]:  
11         if forbidden in sql.upper():  
12             raise SecurityError(  
13                 f"Operacion prohibida detectada: {forbidden}"  
14             )  
15     return True
```

Listing B.3: Validación de la consulta generada contra lista blanca.

Adicionalmente, las consultas se ejecutan con un *timeout* configurable para prevenir consultas costosas que saturen la base de datos. El endpoint unificado `POST /api/v1/query` (backend/app/api/query.py) enruta automáticamente entre el pipeline RAG y el agente SQL según patrones de la consulta (“cuántos”, “total de”, “promedio de” frente a búsqueda documental), de forma transparente para el usuario.

Apéndice C

Integraciones, seguridad y observabilidad en detalle

Este anexo amplía las integraciones externas, las capas de seguridad y el stack de observabilidad resumidos en el Capítulo 6.

C.1 Integración con Microsoft SharePoint

El módulo Indexer funciona como un servicio autónomo (*worker*) que ejecuta un ciclo de sincronización periódica cada 5 minutos.

Autenticación MSAL. La conexión con SharePoint se realiza mediante OAuth 2.0 con flujo de credenciales de cliente (*Client Credentials Flow*). El servicio obtiene un token de acceso del *tenant* de Azure AD mediante la librería MSAL y utiliza la API REST de Microsoft Graph [51] para listar y descargar ficheros.

Sincronización delta incremental. Para evitar reprocesar documentos ya ingeridos, el Indexer consulta la marca temporal de modificación de cada fichero en SharePoint y la compara con los registros almacenados en PostgreSQL. Solo los ficheros nuevos o modificados se descargan, procesan e indexan. Esta estrategia reduce drásticamente el consumo de red y GPU en ciclos de sincronización recurrentes.

C.2 Resolución Semántica de Leyes (BOE)

Para consultas como “Resume la Ley de Protección de Datos”, el sistema implementa un mecanismo que mapea nombres coloquiales a identificadores oficiales del BOE:

```
1 LAW_MAPPINGS = {
```

```

2     "proteccion de datos": "BOE-A-2018-16673",
3     "constitucion": "BOE-A-1978-31229",
4     "propiedad intelectual": "BOE-A-1996-8930",
5     "igualdad": "BOE-A-2007-6115",
6 }

```

Listing C.1: Mapeo semántico de nombres de leyes.

El resolutor elimina palabras vacías comunes del español (“la”, “de”, “del”, “ley”) y busca coincidencias parciales en el diccionario antes de acudir al endpoint de búsqueda general del BOE.

En el alcance actual, el servidor MCP se ha validado en ejecución por terminal con un LLM y un cliente MCP compatible, pero la funcionalidad BOE *dentro de la interfaz web* se resuelve mediante llamadas a la API REST del backend desde el pipeline JARVIS, no por MCP. Conviene explicar por qué, ya que es una fuente habitual de confusión.

Por qué el flujo web no pasa por MCP. MCP es un protocolo cliente-servidor a nivel de aplicación: el cliente reside en la herramienta (Claude Desktop, OpenWebUI), no en el motor de inferencia. Ollama, por tanto, no «usa» MCP; se limita a servir el modelo. Para que OpenWebUI invocara las herramientas del servidor BOE por MCP tendrían que cumplirse dos condiciones que el diseño actual no satisface:

- **Un modelo con *function calling*.** El *tool-calling* por MCP requiere que sea el propio LLM quien decida invocar la herramienta y construya sus argumentos. El modelo principal (**rag-qwen-ft**, alias JARVIS) está ajustado para responder en modo RAG con citas y se expone en LiteLLM con `supports_function_calling: false`; no emite llamadas a herramientas de forma fiable.
- **Que el modelo conduzca la conversación.** El *tool-calling* de OpenWebUI opera en el diálogo modelo↔interfaz, pero en este sistema el método `pipe()` del pipeline JARVIS intercepta cada mensaje *antes* y enruta por su cuenta (detección de intención por expresiones regulares). Como el pipeline ya decide todo, la capa de herramientas MCP de OpenWebUI nunca llega a activarse.

A esto se añade la cronología: el soporte MCP nativo de OpenWebUI maduró después de construir el pipeline, por lo que la vía integrada y validada fue API-vía-pipeline, que además da control directo sobre autenticación, filtrado por *tenant*, métricas y formato de citas.

¿Merece la pena habilitar MCP de forma activa? Para el caso concreto del BOE —un dominio acotado, con siete herramientas y la resolución semántica ya resuelta en el pipeline— la integración por API es suficiente y más controlada; activar MCP no aportaría una mejora proporcional a su coste: obligaría a adoptar un modelo con *function calling* fiable, a reimplementar en la capa MCP las garantías de acceso y trazabilidad que hoy ofrece el pipeline, y a asumir mayor latencia y el riesgo de que el modelo elija o parametrize mal la herramienta (frente al determinismo del enrutamiento por reglas). El MCP activo sí compensaría en otro escenario: cuando se quieran incorporar *muchas* herramientas o fuentes nuevas, interoperar con clientes MCP externos, o adoptar un modelo con *tool-calling* robusto como núcleo del sistema; en ese caso MCP reduce el acoplamiento y escala mejor que seguir añadiendo ramas al pipeline. El camino intermedio más razonable es mantener el pipeline como orquestador pero hacer que sea *él* quien actúe como cliente MCP de las herramientas nuevas, combinando el estándar con el control actual.

C.3 Validación JWT en el Backend

Además de la autenticación perimetral de OpenWebUI vía OIDC, el Backend implementa una segunda capa de validación directa de tokens JWT para clientes externos y herramientas MCP que llaman a los endpoints sin pasar por la interfaz web. La lógica, implementada en `backend/app/core/auth.py`, sigue el flujo estándar de validación de tokens Bearer de Azure AD:

1. El backend descarga dinámicamente el conjunto de claves públicas del *tenant* corporativo desde el endpoint JWKS de Azure AD.
2. Las claves se cachean en memoria para evitar latencia en peticiones sucesivas.
3. Cada token JWT recibido en la cabecera `Authorization: Bearer` se verifica criptográficamente contra esas claves.
4. Los grupos del usuario extraídos del token se mapean a colecciones de Qdrant autorizadas.
5. Si el token no es válido o el usuario carece de acceso a alguna colección, el backend devuelve `HTTP 403 Forbidden` antes de ejecutar ninguna búsqueda.

Esta funcionalidad se activa con la variable de entorno `AZURE_JWT_VALIDATION=true` y es completamente transparente para los clientes que ya utilizan el flujo SSO de OpenWebUI.

Aislamiento de red. Toda la malla de servicios convive dentro de un plano interno `rag-network`, una red Docker de tipo *bridge*. Servicios como PostgreSQL (5432), Redis (6379), Qdrant (6333) y Ollama (11434) no son accesibles desde fuera del servidor bajo ninguna circunstancia, eliminando vectores de ataque sobre estas bases de datos.

C.4 Métricas, telemetría GPU y dashboards

Métricas personalizadas en el backend. El backend FastAPI expone métricas en formato Prometheus a través del endpoint `/metrics`: contador de peticiones HTTP (por endpoint, método y código), histograma de latencia (para calcular p50, p95, p99), distribución de modos de operación (RAG, chat, web, scraping, OCR, BOE) y tasa de errores por tipo.

Telemetría GPU con NVIDIA DCGM Exporter. Captura consumo de VRAM, utilización de CUDA cores, temperatura del chip y potencia consumida. Estas métricas resultaron críticas para determinar qué combinaciones de modelos podían coexistir en la VRAM disponible y cuándo recurrir a cuantizaciones más agresivas (Q4 en lugar de Q8).

Dashboards en Grafana. Se diseñaron tres dashboards interactivos:

1. Dashboard de Backend: latencia media y percentiles, throughput, tasa de errores, distribución de modos y tendencia de uso.
2. Dashboard de GPU: consumo de VRAM con alarmas de umbral (80 %), temperatura, utilización de cores y potencia.
3. Dashboard de Base de Datos: métricas de PostgreSQL (conexiones, queries/s) y de Qdrant (puntos indexados, latencia de búsqueda).

Logs centralizados: Loki y Promtail. Junto a las métricas, el sistema agrega los logs de todos los contenedores con **Promtail**, que los recolecta y los envía a **Loki**. Loki los indexa y los hace consultables desde el mismo Grafana, de modo que métricas y logs conviven en una única herramienta. Esto permite correlacionar un pico de latencia detectado en Prometheus con las líneas de log exactas de ese instante, acelerando el diagnóstico. La configuración reside en `config/promtail/config.yaml` y en el datasource `config/grafana/provisioning/datasources/loki.yml`.

Impacto de la monitorización. El stack de observabilidad condujo a decisiones concretas: se validó que $\sim 35\%$ de las consultas repetidas se beneficiaban de la caché Redis; se detectó que las consultas RAG con más de 5 fragmentos no mejoraban la calidad pero aumentaban la latencia (optimizando k); y se observó que PaddleOCR monopolizaba la GPU durante la ingesta, lo que motivó programarla en horarios de baja actividad.

Apéndice D

Fine-tuning con LoRA: procedimiento

Este anexo detalla el proceso de ajuste fino del modelo principal, resumido en el Capítulo 6.

D.1 Generación del Dataset de Entrenamiento

Para adaptar el modelo base al dominio corporativo, se implementó un pipeline de generación automática de datos de entrenamiento. Un script extrae pares pregunta-respuesta directamente de los fragmentos indexados en Qdrant, utilizando un LLM base local como generador sintético de consultas plausibles.

D.2 Entrenamiento con Low-Rank Adaptation

El ajuste fino se realizó mediante LoRA, que inserta matrices de rango reducido ($r = 8$) en las capas de atención del transformador sin modificar los pesos originales:

$$W' = W + \Delta W = W + BA \quad (\text{D.1})$$

donde $A \in \mathbb{R}^{r \times d}$ y $B \in \mathbb{R}^{k \times r}$ con $r \ll \min(d, k)$. Esto reduce el número de parámetros entrenables de millones a miles, haciendo viable el ajuste fino con una única GPU de consumo.

D.3 Resultado y Evaluación

El modelo resultante, `rag-qwen-ft:latest`, se registró en LiteLLM como el modelo primario para consultas RAG corporativas. En la versión académica del TFG se expone bajo el alias **JARVIS**; en el sistema empresarial derivado, desplegado en

piloto desde diciembre de 2025, bajo el alias **europav-IA**. Ambos sistemas comparten el mismo modelo base y el mismo adaptador LoRA, lo que valida la portabilidad del enfoque de ajuste fino sobre documentación corporativa real.

No obstante, se observó empíricamente que para tareas de RAG puras el *fine-tuning* del LLM resultaba menos eficaz que un buen ajuste del *prompt* combinado con un modelo de embeddings bien adaptado al dominio. La estrategia final del sistema descansa más sobre la calidad del contexto recuperado (chunking, embeddings, búsqueda híbrida, *prompt* restrictivo) que sobre la modificación de los pesos del modelo generativo. El *fine-tuning* sí aportó valor en la coherencia terminológica y en la adherencia al estilo de respuesta corporativo.

Apéndice E

Evolución de modelos y diagrama tecnológico

Este anexo recoge la trayectoria completa de selección del LLM principal y el diagrama general de tecnologías, ambos resumidos en el Capítulo 3.

E.1 Evolución del modelo principal: trayectoria de cuatro etapas

El sistema no llegó al modelo de producción actual en un único paso. La elección del LLM evolucionó a lo largo del proyecto en cuatro etapas, cada una motivada por limitaciones reales detectadas durante el uso en el entorno corporativo.

Tabla E.1: Trayectoria completa de LLMs empleados en el proyecto, desde el prototipo hasta producción.

Etapas	Modelo	Período	Contexto	VRAM	Razón del cambio
1	llama3.1:8b-instruct-q4_0	Oct 2025 (Sprint 1)	4K	~5 GB	Calidad de respuesta insuficiente (Q4)
2	llama3.1:8b-instruct-q8_0	Nov–Dic 2025	4K	~8 GB	Contexto 4K y español limitados; generación repetitiva
3	Qwen 2.5 7B (base)	Dic 2025 (evaluación)	32K	~8 GB	<i>Model loop</i> : tokens de parada incompatibles
4	rag-qwen-ft:latest (LoRA FT)	Dic 2025 – actualidad	32K	~8 GB	Modelo en producción (TFG y sistema empresarial)

Etapas 1 — Llama 3.1 8B Q4 (prototipo rápido, octubre 2025). La cuantización Q4 (4 bits) reducía el consumo de VRAM a ~5 GB, lo que facilitaba pruebas ágiles sobre el hardware disponible. Sin embargo, la calidad de las respuestas en español era notablemente inferior a la de variantes de mayor precisión, y las respuestas largas perdían coherencia con frecuencia.

Etapas 2 — Llama 3.1 8B Q8 (prototipo de mayor calidad, noviembre—diciembre 2025). La migración a la variante Q8 (8 bits) mejoró la precisión de

forma apreciable. Durante las primeras semanas de uso real en el entorno corporativo se identificaron dos limitaciones estructurales: la ventana de contexto de 4 K tokens era insuficiente para consultas RAG con múltiples fragmentos documentales —los fragmentos más alejados se truncaban antes de que el modelo pudiera considerarlos— y el rendimiento en español continuaba siendo inferior al esperado para un asistente empresarial hispanohablante.

Etapas 3 — Qwen 2.5 7B base (evaluación, diciembre 2025). La familia Qwen 2.5, desarrollada por Alibaba Cloud, ofrecía ventana de contexto de 32 K tokens y mejor rendimiento en español. Sin embargo, durante la integración se detectó un problema técnico crítico: el modelo generaba respuestas sin fin (*model loop*). La causa era una incompatibilidad en los *tokens de parada*: mientras Llama 3.1 utiliza `</s>` para indicar el fin de la respuesta, el formato ChatML de Qwen 2.5 emplea `<|im_end|>`. Fue necesario reconstruir el Modelfile de Ollama declarando explícitamente los stop tokens correctos antes de poder usar el modelo de forma fiable.

Etapas 4 — rag-qwen-ft:latest con LoRA (producción, diciembre 2025 – actualidad). Resuelto el problema de los stop tokens, se realizó el ajuste fino con LoRA sobre la documentación corporativa del proyecto. El adaptador resultante, `rag-qwen-ft:latest`, concentra el vocabulario interno, la terminología procedimental y el estilo de respuesta esperado en el entorno de destino. Este modelo es el que está en producción en los dos sistemas derivados del TFG: en la versión académica bajo el alias **JARVIS** y en el sistema empresarial desplegado en piloto bajo el alias **europav-IA** [12], [23].

E.2 Diagrama de flujo de tecnologías usadas

La Figura E.1 resume cómo se conectan las tecnologías principales del sistema. No representa únicamente el recorrido de una consulta, sino también los componentes de soporte que permiten que el sistema funcione en un entorno real: autenticación, almacenamiento, caché y monitorización.

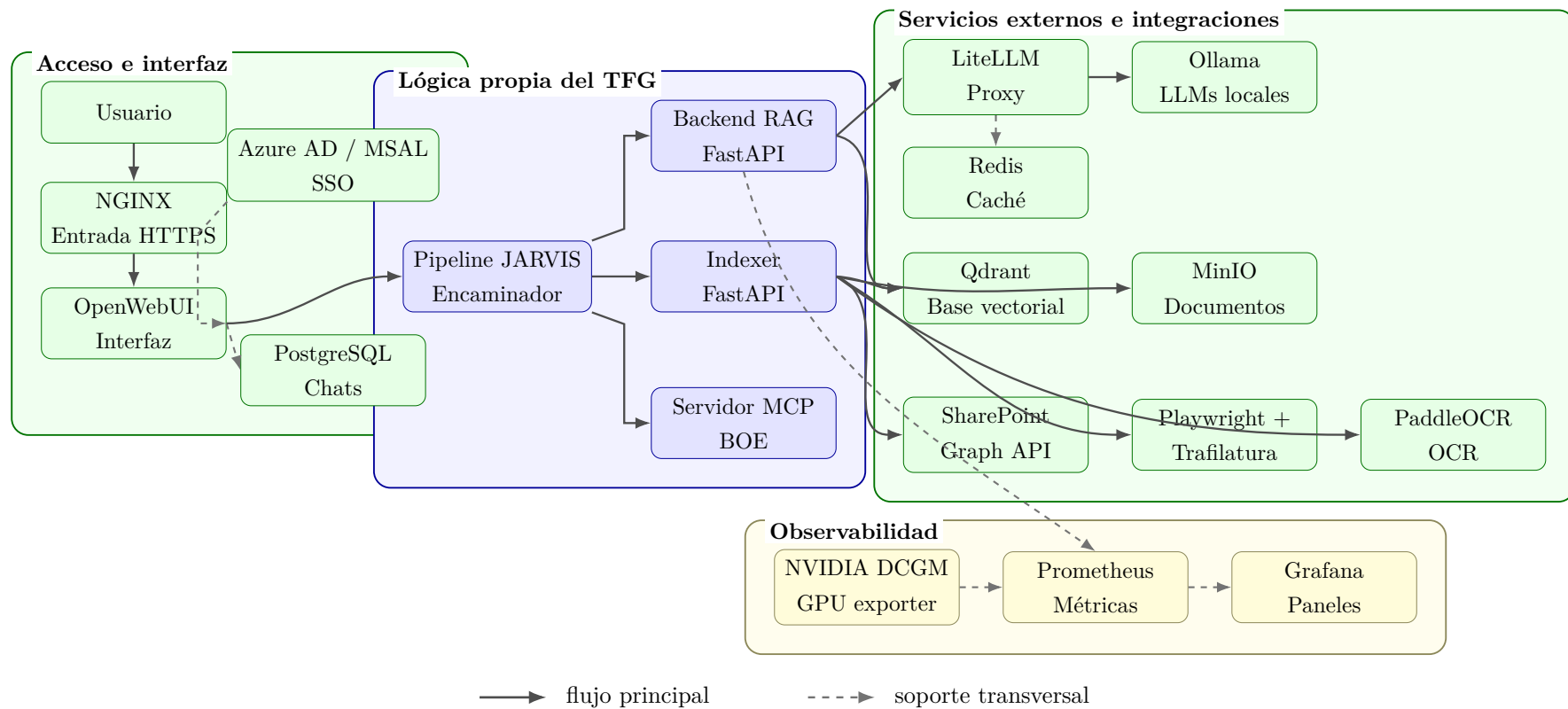


Figura E.1: Diagrama general de las tecnologías utilizadas en el sistema. La disposición apaisada separa el flujo de consulta, la lógica propia del TFG, las integraciones externas y la capa de observabilidad en bloques legibles.

Apéndice F

Contenido del repositorio entregado

Junto con esta memoria se entrega el código fuente completo del proyecto en un fichero comprimido (TFG-RAG-URJC-codigo.zip). Este anexo documenta qué contiene ese archivo, qué se ha excluido deliberadamente y por qué, de modo que quien lo reciba pueda reproducir el sistema con garantías. El mismo contenido está disponible de forma pública en el repositorio del proyecto¹.

F.1 Estructura del archivo

El archivo reproduce la estructura del repositorio versionado (unos 240 ficheros). Los directorios principales son los siguientes:

```
TFG-RAG-URJC/
|-- backend/          <- Servicio FastAPI: RAG, SQL,
    QueryProcessor,
    |
    |                 integraciones, procesamiento y
    |                 almacenamiento
|-- services/         <- Ollama, LiteLLM, Indexer y OpenWebUI (
    pipeline)
|-- mcp-boe-server/   <- Servidor MCP del BOE
|-- scripts/          <- Fine-tuning LoRA, generacion de datasets
    , backups
|-- config/           <- NGINX, Prometheus, Grafana, Promtail,
    plantillas
|-- database/         <- Esquema inicial de PostgreSQL (init.sql)
|-- tests/            <- Pruebas del sistema
|-- docs/             <- Documentacion tecnica (arquitectura,
    despliegue)
```

¹<https://github.com/acidxlemons/TFG-RAG-URJC>

```
|-- github_pages/      <- Pagina web del proyecto y material de
    demo
|-- memoria/          <- Fuentes LaTeX de esta memoria y PDF
    compilado
|-- docker-compose.yml <- Orquestacion de los servicios
|-- .env.example       <- Plantilla de configuracion (sin secretos
    )
|-- README.md         <- Guia de despliegue y puesta en marcha
'-- LICENSE           <- Licencia MIT del software
```

Listing F.1: Contenido principal del archivo entregado.

F.2 Elementos excluidos y su justificación

Para que el archivo sea seguro de compartir y de un tamaño razonable, se han excluido de forma intencionada los siguientes elementos, siguiendo las reglas del fichero `.gitignore` del proyecto:

- **Secretos y credenciales:** el fichero `.env` real y cualquier configuración con identificadores de Azure (`config/sharepoint_sites.json`). Se entrega únicamente la plantilla `.env.example`, sin valores sensibles.
- **Datos locales y volúmenes:** los directorios de datos de las bases de datos (`postgres_data/`, `qdrant_data/`), los documentos indexados (`data/`) y las cachés de OCR y modelos. Son datos de ejecución, no código, y pueden contener información corporativa.
- **Pesos de modelos:** los modelos de lenguaje se descargan mediante Ollama durante el despliegue (véase el `README.md`); no se incluyen por tamaño y por licencia propia de cada modelo.
- **Historial de Git y artefactos de compilación:** el directorio `.git`, entornos virtuales, `node_modules` y ficheros `__pycache__`.

F.3 Verificación del contenido

Antes de la entrega se comprobó que el archivo: (i) **no contiene ningún secreto** —se verificó la ausencia de `.env` y de configuraciones con credenciales—; (ii) incluye todo el **código fuente** de los servicios propios y su configuración de despliegue; (iii) incorpora el `README.md` con las instrucciones necesarias para levantar el sistema con `docker compose up`; y (iv) se corresponde exactamente con la versión publicada del repositorio. El software se distribuye bajo licencia MIT y la presente memoria bajo

licencia Creative Commons BY-SA 4.0, tal como se indica en la página de licencia inicial.

Bibliografía

- [1] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, págs. 9459-9474, 2020. dirección: <https://arxiv.org/abs/2005.11401>
- [2] Microsoft y LinkedIn, *2024 Work Trend Index: AI at Work Is Here. Now Comes the Hard Part*, <https://www.microsoft.com/en-us/worklab/work-trend-index/ai-at-work-is-here-now-comes-the-hard-part>, Accedido: febrero 2026, 2024.
- [3] Cyberhaven, *Shadow AI: The Growing Risk of Unauthorized AI Use in the Enterprise*, <https://www.cyberhaven.com/blog/shadow-ai>, Análisis sobre 1,6 millones de trabajadores. Accedido: febrero 2026, 2024.
- [4] Kalisa AI, *Shadow AI: The Unseen Risk Lurking in Your Organization*, <https://kalisa.ai/shadow-ai/>, Datos corporativos en herramientas IA crecieron 485 % entre 2023–2024. Accedido: febrero 2026, 2024.
- [5] Komprise, *2025 IT Survey: 44 % of Organizations Report Sensitive Data Leakage into AI*, <https://www.komprise.com/resource/2025-it-survey/>, Accedido: febrero 2026, 2025.
- [6] Anthropic, *Model Context Protocol Specification*, <https://modelcontextprotocol.io/>, Accedido: febrero 2026, 2024.
- [7] A. Vaswani et al., «Attention Is All You Need,» *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. dirección: <https://arxiv.org/abs/1706.03762>
- [8] Meta AI, *Llama 3.1 — Model Card and License*, <https://huggingface.co/meta-llama/Llama-3.1-8B>, Accedido: febrero 2026, 2024.
- [9] Google DeepMind, *Gemma Terms of Use*, <https://ai.google.dev/gemma/terms>, Accedido: febrero 2026, 2024.
- [10] Alibaba Cloud, *Qwen2.5 License Agreement*, <https://huggingface.co/Qwen/Qwen2.5-7B/blob/main/LICENSE>, Accedido: febrero 2026, 2024.

- [11] Meta AI, *Introducing Llama 3.1: Our most capable models to date*, <https://ai.meta.com/blog/meta-llama-3-1/>, Accedido: febrero 2026, 2024.
- [12] Qwen Team, *Qwen2.5: A Party of Foundation Models*, <https://qwenlm.github.io/blog/qwen2.5/>, Accedido: febrero 2026, 2024.
- [13] S. Robertson y H. Zaragoza, «The Probabilistic Relevance Framework: BM25 and Beyond,» *Foundations and Trends in Information Retrieval*, vol. 3, n.º 4, págs. 333-389, 2009. DOI: [10.1561/15000000019](https://doi.org/10.1561/15000000019) dirección: <https://doi.org/10.1561/15000000019>
- [14] Weaviate, *Hybrid Search — Weaviate Documentation*, <https://weaviate.io/developers/weaviate/search/hybrid>, Accedido: febrero 2026, 2024.
- [15] Qdrant Team, *Hybrid Search with Qdrant*, <https://qdrant.tech/articles/hybrid-search/>, Accedido: febrero 2026, 2024.
- [16] N. Reimers e I. Gurevych, «Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,» *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019. dirección: <https://arxiv.org/abs/1908.10084>
- [17] N. Reimers e I. Gurevych, *SentenceTransformers Documentation*, <https://www.sbert.net/>, Accedido: febrero 2026, 2024.
- [18] Y. A. Malkov y D. A. Yashunin, «Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,» *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, n.º 4, págs. 824-836, 2020. DOI: [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473) dirección: <https://doi.org/10.1109/TPAMI.2018.2889473>
- [19] Qdrant Team, *Sparse Vectors — Qdrant Documentation*, <https://qdrant.tech/documentation/concepts/vectors/#sparse-vectors>, Accedido: febrero 2026, 2024.
- [20] G. Gerganov, *GGUF Specification*, <https://github.com/ggerganov/ggml/blob/master/docs/gguf.md>, Accedido: febrero 2026, 2024.
- [21] Hugging Face, *GGUF — Hugging Face Documentation*, <https://huggingface.co/docs/hub/gguf>, Accedido: febrero 2026, 2024.
- [22] Ollama Team, *Import a GGUF model into Ollama*, <https://github.com/ollama/ollama/blob/main/docs/import.md>, Accedido: febrero 2026, 2024.
- [23] E. J. Hu et al., «LoRA: Low-Rank Adaptation of Large Language Models,» *arXiv preprint arXiv:2106.09685*, 2021. dirección: <https://arxiv.org/abs/2106.09685>

- [24] Anthropic, *Introducing the Model Context Protocol*, <https://www.anthropic.com/news/model-context-protocol>, Accedido: febrero 2026, 2024.
- [25] Open WebUI, *Model Context Protocol (MCP)*, <https://docs.openwebui.com/features/mcp/>, Accedido: 13 de abril de 2026, 2026.
- [26] Open WebUI Community, *Open WebUI — User-Friendly AI Interface*, <https://docs.openwebui.com/>, Accedido: febrero 2026, 2024.
- [27] Ollama Team, *Ollama — Run Large Language Models Locally*, <https://ollama.com/>, Accedido: febrero 2026, 2024.
- [28] Qdrant Team, *Qdrant — Vector Search Engine Documentation*, <https://qdrant.tech/documentation/>, Accedido: febrero 2026, 2024.
- [29] S. Ramírez, *FastAPI — Modern, Fast Web Framework for Python*, <https://fastapi.tiangolo.com/>, Accedido: febrero 2026, 2024.
- [30] BerriAI, *LiteLLM — Call All LLM APIs using the OpenAI Format*, <https://docs.litellm.ai/>, Accedido: febrero 2026, 2024.
- [31] Docker, *Compose file reference*, <https://docs.docker.com/compose/compose-file/>, Accedido: 9 de marzo de 2026, 2026.
- [32] PaddlePaddle, *PaddleOCR: Awesome multilingual OCR toolkits*, <https://github.com/PaddlePaddle/PaddleOCR>, Accedido: febrero 2026, 2024.
- [33] Microsoft, *Playwright — Web Testing and Automation*, <https://playwright.dev/>, Accedido: febrero 2026, 2024.
- [34] A. Barbaresi, «Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction,» en *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, 2021, págs. 122-131. dirección: <https://aclanthology.org/2021.acl-demo.15>
- [35] Prometheus Authors, *Prometheus Documentation Overview*, <https://prometheus.io/docs/>, Accedido: 9 de marzo de 2026, 2026.
- [36] Grafana Labs, *About Grafana*, <https://grafana.com/docs/grafana/latest/introduction/>, Accedido: 9 de marzo de 2026, 2026.
- [37] NVIDIA, *NVIDIA DCGM Exporter*, <https://github.com/NVIDIA/dcgm-exporter>, Accedido: 9 de marzo de 2026, 2026.
- [38] OpenAI, *ChatGPT Enterprise*, <https://openai.com/chatgpt/enterprise>, Accedido: 9 de marzo de 2026, 2026.

- [39] Microsoft, *Microsoft 365 Copilot*, <https://www.microsoft.com/en-us/microsoft-365-copilot>, Accedido: 9 de marzo de 2026, 2026.
- [40] Google Workspace, *Announcing Gemini for Google Workspace*, <https://workspace.google.com/blog/product-announcements/gemini-for-google-workspace>, Accedido: 9 de marzo de 2026, 2024.
- [41] Google Cloud, *Generative AI on Vertex AI Documentation*, <https://docs.cloud.google.com/vertex-ai/generative-ai/docs>, Accedido: 9 de marzo de 2026, 2025.
- [42] Amazon Web Services, *What is Amazon Bedrock?* <https://docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html>, Accedido: 9 de marzo de 2026, 2026.
- [43] PrivateGPT, *PrivateGPT Documentation Introduction*, <https://docs.privategpt.dev/overview/welcome/introduction>, Accedido: 9 de marzo de 2026, 2026.
- [44] Mintplex Labs, *AnythingLLM Documentation*, <https://docs.anythingllm.com/>, Accedido: 9 de marzo de 2026, 2026.
- [45] Onyx, *Onyx Documentation*, <https://docs.onyx.app/>, Accedido: 9 de marzo de 2026, 2026.
- [46] OpenClaw, *OpenClaw Documentation*, <https://docs.openclaw.ai/index>, Accedido: 9 de marzo de 2026, 2026.
- [47] Google, *NotebookLM adds Deep Research and support for more source types*, <https://blog.google/technology/google-labs/notebooklm-deep-research-file-types/>, Accedido: 9 de marzo de 2026, 2025.
- [48] LangChain, *Retrieval — LangChain Documentation*, <https://python.langchain.com/docs/concepts/retrieval/>, Accedido: febrero 2026, 2024.
- [49] LlamaIndex, *Building a RAG Pipeline — LlamaIndex*, <https://docs.llamaindex.ai/en/stable/understanding/rag/>, Accedido: febrero 2026, 2024.
- [50] Agencia Estatal Boletín Oficial del Estado, *API de datos abiertos del BOE*, <https://www.boe.es/datosabiertos/api/api.php>, Accedido: 9 de marzo de 2026, 2026.
- [51] Microsoft, *Microsoft Graph REST API v1.0 Reference*, <https://learn.microsoft.com/en-us/graph/api/overview>, Accedido: febrero 2026, 2024.

- [52] IBM Technology, *IBM Technology — Canal de YouTube*, <https://www.youtube.com/@IBMTechology>, Contenido divulgativo sobre IA generativa, RAG y arquitecturas de sistemas. Accedido: julio 2026, 2026.
- [53] C. Medin, *Cole Medin — Canal de YouTube*, <https://www.youtube.com/@ColeMedin>, Tutoriales prácticos sobre RAG, agentes y LLMs locales. Accedido: julio 2026, 2026.